

Annual Mean Global Surface Temperature Dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd
```

```
# Assuming your dataset is in a CSV file named 'your_dataset.csv'
# Replace 'your_dataset.csv' with the actual filename and path
df = pd.read_csv('/content/drive/My Drive/Project sem1/Datasets/Annual_Mean_Global_Surface_Temperature.csv')
```

```
# Now you can work with the dataframe 'df'
print(df.head())
```

```
ObjectID      Country ISO2  ISO3 \
0      1  Afghanistan, Islamic Rep. of  AF  AFG
1      2                Africa  NaN  AFRTMP
2      3                Albania  AL  ALB
3      4                Algeria  DZ  DZA
4      5      American Samoa  AS  ASM

Indicator      Unit \
0  Temperature change with respect to a baseline ...  Degree Celsius
1  Temperature change with respect to a baseline ...  Degree Celsius
2  Temperature change with respect to a baseline ...  Degree Celsius
3  Temperature change with respect to a baseline ...  Degree Celsius
4  Temperature change with respect to a baseline ...  Degree Celsius

Source CTS Code \
0  Food and Agriculture Organization of the Unite...  ECCS
1  Food and Agriculture Organization of the Unite...  ECCS
2  Food and Agriculture Organization of the Unite...  ECCS
3  Food and Agriculture Organization of the Unite...  ECCS
4  Food and Agriculture Organization of the Unite...  ECCS

CTS Name \
0  Surface Temperature Change
1  Surface Temperature Change
2  Surface Temperature Change
3  Surface Temperature Change
4  Surface Temperature Change

CTS Full Descriptor ...  2014  2015 \
0  Environment, Climate Change, Climate and Weath...  ...  0.521  1.204
1  Environment, Climate Change, Climate and Weath...  ...  1.013  1.190
2  Environment, Climate Change, Climate and Weath...  ...  1.285  1.667
3  Environment, Climate Change, Climate and Weath...  ...  1.713  1.153
4  Environment, Climate Change, Climate and Weath...  ...  0.840  0.679

2016  2017  2018  2019  2020  2021  2022  2023 \
0  1.612  1.642  1.624  0.991  0.587  1.475  2.154  1.956
1  1.392  1.180  1.178  1.297  1.200  1.396  0.996  1.485
2  1.558  1.196  2.103  1.751  1.567  1.589  1.585  2.122
3  1.787  1.533  1.231  1.107  1.905  2.360  1.776  2.274
4  1.209  1.105  0.859  1.209  1.101  0.938  0.925  0.951

[5 rows x 73 columns]
```

```
df = df.drop('ISO2', axis=1)
```

```
print(df.head())
```

```
ObjectID      Country  ISO3 \
0      1  Afghanistan, Islamic Rep. of  AFG
1      2                Africa  AFRTMP
2      3                Albania  ALB
3      4                Algeria  DZA
4      5      American Samoa  ASM

Indicator      Unit \
0  Temperature change with respect to a baseline ...  Degree Celsius
1  Temperature change with respect to a baseline ...  Degree Celsius
2  Temperature change with respect to a baseline ...  Degree Celsius
3  Temperature change with respect to a baseline ...  Degree Celsius
4  Temperature change with respect to a baseline ...  Degree Celsius

Source CTS Code \
0  Food and Agriculture Organization of the Unite...  ECCS
1  Food and Agriculture Organization of the Unite...  ECCS
2  Food and Agriculture Organization of the Unite...  ECCS
3  Food and Agriculture Organization of the Unite...  ECCS
4  Food and Agriculture Organization of the Unite...  ECCS

CTS Name \
```

```
0 Surface Temperature Change
1 Surface Temperature Change
2 Surface Temperature Change
3 Surface Temperature Change
4 Surface Temperature Change
```

```

      CTS Full Descriptor  1961  ...  2014  \
0 Environment, Climate Change, Climate and Weath... -0.126  ...  0.521
1 Environment, Climate Change, Climate and Weath... -0.017  ...  1.013
2 Environment, Climate Change, Climate and Weath...  0.635  ...  1.285
3 Environment, Climate Change, Climate and Weath...  0.155  ...  1.713
4 Environment, Climate Change, Climate and Weath...  0.121  ...  0.840
```

```

      2015  2016  2017  2018  2019  2020  2021  2022  2023
0  1.204  1.612  1.642  1.624  0.991  0.587  1.475  2.154  1.956
1  1.190  1.392  1.180  1.178  1.297  1.200  1.396  0.996  1.485
2  1.667  1.558  1.196  2.103  1.751  1.567  1.589  1.585  2.122
3  1.153  1.787  1.533  1.231  1.107  1.905  2.360  1.776  2.274
4  0.679  1.209  1.105  0.859  1.209  1.101  0.938  0.925  0.951
```

[5 rows x 72 columns]

df.dtypes

```

ObjectID    0
-----
ObjectID    int64
Country      object
ISO3         object
Indicator    object
Unit         object
...         ...
2019        float64
2020        float64
2021        float64
2022        float64
2023        float64
```

72 rows x 1 columns

dtype: object

```
# Find all unique countries in the DataFrame
unique_countries = df['Country'].unique()
```

```
# Print the unique countries
unique_countries
```

```

array(['Afghanistan, Islamic Rep. of', 'Africa', 'Albania', 'Algeria',
      'American Samoa', 'Americas', 'Andorra, Principality of', 'Angola',
      'Anguilla', 'Antarctica', 'Antigua and Barbuda', 'Argentina',
      'Armenia, Rep. of', 'Aruba, Kingdom of the Netherlands', 'Asia',
      'Australia', 'Austria', 'Azerbaijan, Rep. of', 'Bahamas, The',
      'Bahrain, Kingdom of', 'Bangladesh', 'Barbados',
      'Belarus, Rep. of', 'Belgium', 'Belize', 'Benin', 'Bhutan',
      'Bolivia', 'Bosnia and Herzegovina', 'Botswana', 'Brazil',
      'British Virgin Islands', 'Brunei Darussalam', 'Bulgaria',
      'Burkina Faso', 'Burundi', 'Cabo Verde', 'Cambodia', 'Cameroon',
      'Canada', 'Cayman Islands', 'Central African Rep.', 'Chad',
      'Chile', 'China, P.R.: Hong Kong', 'China, P.R.: Macao',
      'China, P.R.: Mainland', 'Colombia', 'Comoros, Union of the',
      'Congo, Dem. Rep. of the', 'Congo, Rep. of', 'Cook Islands',
      'Costa Rica', 'Côte d'Ivoire', 'Croatia, Rep. of', 'Cuba',
      'Cyprus', 'Czech Rep.', 'Denmark', 'Djibouti', 'Dominica',
      'Dominican Rep.', 'Ecuador', 'Egypt, Arab Rep. of', 'El Salvador',
      'Equatorial Guinea, Rep. of', 'Eritrea, The State of',
      'Estonia, Rep. of', 'Eswatini, Kingdom of',
      'Ethiopia, The Federal Dem. Rep. of', 'Europe',
      'Falkland Islands (Malvinas)', 'Faroe Islands', 'Fiji, Rep. of',
      'Finland', 'France', 'French Polynesia', 'Gabon', 'Gambia, The',
      'Georgia', 'Germany', 'Ghana', 'Gibraltar', 'Greece', 'Greenland',
      'Grenada', 'Guadeloupe', 'Guatemala', 'Guiana, French', 'Guinea',
      'Guinea-Bissau', 'Guyana', 'Haiti', 'Holy See', 'Honduras',
      'Hungary', 'Iceland', 'India', 'Indonesia',
      'Iran, Islamic Rep. of', 'Iraq', 'Ireland', 'Isle of Man',
      'Israel', 'Italy', 'Jamaica', 'Japan', 'Jordan',
      'Kazakhstan, Rep. of', 'Kenya', 'Kiribati',
      'Korea, Dem. People's Rep. of', 'Korea, Rep. of', 'Kuwait',
      'Kyrgyz Rep.', 'Lao People's Dem. Rep.', 'Latvia', 'Lebanon',
```

```
'Lesotho, Kingdom of', 'Liberia', 'Libya', 'Liechtenstein',
'Lithuania', 'Luxembourg', 'Madagascar, Rep. of', 'Malawi',
'Malaysia', 'Maldives', 'Mali', 'Malta',
'Marshall Islands, Rep. of the', 'Martinique',
'Mauritania, Islamic Rep. of', 'Mauritius', 'Mayotte', 'Mexico',
'Micronesia, Federated States of', 'Moldova, Rep. of', 'Monaco',
'Mongolia', 'Montenegro', 'Montserrat', 'Morocco',
'Mozambique, Rep. of', 'Myanmar', 'Namibia', 'Nauru, Rep. of',
'Nepal', 'Netherlands, The', 'New Caledonia', 'New Zealand',
'Nicaragua', 'Niger', 'Nigeria', 'Niue', 'Norfolk Island',
'North Macedonia, Republic of', 'Norway', 'Oceania', 'Oman',
'Pakistan', 'Palau, Rep. of', 'Panama', 'Papua New Guinea',
'Paraguay', 'Peru', 'Philippines', 'Pitcairn Islands',
'Poland, Rep. of', 'Portugal', 'Puerto Rico', 'Qatar', 'Réunion',
'Romania', 'Russian Federation', 'Rwanda', 'Saint Helena',
'Saint Pierre and Miquelon', 'Samoa', 'San Marino, Rep. of',
'São Tomé and Príncipe, Dem. Rep. of', 'Saudi Arabia', 'Senegal',
'Serbia, Rep. of', 'Seychelles', 'Sierra Leone', 'Singapore',
'Slovak Rep.', 'Slovenia, Rep. of', 'Solomon Islands', 'Somalia',
'South Africa', 'South Sudan, Rep. of', 'Spain', 'Sri Lanka',
'St. Kitts and Nevis', 'St. Lucia',
'St. Vincent and the Grenadines', 'Sudan', 'Suriname',
'Svalbard Jan Mayen Islands', 'Sweden', 'Switzerland',
'Syrian Arab Rep.', 'Taiwan Province of China',
'Tajikistan, Rep. of', 'Tanzania, United Rep. of', 'Thailand',
'Timor-Leste, Dem. Rep. of', 'Togo', 'Tokelau', 'Tonga',
'Trinidad and Tobago', 'Tunisia', 'Turkmenistan, Rep. of'
```

```
# Filter the DataFrame for rows where 'Country' is 'India'
india_data = df[df['Country'] == 'India']

# Find and print the null/missing values in the filtered DataFrame
print(india_data.isnull().sum())
print(india_data.isnull().sum().sum())
```

```
ObjectID      0
Country       0
ISO3          0
Indicator     0
Unit          0
..
2019          0
2020          0
2021          0
2022          0
2023          0
Length: 72, dtype: int64
0
```

```
import matplotlib.pyplot as plt

# Filter data for India
india_data = df[df['Country'] == 'India']
years = list(range(1961, 2024))
year_columns = [str(year) for year in years]

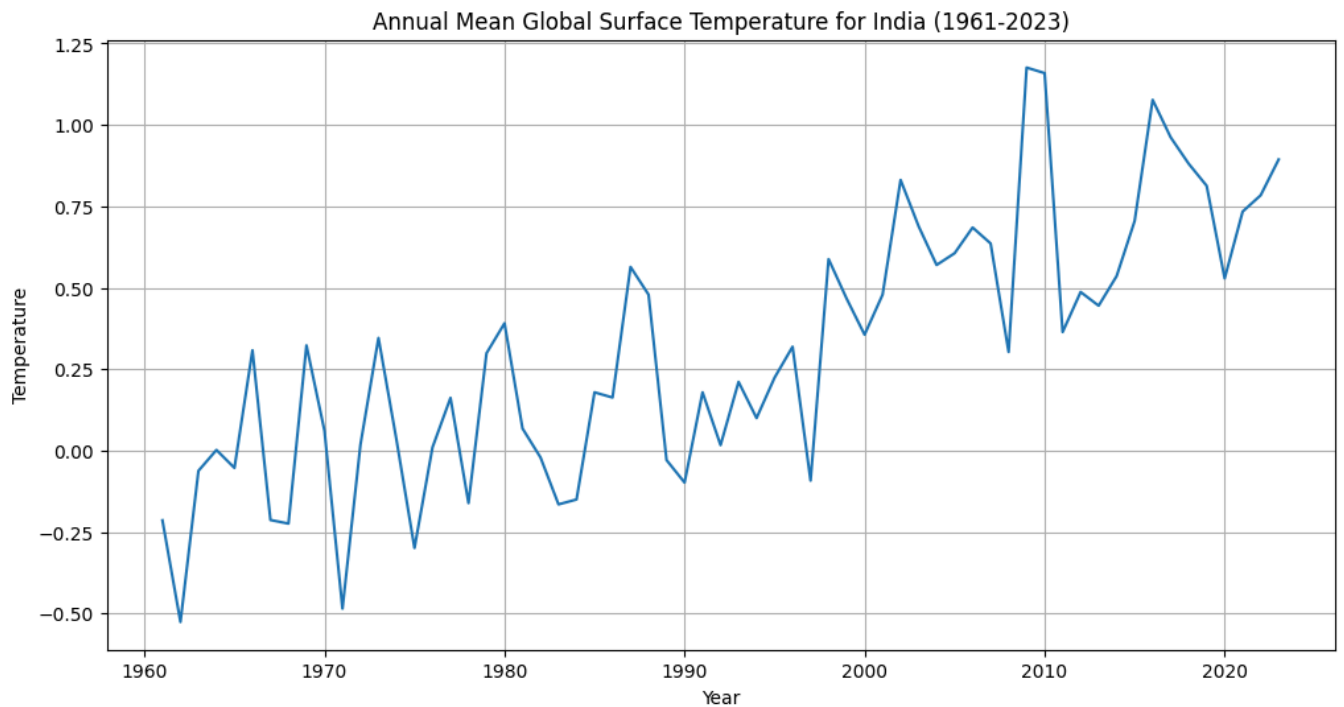
# Select temperatures for India for the specified years
temperatures = india_data.loc[india_data['Country'] == 'India', year_columns].values.flatten()
print(temperatures)

# Create the plot
plt.figure(figsize=(12, 6))
plt.plot(years, temperatures)
plt.xlabel('Year')
plt.ylabel('Temperature')
plt.title('Annual Mean Global Surface Temperature for India (1961-2023)')
plt.grid(True)
plt.show()
```

```

[-0.214 -0.526 -0.062  0.002 -0.053  0.308 -0.213 -0.224  0.323  0.061
-0.485  0.019  0.346  0.033 -0.299  0.01  0.162 -0.161  0.299  0.391
 0.068 -0.021 -0.165 -0.15  0.179  0.163  0.564  0.479 -0.029 -0.098
 0.179  0.017  0.211  0.1  0.224  0.319 -0.092  0.588  0.467  0.356
 0.479  0.831  0.689  0.57  0.606  0.685  0.636  0.303  1.176  1.159
 0.364  0.487  0.445  0.536  0.705  1.077  0.962  0.881  0.813  0.529
 0.734  0.784  0.894]

```



```

from sklearn.linear_model import LinearRegression

# Prepare the data for regression
X = [[year] for year in years]
y = temperatures

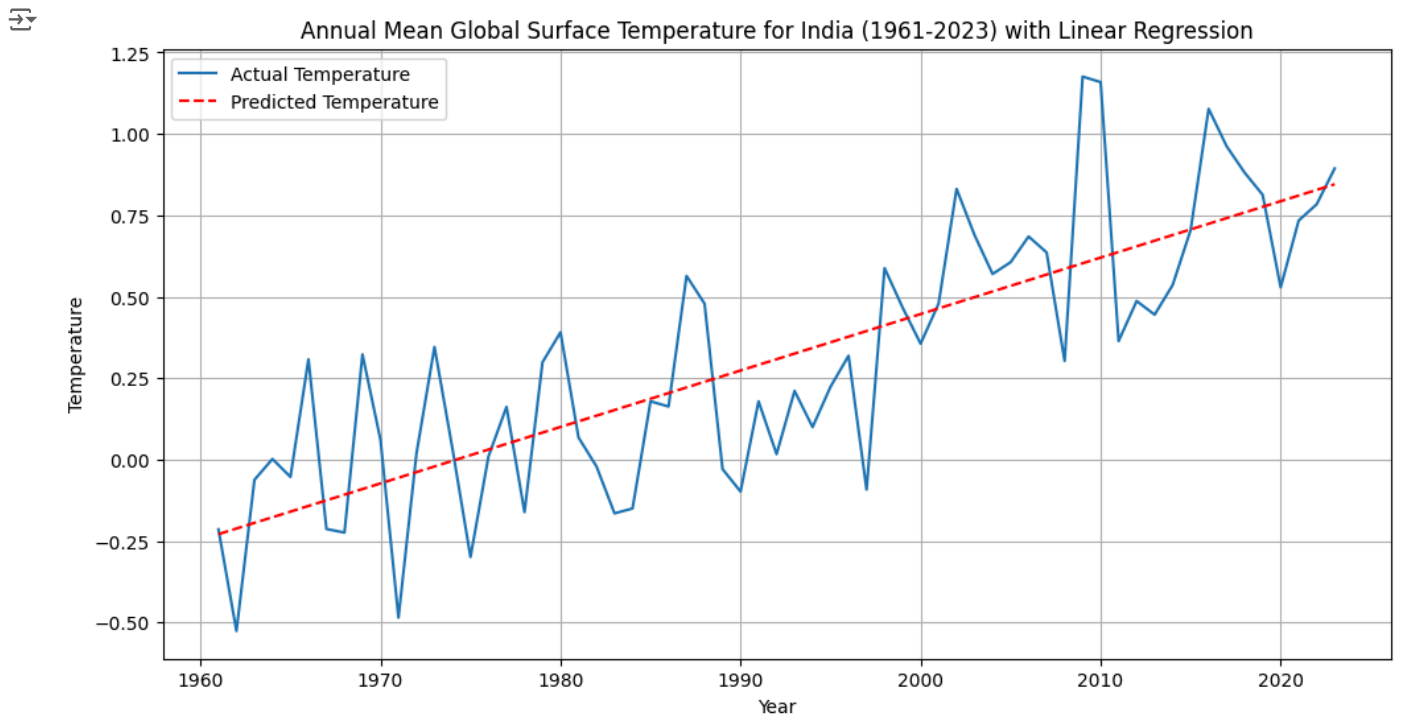
# Create and train the linear regression model
model = LinearRegression()
model.fit(X, y)

# Make predictions
y_pred = model.predict(X)

# Plot the actual and predicted temperatures
plt.figure(figsize=(12, 6))
plt.plot(years, temperatures, label='Actual Temperature')
plt.plot(years, y_pred, label='Predicted Temperature', linestyle='--', color='red')
plt.xlabel('Year')
plt.ylabel('Temperature')
plt.title('Annual Mean Global Surface Temperature for India (1961-2023) with Linear Regression')
plt.legend()
plt.grid(True)
plt.show()

# Print the model's coefficients
print(f"Intercept: {model.intercept_}")
print(f"Slope: {model.coef_[0]}")

```



Intercept: -34.21007232462878
Slope: 0.017328485023041476

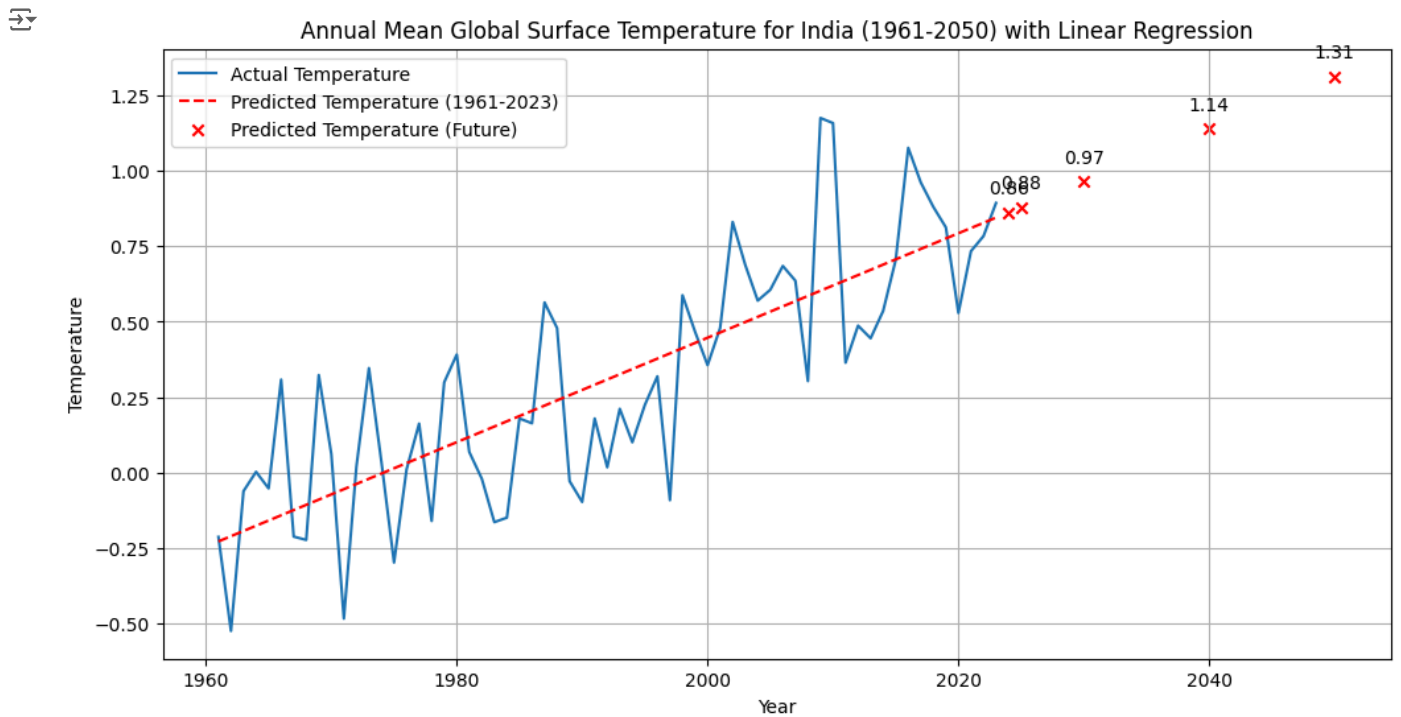
```
# Predict temperatures for future years
future_years = [2024, 2025, 2030, 2040, 2050]
future_X = [[year] for year in future_years]
future_y_pred = model.predict(future_X)

# Plot the actual and predicted temperatures along with future predictions
plt.figure(figsize=(12, 6))
plt.plot(years, temperatures, label='Actual Temperature')
plt.plot(years, y_pred, label='Predicted Temperature (1961-2023)', linestyle='--', color='red')
plt.scatter(future_years, future_y_pred, color='red', label='Predicted Temperature (Future)', marker='x')
plt.xlabel('Year')
plt.ylabel('Temperature')
plt.title('Annual Mean Global Surface Temperature for India (1961-2050) with Linear Regression')
plt.legend()
plt.grid(True)

# Annotate the predicted future temperatures on the plot
for i, txt in enumerate(future_y_pred):
    plt.annotate(f'{txt:.2f}', (future_years[i], future_y_pred[i]), textcoords="offset points", xytext=(0,10), ha='center')

plt.show()

# Print the predicted future temperatures
for year, temp in zip(future_years, future_y_pred):
    print(f"Predicted temperature for {year}: {temp:.2f}")
```



```
import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy import stats
```

```
# Calculate Mean Squared Error (MSE)
mse = mean_squared_error(y, y_pred)
print(f"Mean Squared Error (MSE): {mse}")
```

```
# Calculate Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)
print(f"Root Mean Squared Error (RMSE): {rmse}")
```

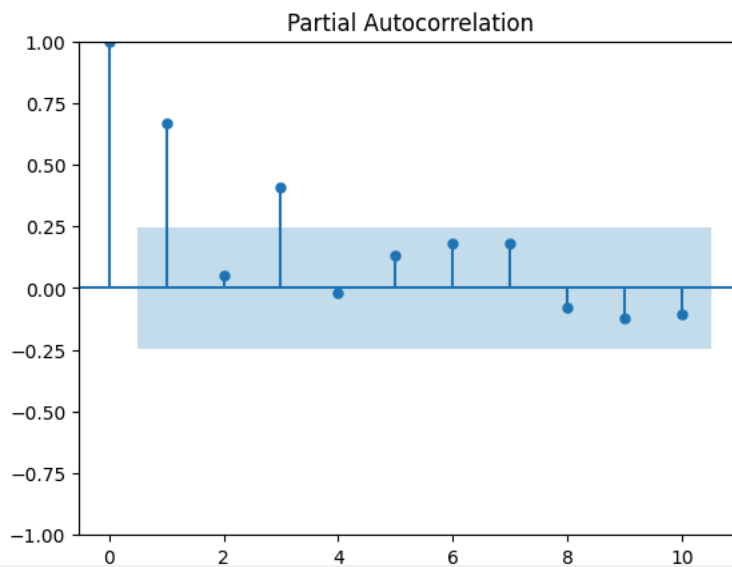
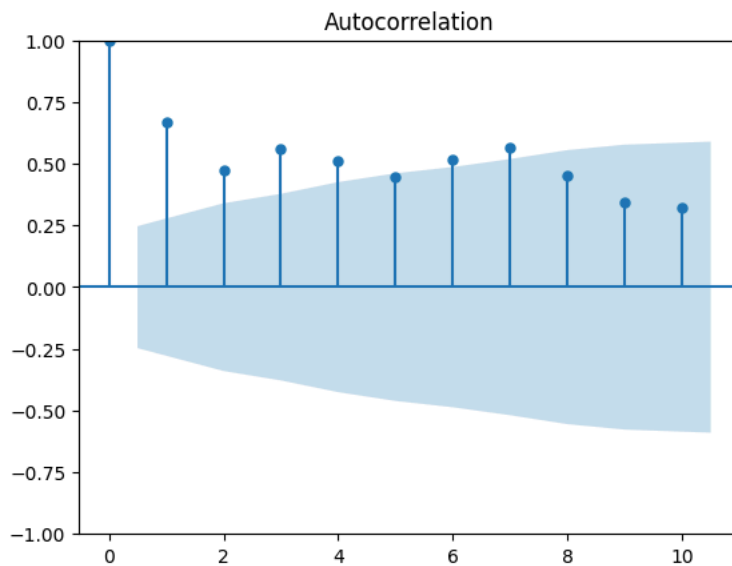
```
# Calculate Mean Absolute Error (MAE)
mae = mean_absolute_error(y, y_pred)
print(f"Mean Absolute Error (MAE): {mae}")
```

Mean Squared Error (MSE): 0.05746845380453541
Root Mean Squared Error (RMSE): 0.23972578877654238
Mean Absolute Error (MAE): 0.19309536559546114

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

```
# Plot ACF
plot_acf(temperatures, lags=10) # Adjust lags as needed
plt.show()
```

```
# Plot PACF
plot_pacf(temperatures, lags=10) # Adjust lags as needed
plt.show()
```



```
from statsmodels.tsa.stattools import adfuller

# Perform the ADF test
result = adfuller(temperatures, autolag='AIC')

print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))

# Interpret the results
if result[1] <= 0.05:
    print("Strong evidence against the null hypothesis")
    print("Reject the null hypothesis")
    print("Data is stationary")
else:
    print("Weak evidence against the null hypothesis")
    print("Fail to reject the null hypothesis")
    print("Data is non-stationary")
```



```
ADF Statistic: -0.333389
p-value: 0.920641
Critical Values:
1%: -3.553
5%: -2.915
10%: -2.595
Weak evidence against the null hypothesis
Fail to reject the null hypothesis
Data is non-stationary
```

```
best_d = None
best_p_value = 1 # Initialize with a high p-value
```

```

for d in range(4): # Try differencing up to 3 times (0 to 3)
    differenced_data = temperatures
    for _ in range(d):
        differenced_data = np.diff(differenced_data)
    if len(differenced_data) < 2:
        break # Stop if differencing results in too few data points

result = adfuller(differenced_data, autolag='AIC')
p_value = result[1]

if p_value < best_p_value:
    best_p_value = p_value
    best_d = d

print(f"\nBest d value found: {best_d} (p-value = {best_p_value:.4f})")

```



Best d value found: 1 (p-value = 0.0000)

```

!pip install pmdarima
from pmdarima import auto_arima

# Fit Auto-ARIMA
stepwise_fit = auto_arima(temperatures, trace=True, suppress_warnings=True)

# Print the best parameters and AIC
print(stepwise_fit.summary())

```



```

Collecting pmdarima
  Downloading pmdarima-2.0.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl.metadata (7.8 kB)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.4.2)
Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (3.0.1)
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.26.4)
Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.2)
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.5.2)
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.13.1)
Requirement already satisfied: statsmodels>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (0.14.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.3)
Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (75.1.0)
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (24.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.22->pmdarima) (3.3.0)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-packages (from statsmodels>=0.13.2->pmdarima) (1.0.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas>=0.19->pmdarima) (1.16.0)
Downloading pmdarima-2.0.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl (2.1 MB)
----- 2.1/2.1 MB 36.9 MB/s eta 0:00:00

Installing collected packages: pmdarima
Successfully installed pmdarima-2.0.4
Performing stepwise search to minimize aic
ARIMA(2,1,2)(0,0,0)[0] intercept : AIC=inf, Time=0.37 sec
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=34.433, Time=0.03 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AIC=32.350, Time=0.03 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : AIC=inf, Time=0.17 sec
ARIMA(0,1,0)(0,0,0)[0] : AIC=32.640, Time=0.01 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : AIC=15.014, Time=0.07 sec
ARIMA(3,1,0)(0,0,0)[0] intercept : AIC=16.852, Time=0.06 sec
ARIMA(2,1,1)(0,0,0)[0] intercept : AIC=inf, Time=0.27 sec
ARIMA(1,1,1)(0,0,0)[0] intercept : AIC=inf, Time=0.24 sec
ARIMA(3,1,1)(0,0,0)[0] intercept : AIC=inf, Time=0.38 sec
ARIMA(2,1,0)(0,0,0)[0] : AIC=14.030, Time=0.11 sec
ARIMA(1,1,0)(0,0,0)[0] : AIC=30.723, Time=0.02 sec
ARIMA(3,1,0)(0,0,0)[0] : AIC=15.942, Time=0.08 sec
ARIMA(2,1,1)(0,0,0)[0] : AIC=12.772, Time=0.32 sec
ARIMA(1,1,1)(0,0,0)[0] : AIC=16.582, Time=0.12 sec
ARIMA(3,1,1)(0,0,0)[0] : AIC=13.417, Time=0.76 sec
ARIMA(2,1,2)(0,0,0)[0] : AIC=inf, Time=2.16 sec
ARIMA(1,1,2)(0,0,0)[0] : AIC=inf, Time=0.72 sec
ARIMA(3,1,2)(0,0,0)[0] : AIC=inf, Time=1.45 sec

```

Best model: ARIMA(2,1,1)(0,0,0)[0]
Total fit time: 7.416 seconds

SARIMAX Results

```

=====
Dep. Variable:          y          No. Observations:          63
Model:                SARIMAX(2, 1, 1)    Log Likelihood        -2.386
Date:                 Sat, 14 Dec 2024    AIC                   12.772
Time:                 02:12:19           BIC                   21.281
Sample:              0                HQIC                   16.113
                    - 63
Covariance Type:      opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]

```

!pip install pmdarima

import pmdarima as pm
from pmdarima import model_selection
import numpy as np
import matplotlib.pyplot as plt

# Split the data into training and testing sets
train_data, test_data = model_selection.train_test_split(temperatures, train_size=0.8)

# Fit an ARIMA model
arima_model = pm.ARIMA(order=(2,1,1))
arima_model.fit(train_data)

# Make predictions
n_periods = len(test_data)
predictions = arima_model.predict(n_periods=n_periods)

# Plot the results
plt.figure(figsize=(12, 6))
plt.plot(years[:len(train_data)], train_data, label='Training Data')
plt.plot(years[len(train_data):], test_data, label='Actual Temperature (Test)')
plt.plot(years[len(train_data):], predictions, label='Predicted Temperature (ARIMA)', linestyle='--', color='red')

plt.xlabel('Year')
plt.ylabel('Temperature')
plt.title('Annual Mean Global Surface Temperature for India (ARIMA)')
plt.legend()
plt.grid(True)
plt.show()

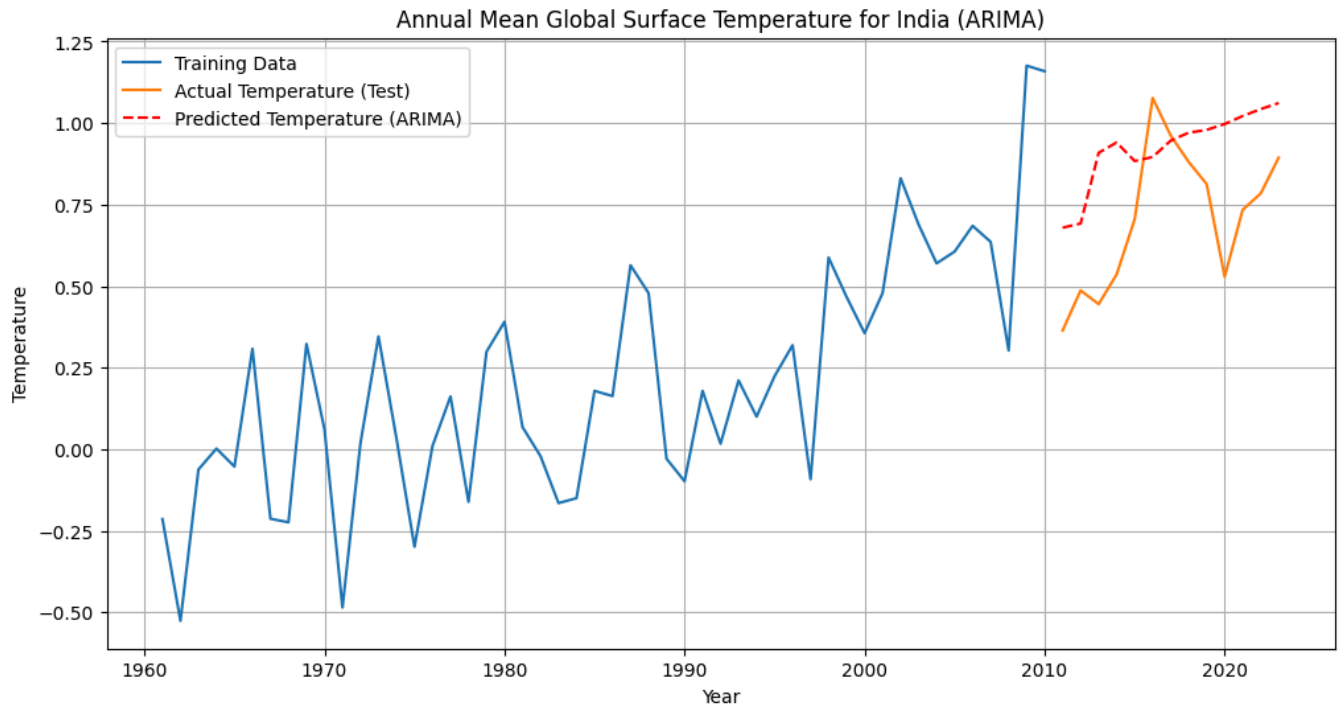
# Evaluate the model (example metrics)
from sklearn.metrics import mean_squared_error, mean_absolute_error

rmse = np.sqrt(mean_squared_error(test_data, predictions))
mae = mean_absolute_error(test_data, predictions)

print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"Mean Absolute Error (MAE): {mae}")

```

Requirement already satisfied: pmdarima in /usr/local/lib/python3.10/dist-packages (2.0.4)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.4.2)
Requirement already satisfied: Cython!=0.29.18,!0.29.31,>=0.29 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (3.0.11)
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.26.4)
Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.2)
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.5.2)
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.13.1)
Requirement already satisfied: statsmodels>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (0.14.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.3)
Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (75.1.0)
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (24.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2.8)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.22->pmdarima)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-packages (from statsmodels>=0.13.2->pmdarima) (1.0.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas>=0.19->pmdarima)



Root Mean Squared Error (RMSE): 0.27978891549344265

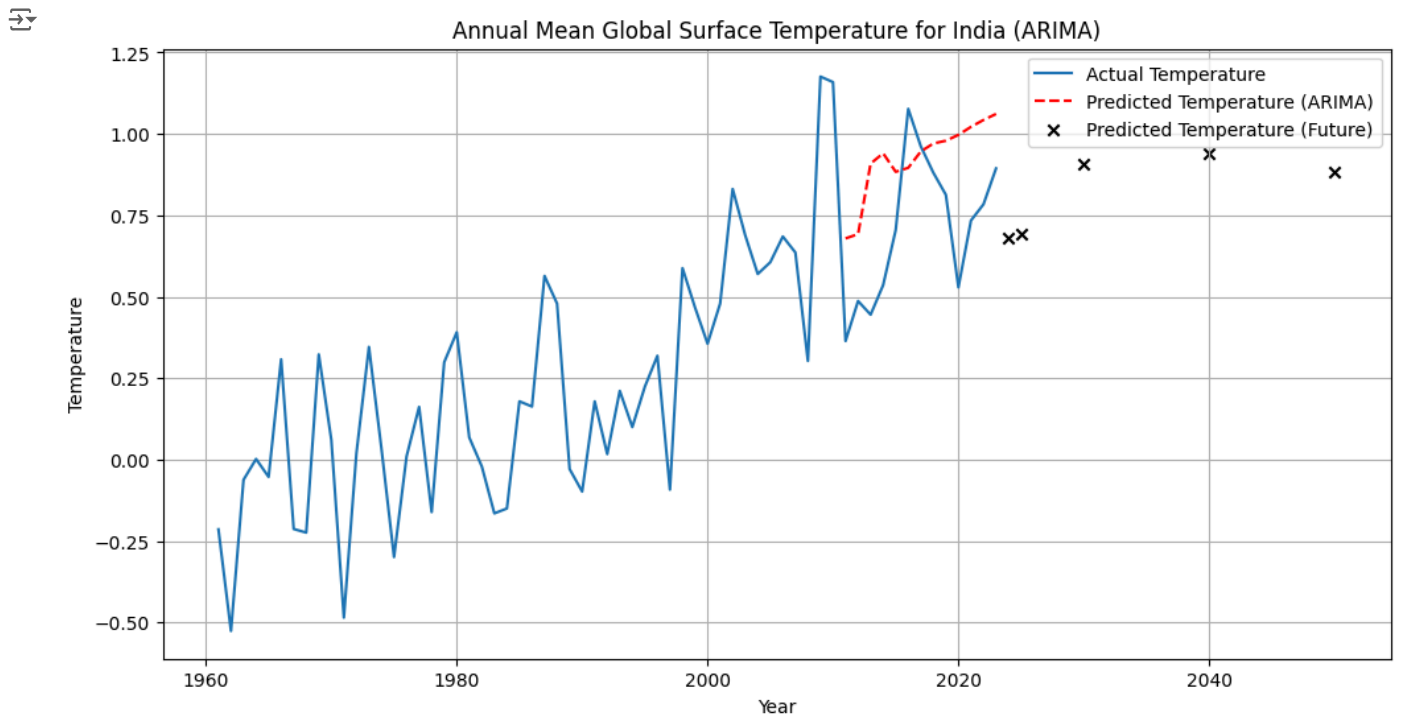
Mean Absolute Error (MAE): 0.24630695281713907

```
# Forecasting future values
future_years = [2024, 2025, 2030, 2040, 2050]
future_predictions = arima_model.predict(n_periods=len(future_years))

plt.figure(figsize=(12, 6))
plt.plot(years, temperatures, label='Actual Temperature')
plt.plot(years[len(train_data):], predictions, label='Predicted Temperature (ARIMA)', linestyle='--', color='red')
plt.scatter(future_years, future_predictions, color='black', label='Predicted Temperature (Future)', marker='x')

plt.xlabel('Year')
plt.ylabel('Temperature')
plt.title('Annual Mean Global Surface Temperature for India (ARIMA)')
plt.legend()
plt.grid(True)
plt.show()

for year, temp in zip(future_years, future_predictions):
    print(f"Predicted temperature for {year}: {temp:.2f}")
```



Predicted temperature for 2024: 0.68
 Predicted temperature for 2025: 0.69
 Predicted temperature for 2030: 0.91
 Predicted temperature for 2040: 0.94
 Predicted temperature for 2050: 0.88

Pros:

Good for short-term forecasting

Only needs historical data

Models non-stationary data

Cons:

Not built for long-term forecasting

Poor at predicting turning points

Computationally expensive

Parameters are subjective

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
import numpy as np
import matplotlib.pyplot as plt

timesteps = 7
X, y = [], []
for i in range(len(temperatures) - timesteps):
    X.append(temperatures[i:(i + timesteps)])
    y.append(temperatures[i + timesteps])
X = np.array(X).reshape(-1, timesteps, 1)
y = np.array(y)

# Split the data into training and testing sets
split_ratio = 0.8
split_index = int(len(X) * split_ratio)
X_train, X_test = X[:split_index], X[split_index:]
y_train, y_test = y[:split_index], y[split_index:]

# Build the LSTM model
model = Sequential()
model.add(LSTM(50, activation='relu', input_shape=(timesteps, 1)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')

# Train the model
model.fit(X_train, y_train, epochs=100, verbose=1)

# Make predictions
```

```
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)
```

```

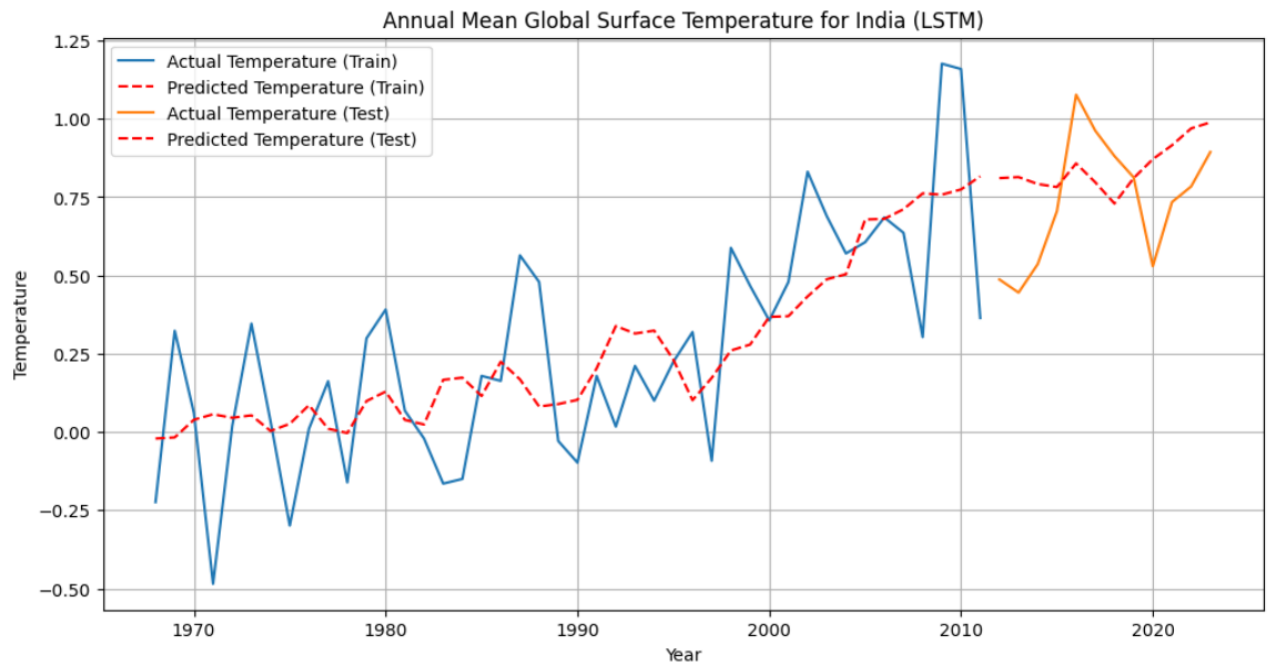
2/2 ————— 0s 10ms/step - loss: 0.0624
Epoch 74/100
2/2 ————— 0s 9ms/step - loss: 0.0648
Epoch 75/100
2/2 ————— 0s 9ms/step - loss: 0.0645
Epoch 76/100
2/2 ————— 0s 9ms/step - loss: 0.0644
Epoch 77/100
2/2 ————— 0s 8ms/step - loss: 0.0641
Epoch 78/100
2/2 ————— 0s 12ms/step - loss: 0.0699
Epoch 79/100
2/2 ————— 0s 11ms/step - loss: 0.0693
Epoch 80/100
2/2 ————— 0s 10ms/step - loss: 0.0681
Epoch 81/100
2/2 ————— 0s 11ms/step - loss: 0.0652
Epoch 82/100
2/2 ————— 0s 8ms/step - loss: 0.0688
Epoch 83/100
2/2 ————— 0s 9ms/step - loss: 0.0669
Epoch 84/100
2/2 ————— 0s 9ms/step - loss: 0.0644
Epoch 85/100
2/2 ————— 0s 9ms/step - loss: 0.0652
Epoch 86/100
2/2 ————— 0s 8ms/step - loss: 0.0623
Epoch 87/100
2/2 ————— 0s 10ms/step - loss: 0.0680
Epoch 88/100
2/2 ————— 0s 9ms/step - loss: 0.0653
Epoch 89/100
2/2 ————— 0s 9ms/step - loss: 0.0671
Epoch 90/100
2/2 ————— 0s 9ms/step - loss: 0.0645
Epoch 91/100
2/2 ————— 0s 9ms/step - loss: 0.0638
Epoch 92/100
2/2 ————— 0s 9ms/step - loss: 0.0671
Epoch 93/100
2/2 ————— 0s 9ms/step - loss: 0.0653
Epoch 94/100
2/2 ————— 0s 9ms/step - loss: 0.0642
Epoch 95/100
2/2 ————— 0s 9ms/step - loss: 0.0626
Epoch 96/100
2/2 ————— 0s 9ms/step - loss: 0.0639
Epoch 97/100
2/2 ————— 0s 9ms/step - loss: 0.0646
Epoch 98/100
2/2 ————— 0s 11ms/step - loss: 0.0644
Epoch 99/100
2/2 ————— 0s 11ms/step - loss: 0.0649
Epoch 100/100
2/2 ————— 0s 10ms/step - loss: 0.0633
2/2 ————— 0s 144ms/step
1/1 ————— 0s 19ms/step

```

```

# Plot the results
plt.figure(figsize=(12, 6))
plt.plot(years[timesteps:split_index+timesteps], y_train, label='Actual Temperature (Train)')
plt.plot(years[timesteps:split_index+timesteps], train_predict, label='Predicted Temperature (Train)', linestyle='--', color='red')
plt.plot(years[split_index+timesteps:], y_test, label='Actual Temperature (Test)')
plt.plot(years[split_index+timesteps:], test_predict, label='Predicted Temperature (Test)', linestyle='--', color='red')
plt.xlabel('Year')
plt.ylabel('Temperature')
plt.title('Annual Mean Global Surface Temperature for India (LSTM)')
plt.legend()
plt.grid(True)
plt.show()

```

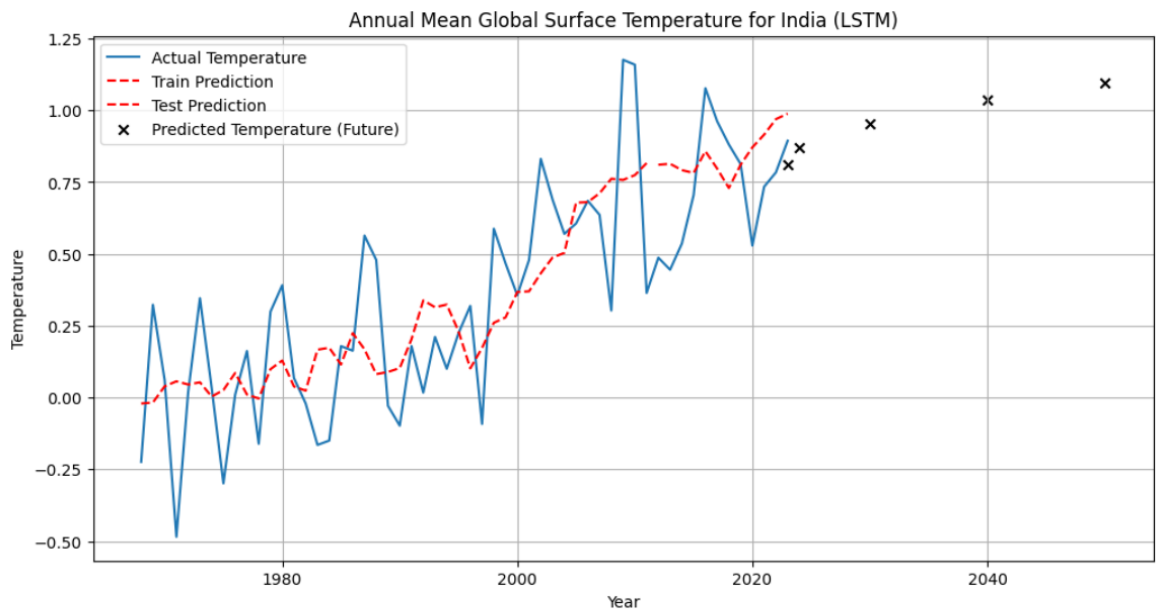


```
# Prepare future data for prediction
future_years = [2023, 2024, 2030, 2040, 2050]
future_X = []
last_sequence = X[-5] # Use the last sequence from the training data
for year in future_years:
    # Predict the next temperature in the sequence
    next_temp_pred = model.predict(np.array([last_sequence]))[0][0]

    # Append the prediction to the sequence (and remove the oldest value)
    new_sequence = np.append(last_sequence[1:], next_temp_pred)
    future_X.append(next_temp_pred)
    last_sequence = new_sequence.reshape(timesteps, 1)

# Plot the results
plt.figure(figsize=(12, 6))
plt.plot(years[timesteps:], y, label='Actual Temperature')
plt.plot(years[timesteps:len(train_predict)+timesteps], train_predict, label='Train Prediction', linestyle='--', color='red')
plt.plot(years[len(train_predict)+timesteps:], test_predict, label='Test Prediction', linestyle='--', color='red')
plt.scatter(future_years, future_X, color='black', label='Predicted Temperature (Future)', marker='x')
plt.xlabel('Year')
plt.ylabel('Temperature')
plt.title('Annual Mean Global Surface Temperature for India (LSTM)')
plt.legend()
plt.grid(True)
plt.show()

for year, temp in zip(future_years, future_X):
    print(f"Predicted temperature for {year}: {temp:.2f}")
```



```
Predicted temperature for 2023: 0.81
Predicted temperature for 2024: 0.87
Predicted temperature for 2030: 0.95
Predicted temperature for 2040: 1.04
Predicted temperature for 2050: 1.10
```

```
[24] # Evaluate the LSTM model
train_rmse = np.sqrt(mean_squared_error(y_train, train_predict))
test_rmse = np.sqrt(mean_squared_error(y_test, test_predict))

print(f'Train RMSE: {train_rmse}')
print(f'Test RMSE: {test_rmse}')

# Summarize the LSTM model
print(model.summary())
```



Train RMSE: 0.2517992425910778

Test RMSE: 0.22429270695072184

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50)	10,400
dense (Dense)	(None, 1)	51

Total params: 31,355 (122.48 KB)

Trainable params: 10,451 (40.82 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 20,904 (81.66 KB)

None

Atmospheric Co2 concentration Dataset

```
from google.colab import drive
drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

import pandas as pd

# Assuming your dataset is in a CSV file named 'your_dataset.csv'
# Replace 'your_dataset.csv' with the actual filename and path
df = pd.read_csv('/content/drive/My Drive/Project sem1/Datasets/CO2.csv')

# Now you can work with the dataframe 'df'
print(df.head())
```

ObjectId	Country	ISO2	ISO3	Indicator	Unit	Source	CTS_Code	CTS_Name	CTS_Full_Descriptor	Date	Value
0	1	World	NaN	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M03	315.70
1	2	World	NaN	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M04	317.45
2	3	World	NaN	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M05	317.51
3	4	World	NaN	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M06	317.24
4	5	World	NaN	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M07	315.86

```
df = df.drop('ISO2', axis=1)

print(df.head())
```

ObjectId	Country	ISO3	Indicator	Unit	Source	CTS_Code	CTS_Name	CTS_Full_Descriptor	Date	Value
0	1	World	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M03	315.70
1	2	World	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M04	317.45
2	3	World	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M05	317.51
3	4	World	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M06	317.24
4	5	World	Monthly Atmospheric Carbon Dioxide Concentrations	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmospe...	ECCA	Atmospheric Carbon Dioxide Concentrations	Environment, Climate Change, Climate and Weath...	1958M07	315.86

df.dtypes



0

ObjectId	int64
Country	object
ISO3	object
Indicator	object
Unit	object
Source	object
CTS_Code	object
CTS_Name	object
CTS_Full_Descriptor	object
Date	object
Value	float64

dtype: object

```
# Replace 'M' with '-' in the 'Date' column
df['Date'] = df['Date'].str.replace('M', '-')

# Convert to datetime objects
df['Date'] = pd.to_datetime(df['Date'], format='%Y-%m')

print(df.dtypes)
print(df.head())
```



ObjectId	int64						
Country	object						
ISO3	object						
Indicator	object						
Unit	object						
Source	object						
CTS_Code	object						
CTS_Name	object						
CTS_Full_Descriptor	object						
Date	datetime64[ns]						
Value	float64						
dtype: object							
	ObjectId	Country	ISO3	Indicator \			
0	1	World	WLD	Monthly Atmospheric Carbon Dioxide Concentrations			
1	2	World	WLD	Monthly Atmospheric Carbon Dioxide Concentrations			
2	3	World	WLD	Monthly Atmospheric Carbon Dioxide Concentrations			
3	4	World	WLD	Monthly Atmospheric Carbon Dioxide Concentrations			
4	5	World	WLD	Monthly Atmospheric Carbon Dioxide Concentrations			
	Unit			Source \			
0	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmosphe...					
1	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmosphe...					
2	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmosphe...					
3	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmosphe...					
4	Parts Per Million	Dr. Pieter Tans, National Oceanic and Atmosphe...					
	CTS_Code			CTS_Name \			
0	ECCA	Atmospheric Carbon Dioxide Concentrations					
1	ECCA	Atmospheric Carbon Dioxide Concentrations					
2	ECCA	Atmospheric Carbon Dioxide Concentrations					
3	ECCA	Atmospheric Carbon Dioxide Concentrations					
4	ECCA	Atmospheric Carbon Dioxide Concentrations					
	CTS_Full_Descriptor			Date	Value		
0	Environment, Climate Change, Climate and Weath...			1958-03-01	315.70		
1	Environment, Climate Change, Climate and Weath...			1958-04-01	317.45		
2	Environment, Climate Change, Climate and Weath...			1958-05-01	317.51		
3	Environment, Climate Change, Climate and Weath...			1958-06-01	317.24		
4	Environment, Climate Change, Climate and Weath...			1958-07-01	315.86		

```
# Check for missing values
print(df.isnull().sum())
```



ObjectId	0
Country	0
ISO3	0
Indicator	0
Unit	0
Source	0
CTS_Code	0
CTS_Name	0
CTS_Full_Descriptor	0
Date	0


```
Value          0
dtype: int64
```

```
# Remove rows where the 'Unit' column contains 'Percent'
df = df[~df['Unit'].str.contains('Percent')]
print(df.head(5))
print(df.tail(5))
```

```
ObjectId Country ISO3 Indicator \
0      1  World  WLD  Monthly Atmospheric Carbon Dioxide Concentrations
1      2  World  WLD  Monthly Atmospheric Carbon Dioxide Concentrations
2      3  World  WLD  Monthly Atmospheric Carbon Dioxide Concentrations
3      4  World  WLD  Monthly Atmospheric Carbon Dioxide Concentrations
4      5  World  WLD  Monthly Atmospheric Carbon Dioxide Concentrations

Unit Source \
0  Parts Per Million  Dr. Pieter Tans, National Oceanic and Atmosphe...
1  Parts Per Million  Dr. Pieter Tans, National Oceanic and Atmosphe...
2  Parts Per Million  Dr. Pieter Tans, National Oceanic and Atmosphe...
3  Parts Per Million  Dr. Pieter Tans, National Oceanic and Atmosphe...
4  Parts Per Million  Dr. Pieter Tans, National Oceanic and Atmosphe...

CTS_Code CTS_Name \
0  ECCA  Atmospheric Carbon Dioxide Concentrations
1  ECCA  Atmospheric Carbon Dioxide Concentrations
2  ECCA  Atmospheric Carbon Dioxide Concentrations
3  ECCA  Atmospheric Carbon Dioxide Concentrations
4  ECCA  Atmospheric Carbon Dioxide Concentrations

CTS_Full_Descriptor Date Value
0  Environment, Climate Change, Climate and Weath... 1958-03-01 315.70
1  Environment, Climate Change, Climate and Weath... 1958-04-01 317.45
2  Environment, Climate Change, Climate and Weath... 1958-05-01 317.51
3  Environment, Climate Change, Climate and Weath... 1958-06-01 317.24
4  Environment, Climate Change, Climate and Weath... 1958-07-01 315.86

ObjectId Country ISO3 \
1560    1561  World  WLD
1562    1563  World  WLD
1564    1565  World  WLD
1566    1567  World  WLD
1568    1569  World  WLD

Indicator Unit \
1560  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million
1562  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million
1564  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million
1566  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million
1568  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million

Source CTS_Code \
1560  Dr. Pieter Tans, National Oceanic and Atmosphe...  ECCA
1562  Dr. Pieter Tans, National Oceanic and Atmosphe...  ECCA
1564  Dr. Pieter Tans, National Oceanic and Atmosphe...  ECCA
1566  Dr. Pieter Tans, National Oceanic and Atmosphe...  ECCA
1568  Dr. Pieter Tans, National Oceanic and Atmosphe...  ECCA

CTS_Name \
1560  Atmospheric Carbon Dioxide Concentrations
1562  Atmospheric Carbon Dioxide Concentrations
1564  Atmospheric Carbon Dioxide Concentrations
1566  Atmospheric Carbon Dioxide Concentrations
1568  Atmospheric Carbon Dioxide Concentrations

CTS_Full_Descriptor Date Value
1560  Environment, Climate Change, Climate and Weath... 2023-09-01 418.51
1562  Environment, Climate Change, Climate and Weath... 2023-10-01 418.82
```

```
df = df[:-1] # removing last 0.68 1568th row

print(df.tail(5))
```

```
ObjectId Country ISO3 \
1558    1559  World  WLD
1560    1561  World  WLD
1562    1563  World  WLD
1564    1565  World  WLD
1566    1567  World  WLD

Indicator Unit \
1558  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million
1560  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million
1562  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million
1564  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million
1566  Monthly Atmospheric Carbon Dioxide Concentrations  Parts Per Million

Source CTS_Code \
1558  Dr. Pieter Tans, National Oceanic and Atmosphe...  ECCA
1560  Dr. Pieter Tans, National Oceanic and Atmosphe...  ECCA
```

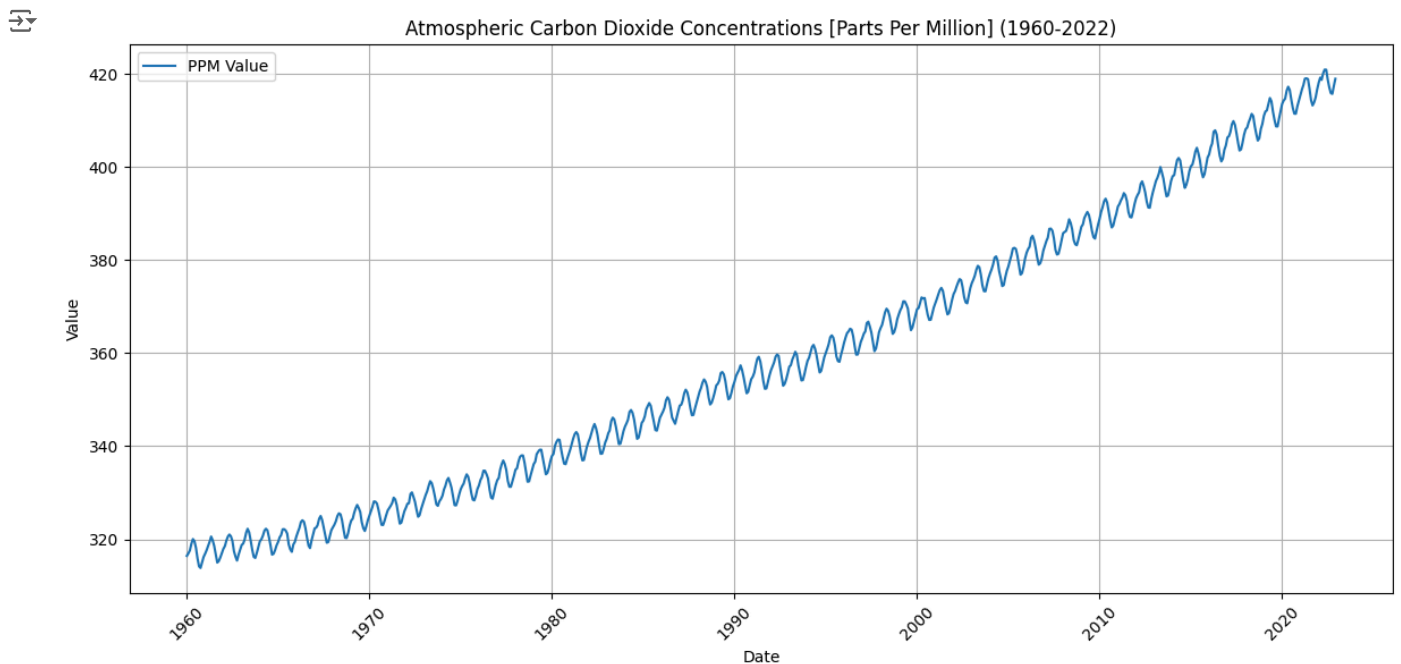
1562	Dr. Pieter Tans, National Oceanic and Atmosphe...	ECCA
1564	Dr. Pieter Tans, National Oceanic and Atmosphe...	ECCA
1566	Dr. Pieter Tans, National Oceanic and Atmosphe...	ECCA

	CTS_Name \
1558	Atmospheric Carbon Dioxide Concentrations
1560	Atmospheric Carbon Dioxide Concentrations
1562	Atmospheric Carbon Dioxide Concentrations
1564	Atmospheric Carbon Dioxide Concentrations
1566	Atmospheric Carbon Dioxide Concentrations

	CTS_Full_Descriptor	Date	Value
1558	Environment, Climate Change, Climate and Weath...	2023-08-01	419.68
1560	Environment, Climate Change, Climate and Weath...	2023-09-01	418.51
1562	Environment, Climate Change, Climate and Weath...	2023-10-01	418.82
1564	Environment, Climate Change, Climate and Weath...	2023-11-01	420.46
1566	Environment, Climate Change, Climate and Weath...	2023-12-01	421.86

```
# Filter data for the specified period
df_filtered = df[(df['Date'] >= '1960-01-01') & (df['Date'] <= '2022-12-31')]
import matplotlib.pyplot as plt
import seaborn as sns
# Plot using seaborn
plt.figure(figsize=(12, 6))
sns.lineplot(x='Date', y='Value', data=df_filtered, label='PPM Value')

plt.xlabel('Date')
plt.ylabel('Value')
plt.title('Atmospheric Carbon Dioxide Concentrations [Parts Per Million] (1960-2022)')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```



```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Prepare the data for regression
df_filtered['Time'] = (df_filtered['Date'] - df_filtered['Date'].min()).dt.days
X = df_filtered[['Time']]
y = df_filtered['Value']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the linear regression model
```

```

model = LinearRegression()
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

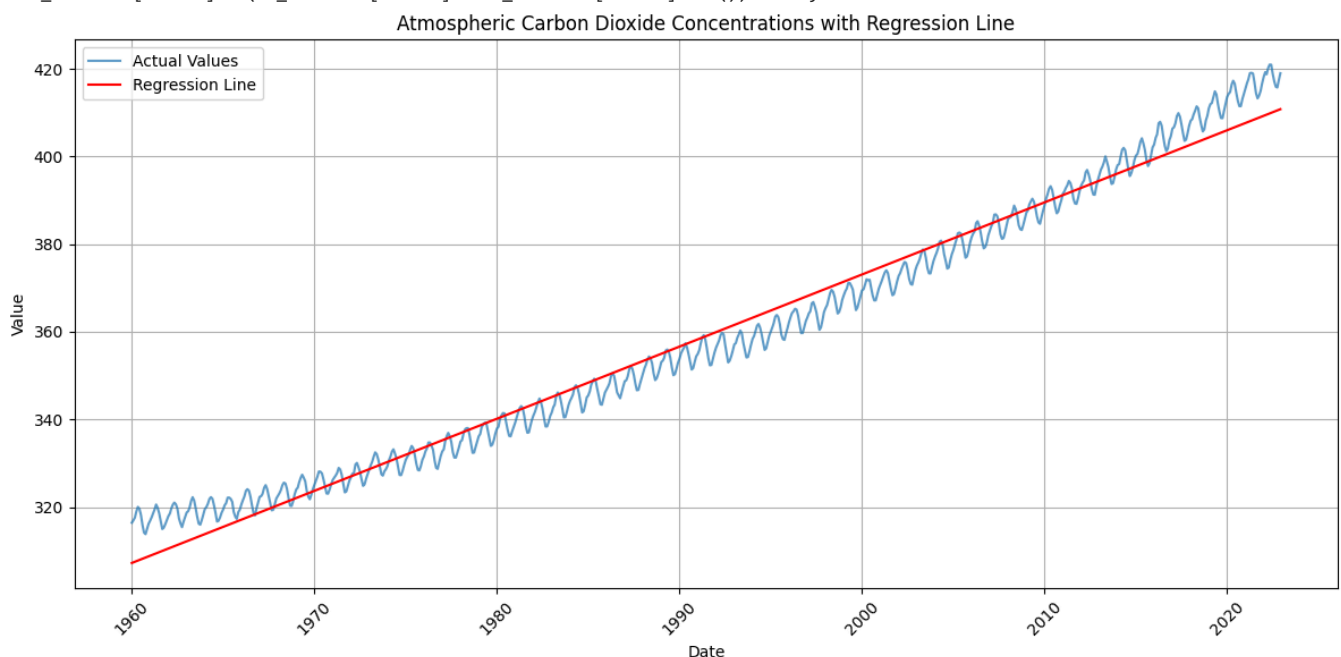
# Plot the regression line along with the original data
plt.figure(figsize=(12, 6))
sns.lineplot(x='Date', y='Value', data=df_filtered, label='Actual Values', alpha=0.7)
plt.plot(df_filtered['Date'], model.predict(X), color='red', label='Regression Line')

plt.xlabel('Date')
plt.ylabel('Value')
plt.title('Atmospheric Carbon Dioxide Concentrations with Regression Line')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()

```

 <ipython-input-10-45ef66755554>:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
`df_filtered['Time'] = (df_filtered['Date'] - df_filtered['Date'].min()).dt.days`



```

# Print the model's coefficients and intercept
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)

# Evaluate the model (example using R-squared)
from sklearn.metrics import r2_score
r2 = r2_score(y_test, y_pred)*100
print(f"R-squared: {r2} %")

# You can add other evaluation metrics as needed, such as Mean Squared Error (MSE) or Mean Absolute Error (MAE)
from sklearn.metrics import mean_squared_error, mean_absolute_error
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print(f"Mean Squared Error: {mse}")
print(f"Mean Absolute Error: {mae}")

```

 Coefficients: [0.00450561]
Intercept: 307.2485750703215
R-squared: 97.74697051786634 %
Mean Squared Error: 19.107936506686393
Mean Absolute Error: 3.6588814342312617

```

# Predict values for future years
future_dates = pd.to_datetime(['2023-12-31', '2024-12-31', '2030-12-31', '2040-12-31', '2050-12-31'])
# Changed the following line to get days attribute from the TimedeltaIndex
future_time = (future_dates - df_filtered['Date'].min()).days
future_predictions = model.predict(future_time.values.reshape(-1, 1))

# Create a DataFrame for future predictions
future_df = pd.DataFrame({'Date': future_dates, 'Value': future_predictions})

# Concatenate the original data and future predictions
combined_df = pd.concat([df_filtered, future_df], ignore_index=True)

# Plot the combined data
plt.figure(figsize=(12, 6))
sns.lineplot(x='Date', y='Value', data=combined_df, label='Actual Values', alpha=0.7)

# Highlight the future predictions
plt.scatter(future_df['Date'], future_df['Value'], color='red', label='Future Predictions')

# Note: Ensure 'Time' column is available in combined_df, or adjust accordingly
plt.plot(combined_df['Date'], model.predict((combined_df['Date'] - combined_df['Date'].min()).dt.days.values.reshape(-1,1)), color='green')

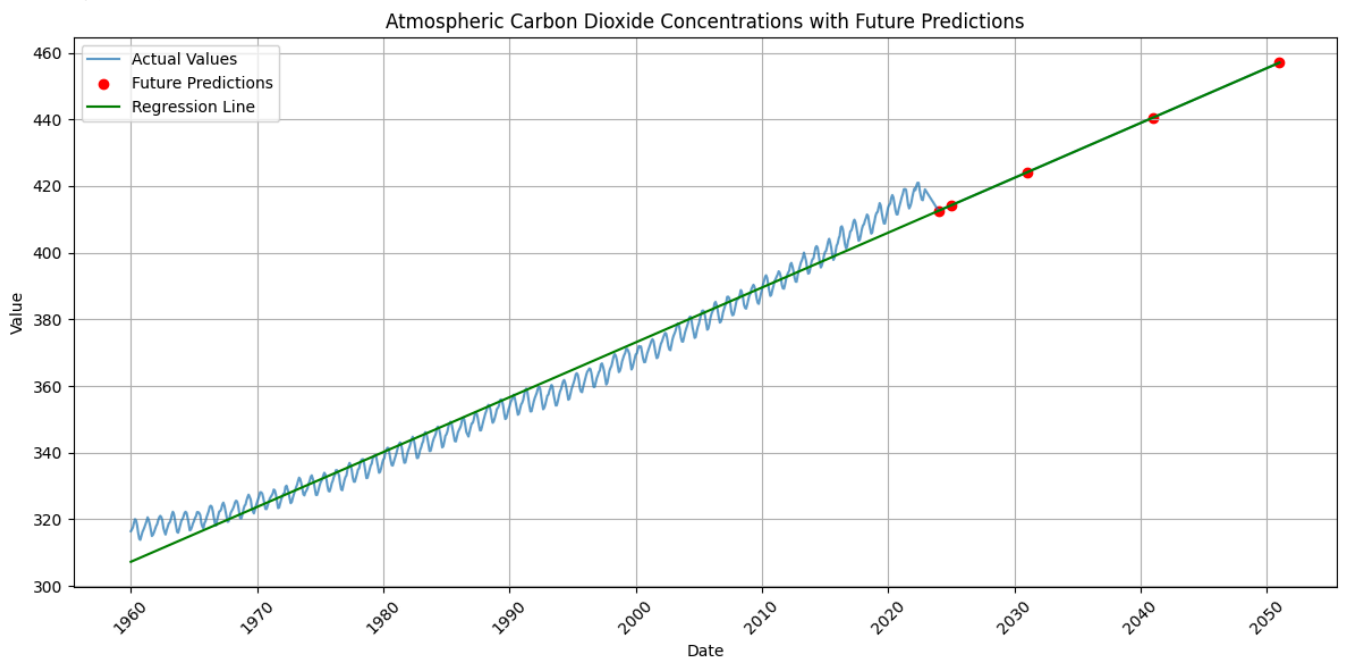
plt.xlabel('Date')
plt.ylabel('Value')
plt.title('Atmospheric Carbon Dioxide Concentrations with Future Predictions')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()

```

```

/usr/local/lib/python3.10/dist-packages/sklearn/base.py:493: UserWarning: X does not have valid feature names, but LinearRegression
warnings.warn(
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:493: UserWarning: X does not have valid feature names, but LinearRegression
warnings.warn(

```



```

# Predict values for future years
future_dates = pd.to_datetime(['2023-12-31', '2024-12-31', '2030-12-31', '2040-12-31', '2050-12-31'])
future_time = (future_dates - df_filtered['Date'].min()).days
future_predictions = model.predict(future_time.values.reshape(-1, 1))

print("Future Predictions:")
for date, prediction in zip(future_dates, future_predictions):
    print(f>Date: {date}, Predicted Value: {prediction}")

# Create a DataFrame for future predictions
future_df = pd.DataFrame({'Date': future_dates, 'Value': future_predictions})

```

```

Future Predictions:
Date: 2023-12-31 00:00:00, Predicted Value: 412.5672139719967

```

```

Date: 2024-12-31 00:00:00, Predicted Value: 414.2162673126603
Date: 2030-12-31 00:00:00, Predicted Value: 424.08805930554036
Date: 2040-12-31 00:00:00, Predicted Value: 440.5470534406342
Date: 2050-12-31 00:00:00, Predicted Value: 457.0015419655077
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:493: UserWarning: X does not have valid feature names, but LinearRegression
warnings.warn(

```

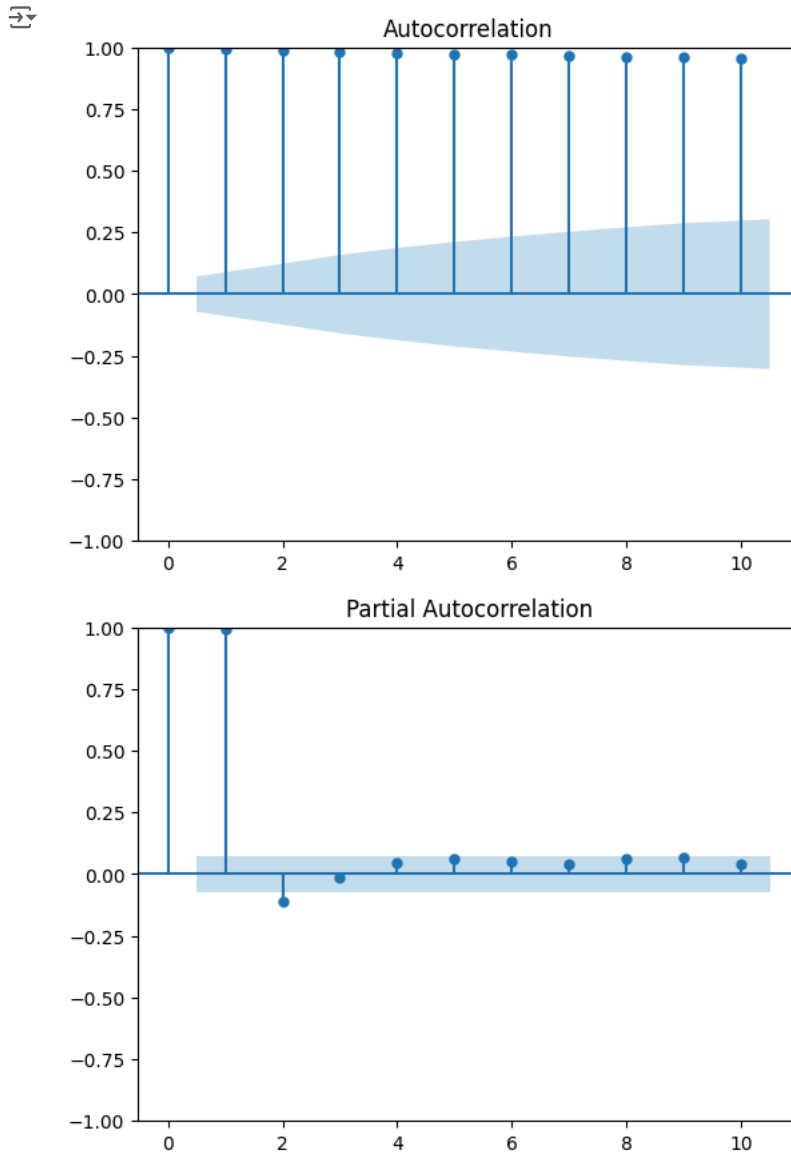
```

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Calculate ACF and PACF
plot_acf(df_filtered['Value'], lags=10) # Adjust lags as needed
plt.show()

plot_pacf(df_filtered['Value'], lags=10) # Adjust lags as needed
plt.show()

```



```

from statsmodels.tsa.stattools import adfuller

# Perform the Augmented Dickey-Fuller test
result = adfuller(df_filtered['Value'])

# Print the test results
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))

# Interpret the results
p_value = result[1]
if p_value <= 0.05:
    print("The series is stationary (reject the null hypothesis)")
else:

```

```

print("The series is non-stationary (fail to reject the null hypothesis)")

p = p_value
q = result[0] #ADF Statistic

print(f"p-value: {p}")
print(f"ADF Statistic: {q}")

```

```

➦ ADF Statistic: 5.075071
p-value: 1.000000
Critical Values:
    1%: -3.439
    5%: -2.865
   10%: -2.569
The series is non-stationary (fail to reject the null hypothesis)
p-value: 1.0
ADF Statistic: 5.075071484750594

```

```

import numpy as np
# Perform the Augmented Dickey-Fuller test
result = adfuller(df_filtered['Value'])
p_value = result[1]
adf_statistic = result[0]

print(f"Initial p-value: {p_value}")
print(f"Initial ADF Statistic: {adf_statistic}")

def adjust_data(data, d):
    #Example: differencing the data 'd' times
    adjusted_data = data.copy()
    for _ in range(d):
        adjusted_data = adjusted_data.diff().dropna()
    return adjusted_data

#Testing various differencing values
for d in range(1, 4): # Test d values from 1 to 3 (inclusive)
    adjusted_series = adjust_data(df_filtered['Value'],d)

    if len(adjusted_series)>0: #Check if the adjusted series is not empty after differencing.
        result_adjusted = adfuller(adjusted_series)
        p_value_adjusted = result_adjusted[1]
        adf_statistic_adjusted = result_adjusted[0]

        print(f"\nDifferencing order (d): {d}")
        print(f"Adjusted p-value: {p_value_adjusted}")
        print(f"Adjusted ADF Statistic: {adf_statistic_adjusted}")

        if p_value_adjusted <= 0.05:
            print(f"For d = {d}, the series is likely stationary.")
            break # Stop if stationary is found.
    else:
        print(f"\nDifferencing order (d): {d} leads to an empty series.")

```

```

➦ Initial p-value: 1.0
Initial ADF Statistic: 5.075071484750594

Differencing order (d): 1
Adjusted p-value: 1.0477685570090106e-05
Adjusted ADF Statistic: -5.1625842162862385
For d = 1, the series is likely stationary.

```

```

!pip install pmdarima
from pmdarima import auto_arima

# Assuming 'df_filtered' and 'y' are defined as in the previous code
# Fit Auto-ARIMA
stepwise_fit = auto_arima(y, trace=True, suppress_warnings=True)

# Print the best model's parameters and AIC
print(stepwise_fit.summary())

```

```

➦ Collecting pmdarima
  Downloading pmdarima-2.0.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl.metadata (7.8 kB)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.4.2)
Requirement already satisfied: Cython!=0.29.18,!0.29.31,>=0.29 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (3.0.1)
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.26.4)
Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.2)
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.5.2)
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.13.1)
Requirement already satisfied: statsmodels>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (0.14.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.3)
Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (75.1.0)
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (24.2)

```

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.22->pmdarima) (3.3.0)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-packages (from statsmodels>=0.13.2->pmdarima) (1.0.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas>=0.19->pmdarima) (1.16.0)
Downloading pmdarima-2.0.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl (2.1 MB)
2.1/2.1 MB 13.2 MB/s eta 0:00:00

Installing collected packages: pmdarima
Successfully installed pmdarima-2.0.4

Performing stepwise search to minimize aic

ARIMA(2,1,2)(0,0,0)[0] intercept	:	AIC=1510.855, Time=1.66 sec
ARIMA(0,1,0)(0,0,0)[0] intercept	:	AIC=2469.183, Time=0.04 sec
ARIMA(1,1,0)(0,0,0)[0] intercept	:	AIC=1961.209, Time=0.11 sec
ARIMA(0,1,1)(0,0,0)[0] intercept	:	AIC=2019.580, Time=0.23 sec
ARIMA(0,1,0)(0,0,0)[0]	:	AIC=2476.216, Time=0.04 sec
ARIMA(1,1,2)(0,0,0)[0] intercept	:	AIC=1791.916, Time=0.31 sec
ARIMA(2,1,1)(0,0,0)[0] intercept	:	AIC=1509.595, Time=0.59 sec
ARIMA(1,1,1)(0,0,0)[0] intercept	:	AIC=1860.172, Time=0.24 sec
ARIMA(2,1,0)(0,0,0)[0] intercept	:	AIC=1755.361, Time=0.16 sec
ARIMA(3,1,1)(0,0,0)[0] intercept	:	AIC=1510.991, Time=0.93 sec
ARIMA(3,1,0)(0,0,0)[0] intercept	:	AIC=1681.833, Time=0.25 sec
ARIMA(3,1,2)(0,0,0)[0] intercept	:	AIC=1511.946, Time=1.73 sec
ARIMA(2,1,1)(0,0,0)[0]	:	AIC=1606.290, Time=0.14 sec

Best model: ARIMA(2,1,1)(0,0,0)[0] intercept

Total fit time: 6.424 seconds

SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:          756
Model:                SARIMAX(2, 1, 1)      Log Likelihood      -749.798
Date:                Thu, 12 Dec 2024      AIC                  1509.595
Time:                07:05:40      BIC                  1532.729
Sample:              0      HQIC                  1518.506
                   - 756
Covariance Type:      opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
intercept	0.0412	0.003	11.886	0.000	0.034	0.048
ar.L1	1.5441	0.022	70.329	0.000	1.501	1.587
ar.L2	-0.8465	0.023	-37.316	0.000	-0.891	-0.802
ma.L1	-0.8989	0.018	-49.566	0.000	-0.934	-0.863
sigma2	0.4251	0.022	19.561	0.000	0.382	0.468

```
from statsmodels.tsa.arima.model import ARIMA
import numpy as np
```

```
p = 2
d = 1
q = 1
```

```
# Fit the ARIMA model
```

```
model = ARIMA(df_filtered['Value'], order=(p, d, q))
model_fit = model.fit()
```

```
# Make predictions for future dates
```

```
future_predictions_arima = model_fit.predict(start=len(df_filtered), end=len(df_filtered) + len(future_dates) - 1)
```

```
# Create a DataFrame for future ARIMA predictions
```

```
future_df_arima = pd.DataFrame({'Date': future_dates, 'Value': future_predictions_arima})
```

```
print("Future Predictions using ARIMA:")
```

```
for date, prediction in zip(future_dates, future_predictions_arima):
    print(f"Date: {date}, Predicted Value: {prediction}")
```

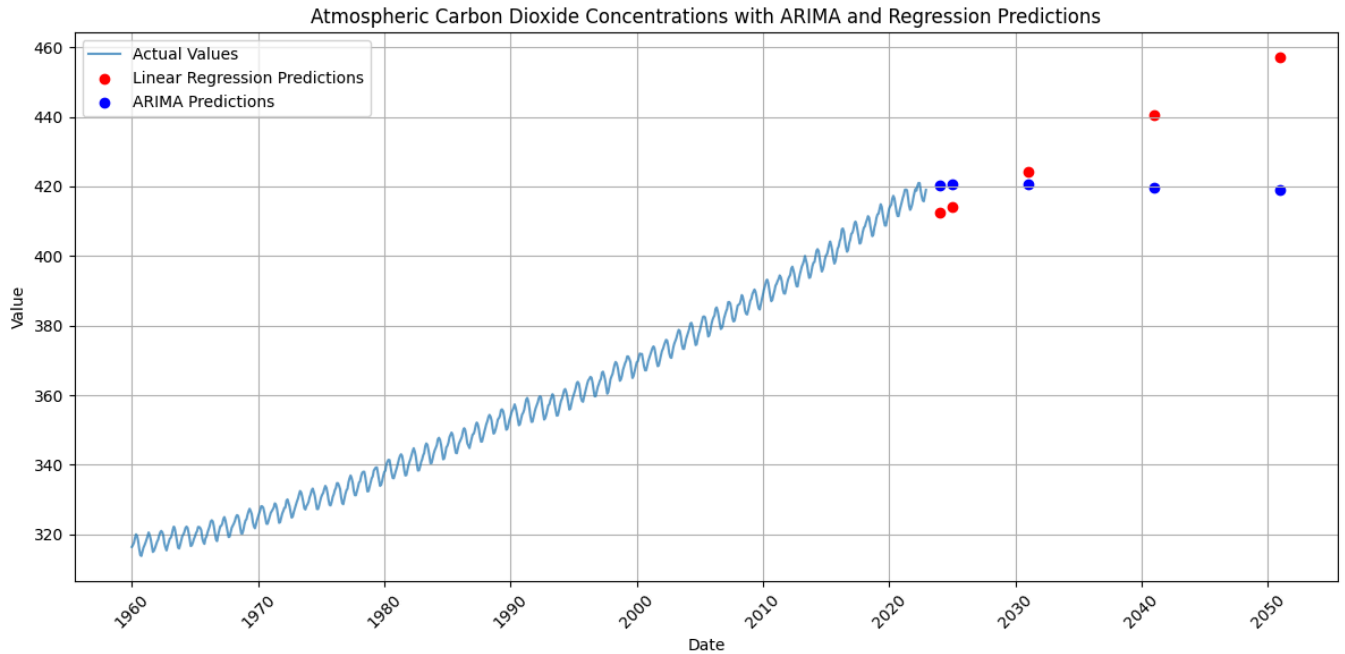
```
# Plot the combined data with ARIMA predictions
```

```
plt.figure(figsize=(12, 6))
sns.lineplot(x='Date', y='Value', data=df_filtered, label='Actual Values', alpha=0.7)
plt.scatter(future_df['Date'], future_df['Value'], color='red', label='Linear Regression Predictions')
plt.scatter(future_df_arima['Date'], future_df_arima['Value'], color='blue', label='ARIMA Predictions')
plt.xlabel('Date')
plt.ylabel('Value')
plt.title('Atmospheric Carbon Dioxide Concentrations with ARIMA and Regression Predictions')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```

```

/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: An unsupported index was provided. As a
self._init_dates(dates, freq)
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: An unsupported index was provided. As a
self._init_dates(dates, freq)
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: An unsupported index was provided. As a
self._init_dates(dates, freq)
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available. Predict
return get_prediction_index(
/usr/local/lib/python3.10/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available. In th
return get_prediction_index(
Future Predictions using ARIMA:
Date: 2023-12-31 00:00:00, Predicted Value: 420.1446884419008
Date: 2024-12-31 00:00:00, Predicted Value: 420.6556712411023
Date: 2030-12-31 00:00:00, Predicted Value: 420.48374825519846
Date: 2040-12-31 00:00:00, Predicted Value: 419.7988454326123
Date: 2050-12-31 00:00:00, Predicted Value: 418.8932374688619

```



```

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Normalize the 'Value' column
scaler = MinMaxScaler()
df_filtered['Value_scaled'] = scaler.fit_transform(df_filtered['Value'].values.reshape(-1, 1))

# Prepare the data for LSTM
def create_dataset(dataset, look_back=1):
    X, Y = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
        X.append(a)
        Y.append(dataset[i + look_back, 0])
    return np.array(X), np.array(Y)

look_back = 10 # Adjust this value as needed
X, y = create_dataset(df_filtered['Value_scaled'].values.reshape(-1, 1), look_back)

# Reshape X for LSTM input
X = np.reshape(X, (X.shape[0], X.shape[1], 1))

# Split data into train and test sets
train_size = int(len(X) * 0.8)
test_size = len(X) - train_size
X_train, X_test = X[0:train_size,:], X[train_size:len(X),:]
y_train, y_test = y[0:train_size], y[train_size:len(y)]

# Create and train the LSTM model
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
model.add(LSTM(units=50))

```



```

model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1) # Adjust epochs and batch_size

# Make predictions
train_predict = model.predict(X_train)
test_predict = model.predict(X_test)

# Inverse transform the predictions
train_predict = scaler.inverse_transform(train_predict)
y_train = scaler.inverse_transform([y_train])
test_predict = scaler.inverse_transform(test_predict)
y_test = scaler.inverse_transform([y_test])

# Plot the predictions
plt.figure(figsize=(12, 6))
plt.plot(scaler.inverse_transform(df_filtered['Value_scaled'].values.reshape(-1,1)), label='Actual')
plt.plot(np.concatenate((train_predict, test_predict)), label='Predicted')
plt.xlabel('Time')
plt.ylabel('CO2 Value')
plt.title('LSTM Predictions for CO2')
plt.legend()
plt.show()

```

 <ipython-input-22-cae99dfa131d>:7: SettingWithCopyWarning:
 A value is trying to be set on a copy of a slice from a DataFrame.
 Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers

```

df_filtered['Value_scaled'] = scaler.fit_transform(df_filtered['Value'].values.reshape(-1, 1))
/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/rnn.py:204: UserWarning: Do not pass an `input_shape`/`input_dim` ar
    super().__init__(**kwargs)

```

```

Epoch 1/50
19/19 ━━━━━━━━━━━ 6s 16ms/step - loss: 0.0578
Epoch 2/50
19/19 ━━━━━━━━━━━ 0s 18ms/step - loss: 0.0045
Epoch 3/50
19/19 ━━━━━━━━━━━ 1s 17ms/step - loss: 0.0011
Epoch 4/50
19/19 ━━━━━━━━━━━ 1s 20ms/step - loss: 6.7726e-04
Epoch 5/50
19/19 ━━━━━━━━━━━ 1s 21ms/step - loss: 7.0704e-04
Epoch 6/50
19/19 ━━━━━━━━━━━ 0s 17ms/step - loss: 6.5038e-04
Epoch 7/50
19/19 ━━━━━━━━━━━ 0s 9ms/step - loss: 6.7625e-04
Epoch 8/50
19/19 ━━━━━━━━━━━ 0s 8ms/step - loss: 6.6186e-04
Epoch 9/50
19/19 ━━━━━━━━━━━ 0s 8ms/step - loss: 7.3815e-04
Epoch 10/50
19/19 ━━━━━━━━━━━ 0s 8ms/step - loss: 6.8707e-04
Epoch 11/50
19/19 ━━━━━━━━━━━ 0s 14ms/step - loss: 6.4444e-04
Epoch 12/50
19/19 ━━━━━━━━━━━ 0s 16ms/step - loss: 7.3039e-04
Epoch 13/50
19/19 ━━━━━━━━━━━ 1s 15ms/step - loss: 7.4004e-04
Epoch 14/50
19/19 ━━━━━━━━━━━ 0s 16ms/step - loss: 6.5039e-04
Epoch 15/50
19/19 ━━━━━━━━━━━ 1s 13ms/step - loss: 6.5172e-04
Epoch 16/50
19/19 ━━━━━━━━━━━ 0s 14ms/step - loss: 6.3937e-04
Epoch 17/50
19/19 ━━━━━━━━━━━ 0s 14ms/step - loss: 6.6003e-04
Epoch 18/50
19/19 ━━━━━━━━━━━ 0s 15ms/step - loss: 6.4712e-04
Epoch 19/50
19/19 ━━━━━━━━━━━ 1s 13ms/step - loss: 6.5149e-04
Epoch 20/50
19/19 ━━━━━━━━━━━ 0s 15ms/step - loss: 6.1914e-04
Epoch 21/50
19/19 ━━━━━━━━━━━ 1s 10ms/step - loss: 6.7000e-04
Epoch 22/50
19/19 ━━━━━━━━━━━ 0s 9ms/step - loss: 6.4874e-04
Epoch 23/50
19/19 ━━━━━━━━━━━ 0s 9ms/step - loss: 6.8052e-04
Epoch 24/50
19/19 ━━━━━━━━━━━ 0s 9ms/step - loss: 7.3382e-04
Epoch 25/50

```

```

future_dates = pd.to_datetime(['2023-12-31', '2024-12-31', '2030-12-31', '2040-12-31', '2050-12-31'])

```

```

future_time = (future_dates - df_filtered['Date'].min()).days
future_time_scaled = scaler.transform(future_time.values.reshape(-1, 1))

```

```

# Create sequences for future predictions
future_X = []
last_sequence = X[-1]

for i in range(len(future_dates)):
    last_sequence = np.append(last_sequence[1:], future_time_scaled[i])
    future_X.append(last_sequence)

future_X = np.array(future_X).reshape(len(future_dates), look_back, 1)

# Predict future values using LSTM
future_predictions_lstm = model.predict(future_X)

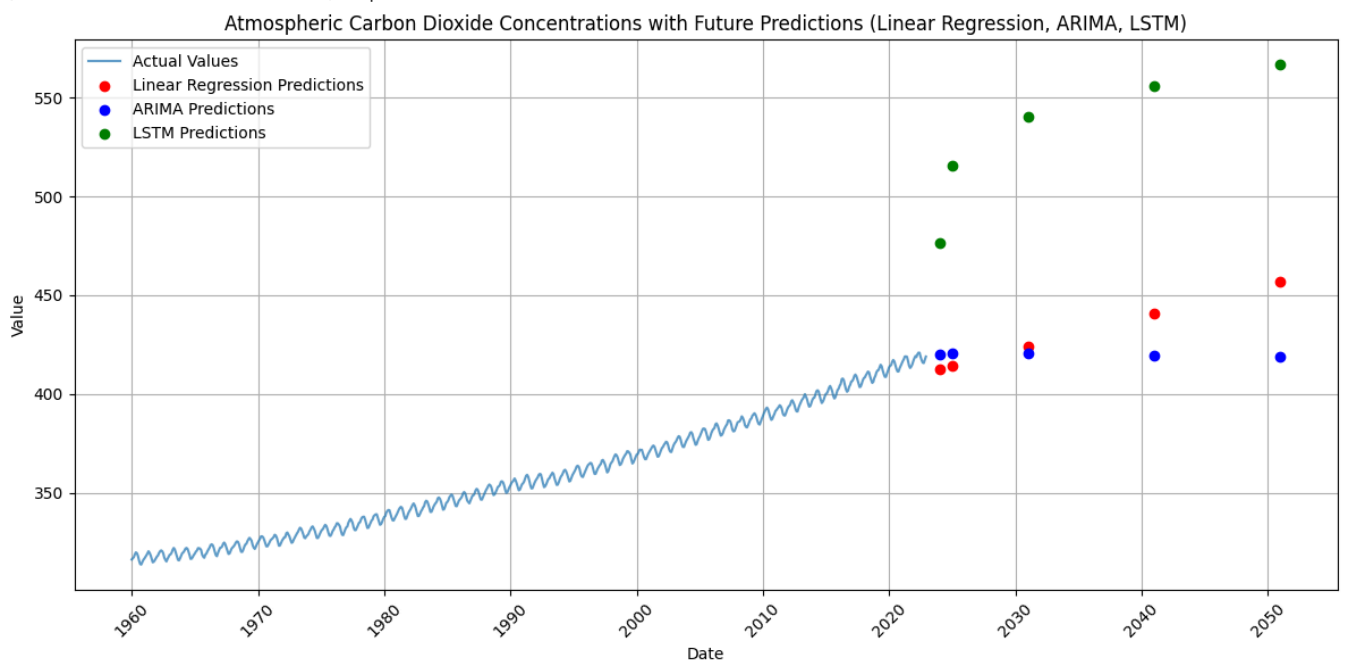
# Inverse transform to get actual values
future_predictions_lstm = scaler.inverse_transform(future_predictions_lstm)

# Create DataFrame for LSTM future predictions
future_df_lstm = pd.DataFrame({'Date': future_dates, 'Value': future_predictions_lstm.flatten()})

# Plot all predictions together
plt.figure(figsize=(12, 6))
sns.lineplot(x='Date', y='Value', data=df_filtered, label='Actual Values', alpha=0.7)
plt.scatter(future_df['Date'], future_df['Value'], color='red', label='Linear Regression Predictions')
plt.scatter(future_df_arima['Date'], future_df_arima['Value'], color='blue', label='ARIMA Predictions')
plt.scatter(future_df_lstm['Date'], future_df_lstm['Value'], color='green', label='LSTM Predictions')
plt.xlabel('Date')
plt.ylabel('Value')
plt.title('Atmospheric Carbon Dioxide Concentrations with Future Predictions (Linear Regression, ARIMA, LSTM)')
plt.grid(True)
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()

```

1/1 — 0s 20ms/step



```

# Print the model summary LSTM
model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 10, 50)	10,400
lstm_1 (LSTM)	(None, 50)	20,200
dense (Dense)	(None, 1)	51

Total params: 91,955 (359.20 KB)
Trainable params: 30,651 (119.73 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 61,304 (239.47 KB)

Change in Mean Sea Level Dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd
```

```
df = pd.read_csv('/content/drive/My Drive/Project sem1/Datasets/Change_in_Mean_Sea_Levels.csv')
```

```
# Now you can work with the dataframe 'df'
print(df.head())
```

```
↗
   ObjectId Country  ISO2 ISO3 \
0         1   World   NaN  WLD
1         2   World   NaN  WLD
2         3   World   NaN  WLD
3         4   World   NaN  WLD
4         5   World   NaN  WLD

                                Indicator      Unit \
0  Change in mean sea level: Sea level: TOPEX.Pos...  Millimeters
1  Change in mean sea level: Sea level: TOPEX.Pos...  Millimeters
2  Change in mean sea level: Sea level: TOPEX.Pos...  Millimeters
3  Change in mean sea level: Sea level: TOPEX.Pos...  Millimeters
4  Change in mean sea level: Sea level: TOPEX.Pos...  Millimeters

                                Source CTS_Code \
0  National Oceanic and Atmospheric Administratio...  ECCL
1  National Oceanic and Atmospheric Administratio...  ECCL
2  National Oceanic and Atmospheric Administratio...  ECCL
3  National Oceanic and Atmospheric Administratio...  ECCL
4  National Oceanic and Atmospheric Administratio...  ECCL

                                CTS_Name \
0  Change in Mean Sea Level
1  Change in Mean Sea Level
2  Change in Mean Sea Level
3  Change in Mean Sea Level
4  Change in Mean Sea Level

                                CTS_Full_Descriptor      Measure \
0  Environment, Climate Change, Climate Indicator...  Andaman Sea
1  Environment, Climate Change, Climate Indicator...  Arabian Sea
2  Environment, Climate Change, Climate Indicator...  Atlantic Ocean
3  Environment, Climate Change, Climate Indicator...  Baltic Sea
4  Environment, Climate Change, Climate Indicator...  Bay Bengal

    Date  Value
0  D12/17/1992  -10.34
1  D12/17/1992  -18.46
2  D12/17/1992  -15.41
3  D12/17/1992  196.85
4  D12/17/1992   3.27
```

```
df = df.drop('ISO2', axis=1)
```

```
print(df.head())
```

```
↗
   ObjectId Country ISO3                                Indicator \
0         1   World  WLD  Change in mean sea level: Sea level: TOPEX.Pos...
1         2   World  WLD  Change in mean sea level: Sea level: TOPEX.Pos...
2         3   World  WLD  Change in mean sea level: Sea level: TOPEX.Pos...
3         4   World  WLD  Change in mean sea level: Sea level: TOPEX.Pos...
4         5   World  WLD  Change in mean sea level: Sea level: TOPEX.Pos...

                                Unit      Source CTS_Code \
0  Millimeters  National Oceanic and Atmospheric Administratio...  ECCL
1  Millimeters  National Oceanic and Atmospheric Administratio...  ECCL
2  Millimeters  National Oceanic and Atmospheric Administratio...  ECCL
3  Millimeters  National Oceanic and Atmospheric Administratio...  ECCL
4  Millimeters  National Oceanic and Atmospheric Administratio...  ECCL

                                CTS_Name \
0  Change in Mean Sea Level
1  Change in Mean Sea Level
2  Change in Mean Sea Level
3  Change in Mean Sea Level
4  Change in Mean Sea Level

                                CTS_Full_Descriptor      Measure \
0  Environment, Climate Change, Climate Indicator...  Andaman Sea
1  Environment, Climate Change, Climate Indicator...  Arabian Sea
2  Environment, Climate Change, Climate Indicator...  Atlantic Ocean
3  Environment, Climate Change, Climate Indicator...  Baltic Sea
4  Environment, Climate Change, Climate Indicator...  Bay Bengal
```

	Date	Value
0	D12/17/1992	-10.34
1	D12/17/1992	-18.46
2	D12/17/1992	-15.41
3	D12/17/1992	196.85
4	D12/17/1992	3.27

```
df.dtypes
```

```

ObjectID      int64
Country       object
ISO3          object
Indicator      object
Unit          object
Source        object
CTS_Code      object
CTS_Name      object
CTS_Full_Descriptor object
Measure       object
Date          object
Value         float64
dtype: object

```

```
df['Date'] = pd.to_datetime(df['Date'].str.lstrip('D'), format='%m/%d/%Y', errors='coerce')
```

```
# Verify the updated data type
print(df.dtypes)
```

```

ObjectID      int64
Country       object
ISO3          object
Indicator      object
Unit          object
Source        object
CTS_Code      object
CTS_Name      object
CTS_Full_Descriptor object
Measure       object
Date          datetime64[ns]
Value         float64
dtype: object

```

```
print(df['Date'])
```

```

0      1992-12-17
1      1992-12-17
2      1992-12-17
3      1992-12-17
4      1992-12-17
...
35599  2022-11-08
35600  2022-11-08
35601  2022-11-08
35602  2022-11-08
35603  2022-11-08
Name: Date, Length: 35604, dtype: datetime64[ns]

```

```
unique_measures = df['Measure'].unique()
unique_measures
```

```

array(['Andaman Sea', 'Arabian Sea', 'Atlantic Ocean', 'Baltic Sea',
      'Bay Bengal', 'Caribbean Sea', 'Gulf Mexico', 'Indian Ocean',
      'Indonesian', 'Mediterranean', 'Nino', 'North Pacific',
      'North Sea', 'Pacific Ocean', 'Persian Gulf', 'Sea Japan',
      'Sea Okhotsk', 'South China', 'Southern Ocean', 'Tropics', 'World',
      'Yellow Sea', 'Adriatic Sea', 'Bering Sea', 'North Atlantic'],
      dtype=object)

```

```
import matplotlib.pyplot as plt
```

```
plt.style.use('default')
```

```

# Create an empty dictionary to store the yearly changes
yearly_changes = {}

# Iterate over unique measures
for measure in unique_measures:
    yearly_changes[measure] = {}
    # Filter data for the current measure
    measure_df = df[df['Measure'] == measure]

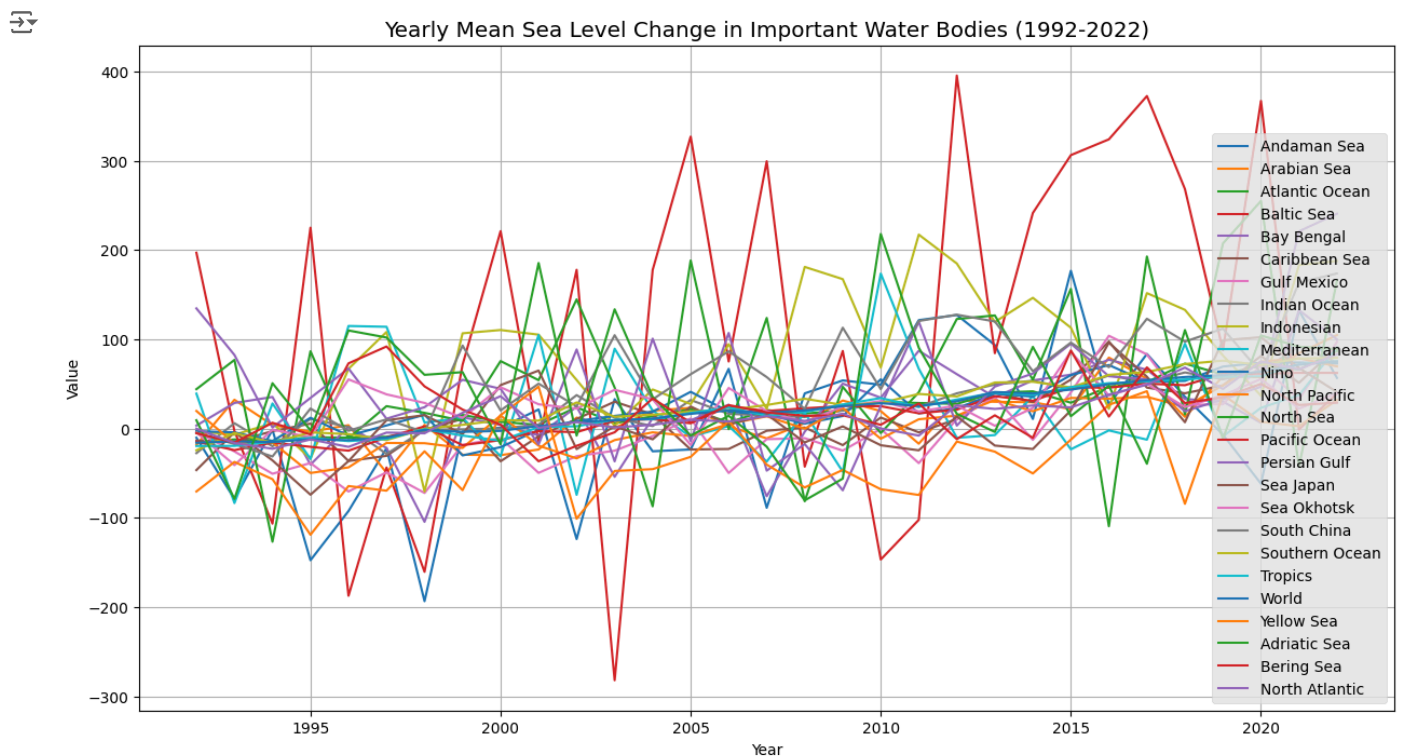
    # Iterate through years from 1992 to 2022
    for year in range(1992, 2023):
        # Filter data for the current year
        year_df = measure_df[measure_df['Date'].dt.year == year]

        # Check if data exists for the year. If not, add NaN.
        if not year_df.empty:
            yearly_changes[measure][year] = year_df['Value'].iloc[0]
        else:
            yearly_changes[measure][year] = float('nan')

# Plotting the yearly changes for each unique measure
plt.figure(figsize=(15, 8)) # Adjust the figure size as needed

for measure, values in yearly_changes.items():
    years = list(values.keys())
    changes = list(values.values())
    plt.plot(years, changes, label=measure)
with plt.style.context('ggplot'):
    plt.xlabel('Year')
    plt.ylabel('Value')
    plt.title('Yearly Mean Sea Level Change in Important Water Bodies (1992-2022)')
    plt.legend()
    plt.grid(True)
    plt.show()

```



```

import matplotlib.pyplot as plt

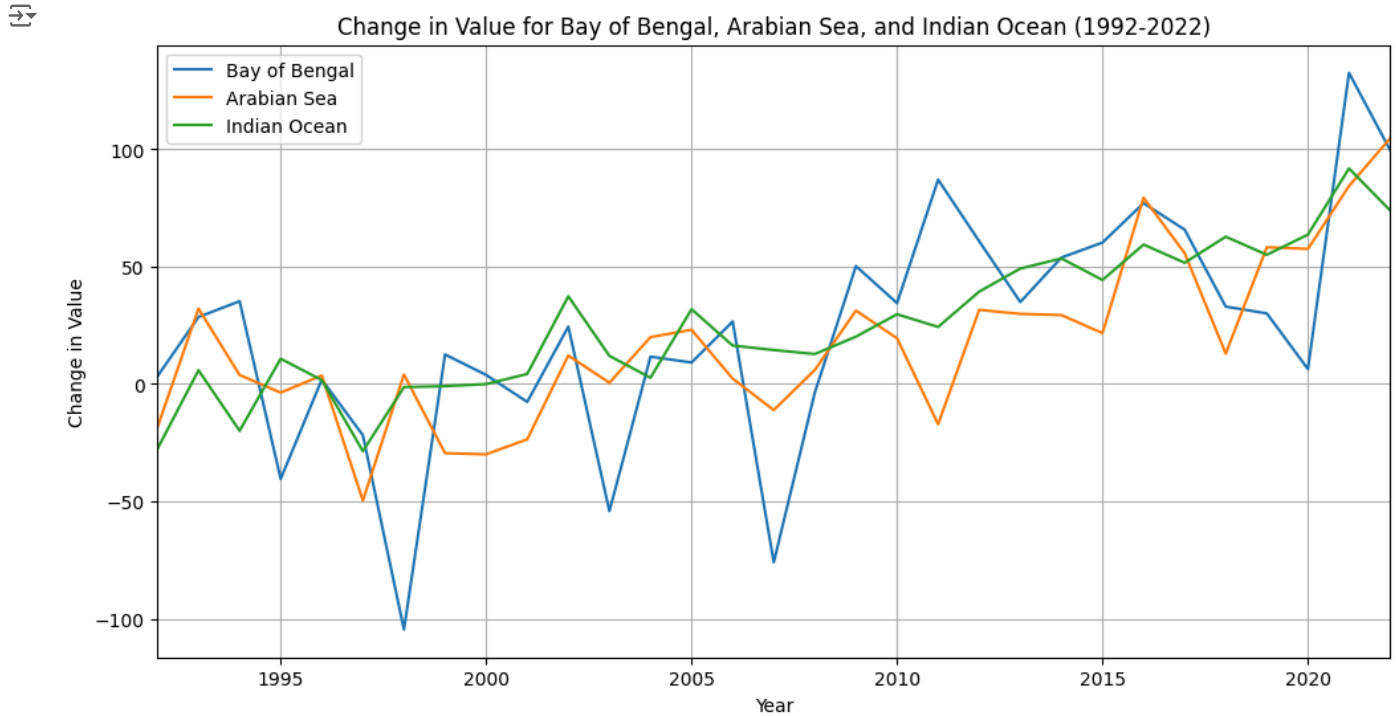
bay_of_bengal_data = yearly_changes.get('Bay Bengal', {})
arabian_sea_data = yearly_changes.get('Arabian Sea', {})
indian_ocean_data = yearly_changes.get('Indian Ocean', {})

plt.figure(figsize=(12, 6))

```

```
plt.plot(list(bay_of_bengal_data.keys()), list(bay_of_bengal_data.values()), label='Bay of Bengal')
plt.plot(list(arabian_sea_data.keys()), list(arabian_sea_data.values()), label='Arabian Sea')
plt.plot(list(indian_ocean_data.keys()), list(indian_ocean_data.values()), label='Indian Ocean')

plt.xlabel('Year')
plt.ylabel('Change in Value')
plt.title('Change in Value for Bay of Bengal, Arabian Sea, and Indian Ocean (1992-2022)')
plt.legend()
plt.grid(True)
plt.xlim(1992, 2022)
plt.show()
```



```
print(arabian_sea_data)
```

```
{1992: -18.46, 1993: 32.14, 1994: 3.84, 1995: -3.66, 1996: 3.64, 1997: -49.66, 1998: 4.04, 1999: -29.46, 2000: -29.96, 2001: -23.56,
```

```
print(indian_ocean_data)
```

```
{1992: -27.63, 1993: 5.87, 1994: -19.93, 1995: 10.77, 1996: 1.67, 1997: -28.63, 1998: -1.33, 1999: -0.93, 2000: -0.03, 2001: 4.27, 2002: 38.63,
```

```
print(bay_of_bengal_data)
```

```
{1992: 3.27, 1993: 28.57, 1994: 35.27, 1995: -40.43, 1996: 1.87, 1997: -21.83, 1998: -104.63, 1999: 12.57, 2000: 3.87, 2001: -7.63, 2002: 25.27,
```

```
import statsmodels.api as sm
```

```
bay_of_bengal_data = yearly_changes.get('Bay Bengal', {})
arabian_sea_data = yearly_changes.get('Arabian Sea', {})
indian_ocean_data = yearly_changes.get('Indian Ocean', {})

# Function to perform linear regression and return the regression line
def get_regression_line(data):
    years = list(data.keys())
    values = list(data.values())

    # Remove NaN values
    valid_indices = [i for i, val in enumerate(values) if not pd.isna(val)]
    years_valid = [years[i] for i in valid_indices]
    values_valid = [values[i] for i in valid_indices]

    X = sm.add_constant(years_valid) # Add a constant for the intercept
    model = sm.OLS(values_valid, X).fit()
    return model.params[0], model.params[1]
```

```
# Get regression lines for each dataset
```

```

intercept_bay, slope_bay = get_regression_line(bay_of_bengal_data)
intercept_arabian, slope_arabian = get_regression_line(arabian_sea_data)
intercept_indian, slope_indian = get_regression_line(indian_ocean_data)

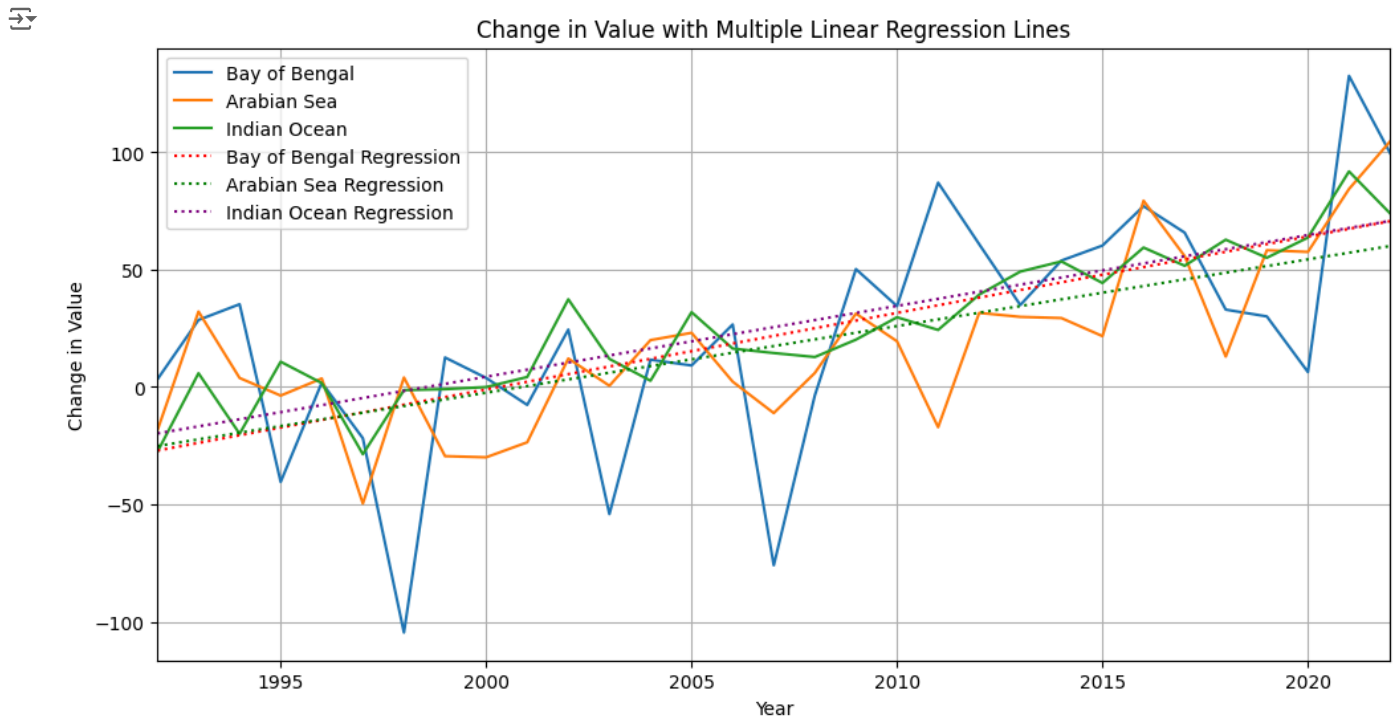
# Create the plot
plt.figure(figsize=(12, 6))

# Plot each dataset
plt.plot(list(bay_of_bengal_data.keys()), list(bay_of_bengal_data.values()), label='Bay of Bengal')
plt.plot(list(arabian_sea_data.keys()), list(arabian_sea_data.values()), label='Arabian Sea')
plt.plot(list(indian_ocean_data.keys()), list(indian_ocean_data.values()), label='Indian Ocean')

# Plot regression lines
years = range(1992, 2023)
plt.plot(years, [intercept_bay + slope_bay * year for year in years], color='red', label='Bay of Bengal Regression',linestyle='dotted')
plt.plot(years, [intercept_arabian + slope_arabian * year for year in years], color='green', label='Arabian Sea Regression',linestyle='dotted')
plt.plot(years, [intercept_indian + slope_indian * year for year in years], color='purple', label='Indian Ocean Regression',linestyle='dotted')

# Customize the plot
plt.xlabel('Year')
plt.ylabel('Change in Value')
plt.title('Change in Value with Multiple Linear Regression Lines')
plt.legend()
plt.grid(True)
plt.xlim(1992, 2022) # Set x-axis limits
plt.show()

```



```

import numpy as np

def regression_summary(data, name):
    years = list(data.keys())
    values = list(data.values())

    # Remove NaN values
    valid_indices = [i for i, val in enumerate(values) if not pd.isna(val)]
    years_valid = np.array([years[i] for i in valid_indices])
    values_valid = np.array([values[i] for i in valid_indices])

    X = sm.add_constant(years_valid)
    model = sm.OLS(values_valid, X).fit()
    print(f"Summary for {name}:")
    print(model.summary())

    predictions = model.predict(X)
    mse = np.mean((values_valid - predictions)**2)
    rmse = np.sqrt(mse)
    mae = np.mean(np.abs(values_valid - predictions))
    print(f"Mean Squared Error (MSE): {mse}")
    print(f"Root Mean Squared Error (RMSE): {rmse}")
    print(f"Mean Absolute Error (MAE): {mae}")
    print("-" * 20)

```



```

regression_summary(bay_of_bengal_data, "Bay of Bengal")
regression_summary(arabian_sea_data, "Arabian Sea")
regression_summary(indian_ocean_data, "Indian Ocean")

```

```

=====
Date:                Sat, 14 Dec 2024    Prob (F-statistic):    2.75e-06
Time:                03:34:08            Log-Likelihood:       -141.97
No. Observations:    31                  AIC:                 287.9
Df Residuals:        29                  BIC:                 290.8
Df Model:            1
Covariance Type:     nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-5684.0382	982.836	-5.783	0.000	-7694.163	-3673.913
x1	2.8408	0.490	5.801	0.000	1.839	3.842

```

=====
Omnibus:                0.162    Durbin-Watson:           1.511
Prob(Omnibus):          0.922    Jarque-Bera (JB):        0.228
Skew:                   0.150    Prob(JB):                0.892
Kurtosis:               2.706    Cond. No.:               4.50e+05
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 4.5e+05. This might indicate that there are strong multicollinearity or other numerical problems.
Mean Squared Error (MSE): 556.3479337135805
Root Mean Squared Error (RMSE): 23.587028929341237
Mean Absolute Error (MAE): 18.99426404786677

Summary for Indian Ocean:

OLS Regression Results

```

=====
Dep. Variable:          y    R-squared:                0.837
Model:                  OLS    Adj. R-squared:         0.831
Method:                 Least Squares    F-statistic:         149.0
Date:                   Sat, 14 Dec 2024    Prob (F-statistic):    5.97e-13
Time:                   03:34:08    Log-Likelihood:       -120.80
No. Observations:       31    AIC:                 245.6
Df Residuals:           29    BIC:                 248.5
Df Model:               1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-6034.4112	496.438	-12.155	0.000	-7049.741	-5019.081
x1	3.0194	0.247	12.207	0.000	2.514	3.525

```

=====
Omnibus:                2.611    Durbin-Watson:           2.123
Prob(Omnibus):          0.271    Jarque-Bera (JB):        1.884
Skew:                   0.604    Prob(JB):                0.390
Kurtosis:               3.006    Cond. No.:               4.50e+05
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 4.5e+05. This might indicate that there are strong multicollinearity or other numerical problems.
Mean Squared Error (MSE): 141.9432567104577
Root Mean Squared Error (RMSE): 11.913994154373993
Mean Absolute Error (MAE): 9.27123803329857

```
# Predict values for future years
```

```
future_years = [2023, 2024, 2030, 2040, 2050]
```

```

predicted_bay_of_bengal = [intercept_bay + slope_bay * year for year in future_years]
predicted_arabian_sea = [intercept_arabian + slope_arabian * year for year in future_years]
predicted_indian_ocean = [intercept_indian + slope_indian * year for year in future_years]

```

```
# Print the predictions
```

```

print("Predictions for future years:")
for i in range(len(future_years)):
    print(f"Year: {future_years[i]}")
    print(f" Bay of Bengal: {predicted_bay_of_bengal[i]}")
    print(f" Arabian Sea: {predicted_arabian_sea[i]}")
    print(f" Indian Ocean: {predicted_indian_ocean[i]}")

```

```
# Plot the predictions along with the existing data and regression lines
```

```
plt.figure(figsize=(12, 6))
```

```
# Plot each dataset
```

```

plt.plot(list(bay_of_bengal_data.keys()), list(bay_of_bengal_data.values()), label='Bay of Bengal')
plt.plot(list(arabian_sea_data.keys()), list(arabian_sea_data.values()), label='Arabian Sea')
plt.plot(list(indian_ocean_data.keys()), list(indian_ocean_data.values()), label='Indian Ocean')

```

```
# Plot regression lines
```

```

# Plot Regression Lines
years = range(1992, 2051) # Extended to include future years
plt.plot(years, [intercept_bay + slope_bay * year for year in years], color='red', label='Bay of Bengal Regression', linestyle='dotted')
plt.plot(years, [intercept_arabian + slope_arabian * year for year in years], color='green', label='Arabian Sea Regression', linestyle='dotted')
plt.plot(years, [intercept_indian + slope_indian * year for year in years], color='purple', label='Indian Ocean Regression', linestyle='dotted')

# Plot future predictions
plt.plot(future_years, predicted_bay_of_bengal, 'ro', label='Bay of Bengal Prediction')
plt.plot(future_years, predicted_arabian_sea, 'go', label='Arabian Sea Prediction')
plt.plot(future_years, predicted_indian_ocean, 'mo', label='Indian Ocean Prediction')

# Customize the plot
plt.xlabel('Year')
plt.ylabel('Change in Value')
plt.title('Change in Value with Predictions')
plt.legend()
plt.grid(True)
plt.xlim(1992, 2050) # Set x-axis limits
plt.show()

```

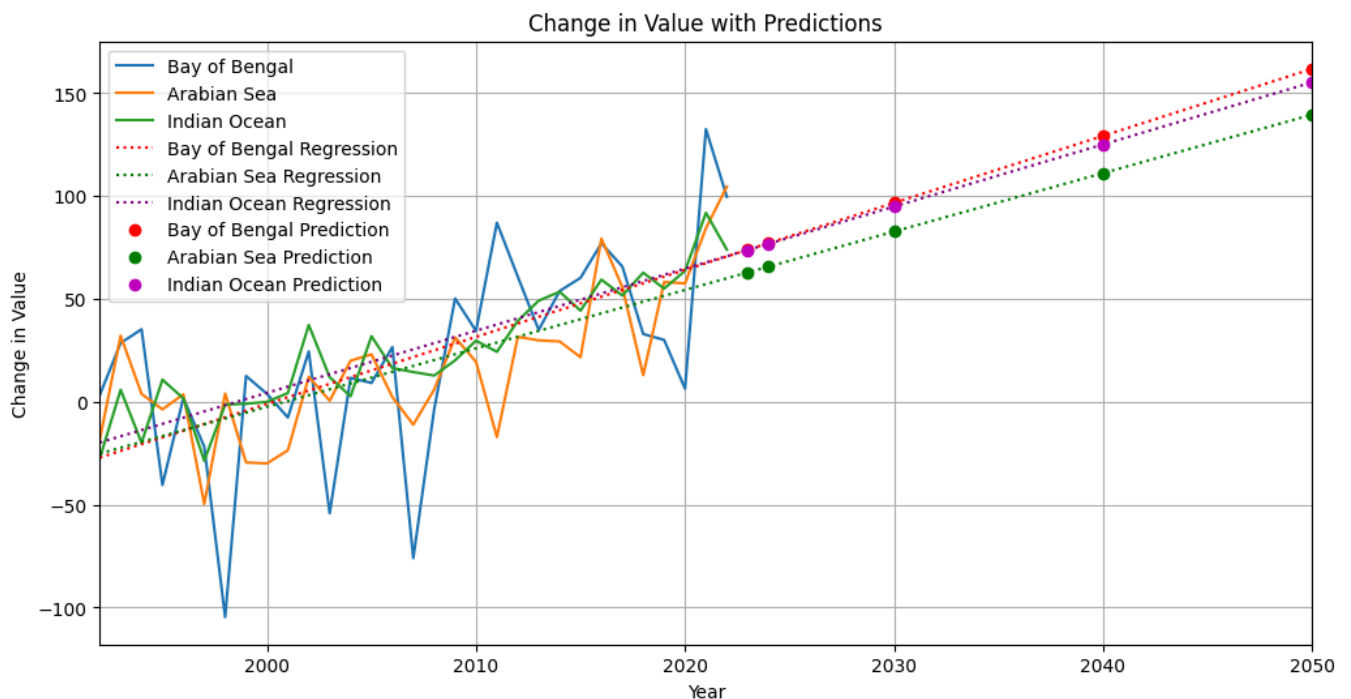


Predictions for future years:

```

Year: 2023
  Bay of Bengal: 73.92012903225714
  Arabian Sea: 62.905096774193225
  Indian Ocean: 73.81703225806359
Year: 2024
  Bay of Bengal: 77.17846370967618
  Arabian Sea: 65.74589919354821
  Indian Ocean: 76.83642338709615
Year: 2030
  Bay of Bengal: 96.72847177419226
  Arabian Sea: 82.79071370967722
  Indian Ocean: 94.95277016128966
Year: 2040
  Bay of Bengal: 129.31181854838542
  Arabian Sea: 111.19873790322526
  Indian Ocean: 125.14668145161158
Year: 2050
  Bay of Bengal: 161.89516532257858
  Arabian Sea: 139.6067620967733
  Indian Ocean: 155.3405927419335

```



```

import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

indian_ocean_data = yearly_changes.get('Indian Ocean', {})
values = list(indian_ocean_data.values())

# Remove NaN values
values = [x for x in values if not pd.isna(x)]

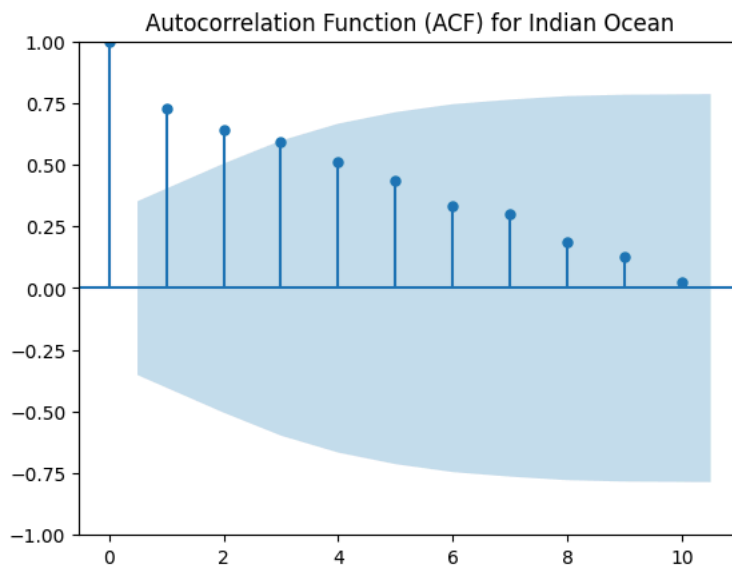
# Plot ACF
plt.figure(figsize=(10, 5))
plot_acf(values, lags=10, alpha=0.05) # Adjust lags as needed

```

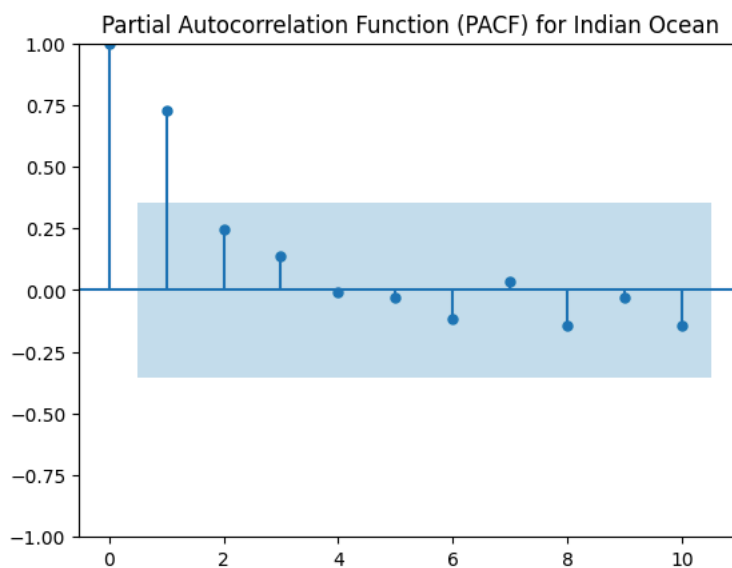
```
plt.title('Autocorrelation Function (ACF) for Indian Ocean')
plt.show()

# Plot PACF
plt.figure(figsize=(10, 5))
plot_pacf(values, lags=10, alpha=0.05, method='ywmm') # Adjust lags as needed
plt.title('Partial Autocorrelation Function (PACF) for Indian Ocean')
plt.show()
```

↗ <Figure size 1000x500 with 0 Axes>



<Figure size 1000x500 with 0 Axes>



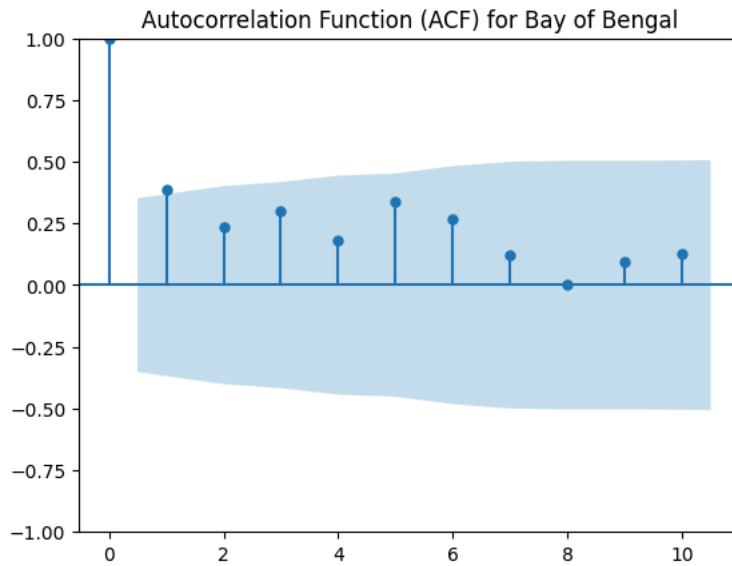
```
bay_of_bengal_data = yearly_changes.get('Bay Bengal', {})
values = list(bay_of_bengal_data.values())

# Remove NaN values
values = [x for x in values if not pd.isna(x)]

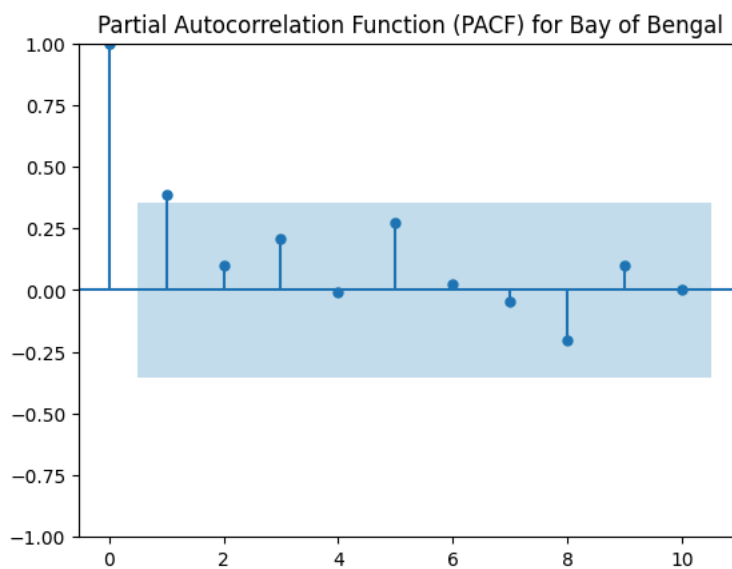
# Plot ACF
plt.figure(figsize=(10, 5))
plot_acf(values, lags=10, alpha=0.05) # Adjust lags as needed
plt.title('Autocorrelation Function (ACF) for Bay of Bengal')
plt.show()

# Plot PACF
plt.figure(figsize=(10, 5))
plot_pacf(values, lags=10, alpha=0.05, method='ywmm') # Adjust lags as needed
plt.title('Partial Autocorrelation Function (PACF) for Bay of Bengal')
plt.show()
```

<Figure size 1000x500 with 0 Axes>

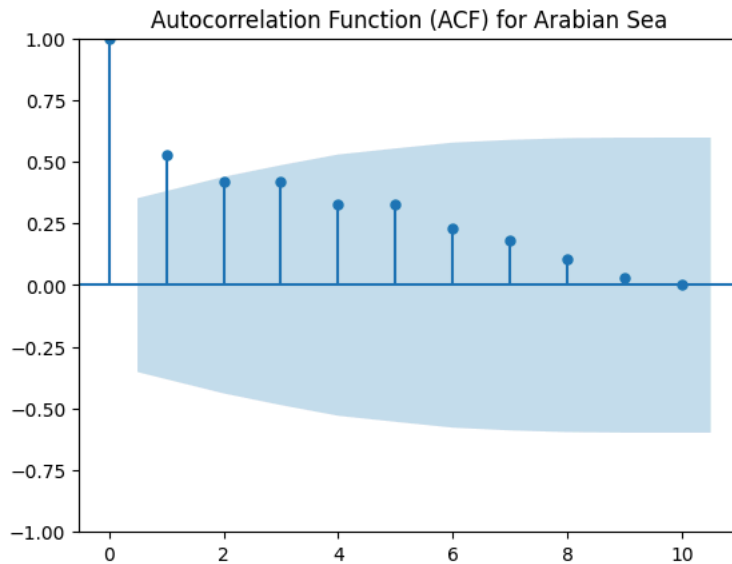


<Figure size 1000x500 with 0 Axes>

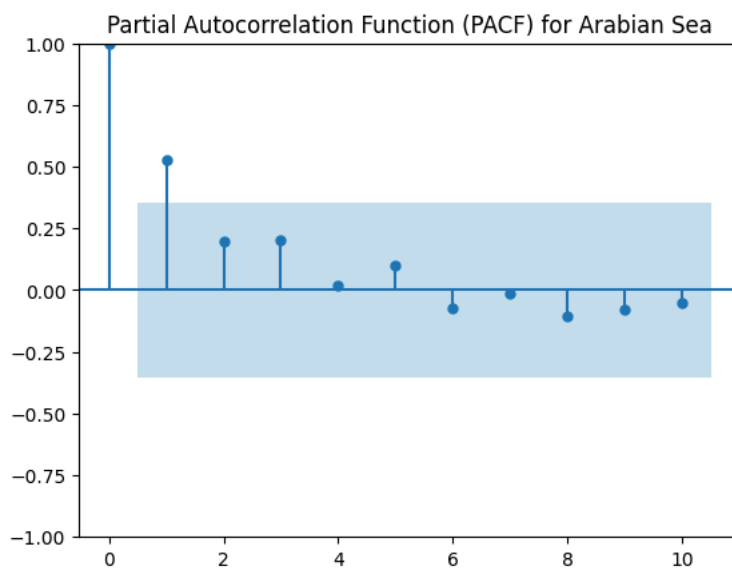


```
arabian_sea_data = yearly_changes.get('Arabian Sea', {})  
values = list(arabian_sea_data.values())  
  
# Remove NaN values  
values = [x for x in values if not pd.isna(x)]  
  
# Plot ACF  
plt.figure(figsize=(10, 5))  
plot_acf(values, lags=10, alpha=0.05) # Adjust lags as needed  
plt.title('Autocorrelation Function (ACF) for Arabian Sea')  
plt.show()  
  
# Plot PACF  
plt.figure(figsize=(10, 5))  
plot_pacf(values, lags=10, alpha=0.05, method='ywmm') # Adjust lags as needed  
plt.title('Partial Autocorrelation Function (PACF) for Arabian Sea')  
plt.show()
```

<Figure size 1000x500 with 0 Axes>



<Figure size 1000x500 with 0 Axes>



```
from statsmodels.tsa.stattools import adfuller

result = adfuller(values)

# The p-value is in the result[1] position.
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])

# Determine the optimal d value based on the p-value
# We don't need to look for p and q values from the plots here because we are calculating the p-value for the Augmented Dickey Fuller tes

if result[1] <= 0.05:
    print("The time series is likely stationary (d=0).")
    d = 0
else:
    print("The time series is likely non-stationary.")
    print("Difference the series once and repeat ADF test. You might need to difference it multiple times")
    # Difference the series once.
    values_diff = np.diff(values)

    # Perform ADF test on differenced data
    result_diff = adfuller(values_diff)
    print('ADF Statistic (Differenced): %f' % result_diff[0])
    print('p-value (Differenced): %f' % result_diff[1])

    if result_diff[1] <= 0.05:
        print("The differenced time series is likely stationary (d=1).")
        d = 1
    else:
        print("The differenced time series is likely non-stationary.")
        print("Try differencing again (d=2)")
        d = 2 # Or try further differencing until the series becomes stationary
```

```
print(f"The appropriate value for d is: {d}")
```

```
ADF Statistic: 0.250121
p-value: 0.974937
The time series is likely non-stationary.
Difference the series once and repeat ADF test. You might need to difference it multiple times
ADF Statistic (Differenced): -5.978864
p-value (Differenced): 0.000000
The differenced time series is likely stationary (d=1).
The appropriate value for d is: 1
```

```
!pip install pmdarima
# Import the necessary library
from pmdarima import auto_arima

indian_ocean_data = yearly_changes.get('Indian Ocean', {})
values = list(indian_ocean_data.values())
values = [x for x in values if not pd.isna(x)]

# Use auto_arima to find the best (p, d, q) order
stepwise_model = auto_arima(values, start_p=0, start_q=0,
                             max_p=5, max_q=5, m=12,
                             start_P=0, seasonal=False,
                             d=None, D=1, trace=True,
                             error_action='ignore',
                             suppress_warnings=True,
                             stepwise=True)

print(stepwise_model.summary())
```

```
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.5.2)
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (1.13.1)
Requirement already satisfied: statsmodels>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (0.14.4)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (2.2.3)
Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (75.1.0)
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.10/dist-packages (from pmdarima) (24.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.19->pmdarima) (2024.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=0.22->pmdarima) (3.3.0)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-packages (from statsmodels>=0.13.2->pmdarima) (1.0.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.8.2->pandas>=0.19->pmdarima) (1.16.0)
Downloading pmdarima-2.0.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.manylinux_2_28_x86_64.whl (2.1 MB)
2.1/2.1 MB 24.8 MB/s eta 0:00:00
```

Installing collected packages: pmdarima

Successfully installed pmdarima-2.0.4

/usr/local/lib/python3.10/dist-packages/pmdarima/arima/_validation.py:62: UserWarning: m (12) set for non-seasonal fit. Setting to 0 warnings.warn("m (%i) set for non-seasonal fit. Setting to 0" % m)

Performing stepwise search to minimize aic

```
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=261.361, Time=0.06 sec
ARIMA(1,1,0)(0,0,0)[0] intercept : AIC=253.678, Time=0.03 sec
ARIMA(0,1,1)(0,0,0)[0] intercept : AIC=inf, Time=0.12 sec
ARIMA(0,1,0)(0,0,0)[0] intercept : AIC=260.446, Time=0.01 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : AIC=247.449, Time=0.07 sec
ARIMA(3,1,0)(0,0,0)[0] intercept : AIC=249.310, Time=0.10 sec
ARIMA(2,1,1)(0,0,0)[0] intercept : AIC=249.224, Time=0.09 sec
ARIMA(1,1,1)(0,0,0)[0] intercept : AIC=inf, Time=0.15 sec
ARIMA(3,1,1)(0,0,0)[0] intercept : AIC=inf, Time=0.26 sec
ARIMA(2,1,0)(0,0,0)[0] intercept : AIC=252.884, Time=0.03 sec
```

Best model: ARIMA(2,1,0)(0,0,0)[0] intercept

Total fit time: 0.962 seconds

SARIMAX Results

```
=====
Dep. Variable:          y      No. Observations:          31
Model:                SARIMAX(2, 1, 0)      Log Likelihood      -119.724
Date:                Sat, 14 Dec 2024      AIC                  247.449
Time:                 03:35:43      BIC                  253.053
Sample:              0      HQIC                  249.242
                    - 31
Covariance Type:      opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
intercept	7.4877	3.107	2.410	0.016	1.397	13.578
ar.L1	-0.7799	0.246	-3.174	0.002	-1.262	-0.298
ar.L2	-0.5269	0.204	-2.589	0.010	-0.926	-0.128
sigma2	166.0032	48.897	3.395	0.001	70.167	261.840

```
=====
Ljung-Box (L1) (Q):          0.04      Jarque-Bera (JB):          0.29
```

```
import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

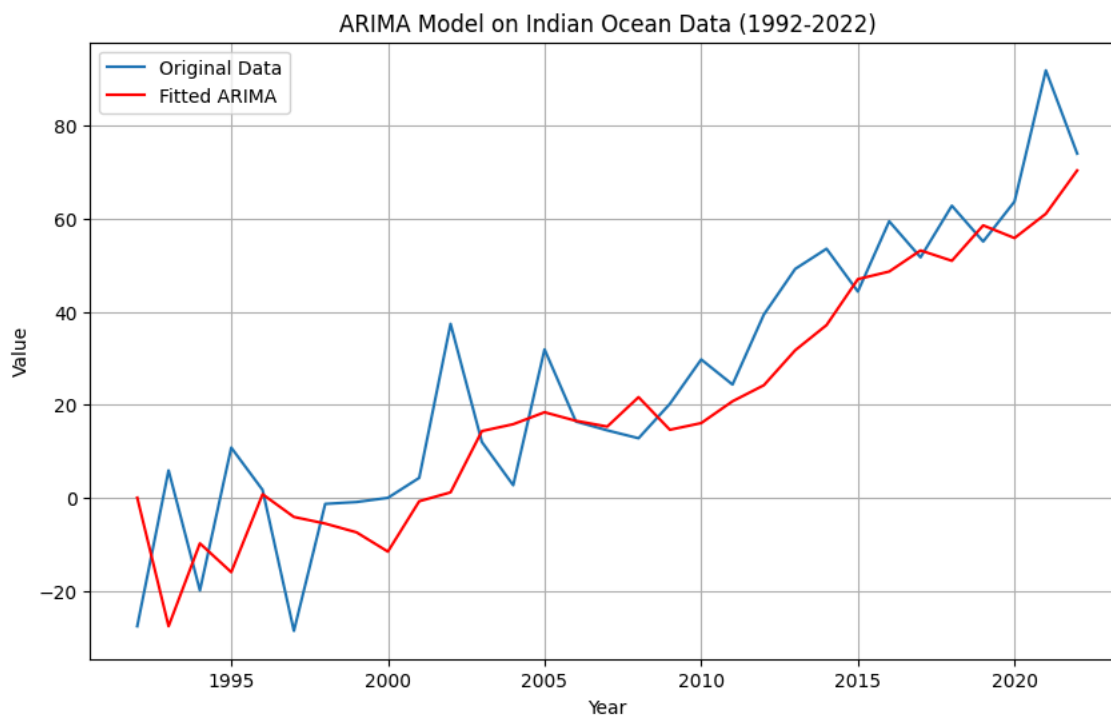
# Assuming indian_ocean_data is defined from the previous code
indian_ocean_data = yearly_changes.get('Indian Ocean', {})
values = list(indian_ocean_data.values())
values = [x for x in values if not pd.isna(x)]
years = list(indian_ocean_data.keys())

# Fit the ARIMA model (example order (2, 1, 0))
model = ARIMA(values, order=(2, 1, 0))
model_fit = model.fit()

# Plot the original data
plt.figure(figsize=(10, 6))
plt.plot(years, values, label='Original Data')

# Plot the fitted ARIMA model
plt.plot(years, model_fit.fittedvalues, label='Fitted ARIMA', color='red')

plt.xlabel('Year')
plt.ylabel('Value')
plt.title('ARIMA Model on Indian Ocean Data (1992-2022)')
plt.legend()
plt.grid(True)
plt.show()
```



```
model = ARIMA(values, order=(2, 1, 0))
model_fit = model.fit() # This line is crucial - it fits the model and assigns it to model_fit

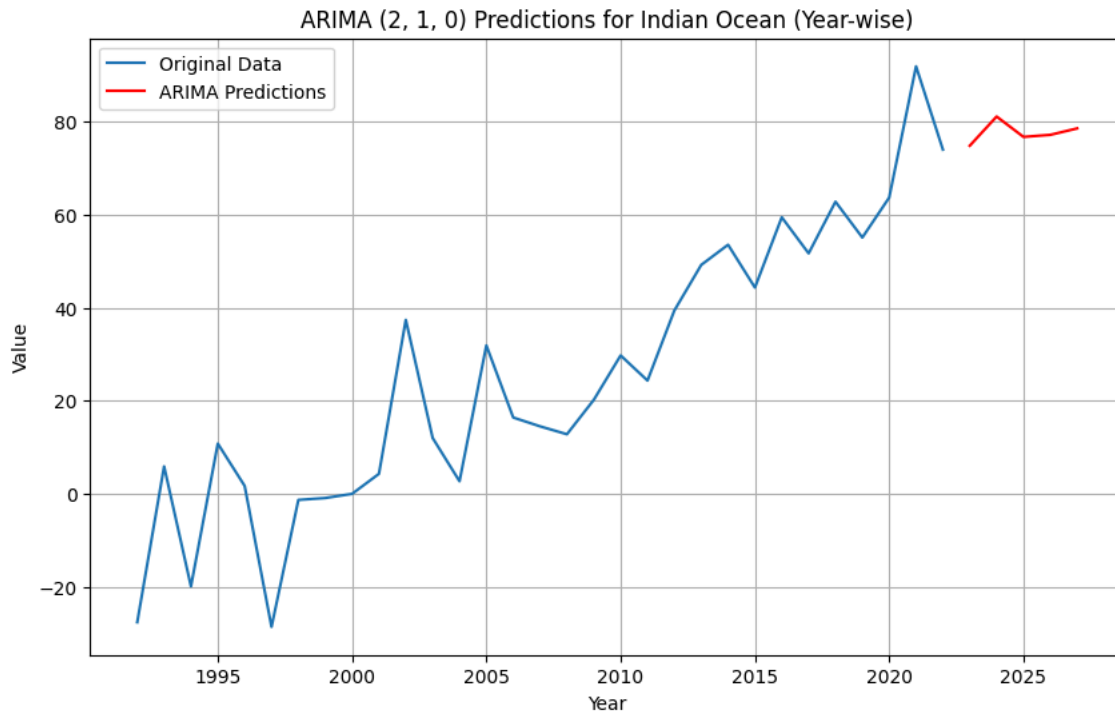
predictions = model_fit.predict(start=len(values), end=len(values) + 4)

# Plotting the original data and predictions year-wise
plt.figure(figsize=(10, 6))
plt.plot(range(1992, 2023), values, label='Original Data') # Plot original data with years
future_years = range(2023, 2028)
plt.plot(future_years, predictions, label='ARIMA Predictions', color='red') # Plot predictions

plt.xlabel('Year')
plt.ylabel('Value')
plt.title('ARIMA (2, 1, 0) Predictions for Indian Ocean (Year-wise)')
plt.legend()
plt.grid(True)
plt.show()

# Print the predictions year-wise (assuming the data starts from 1992)
for i, prediction in enumerate(predictions):
    year = 2023 + i # Since predictions start from 2023
```

```
print(f"Year: {year}, Predicted Value: {prediction}")
```



```
Year: 2023, Predicted Value: 74.80899936594646
Year: 2024, Predicted Value: 81.08379345353151
Year: 2025, Predicted Value: 76.70509012136324
Year: 2026, Predicted Value: 77.14779710312341
Year: 2027, Predicted Value: 78.52911964901244
```

```
bay_of_bengal_data = yearly_changes.get('Bay Bengal', {})
values = list(bay_of_bengal_data.values())
values = [x for x in values if not pd.isna(x)]
```

```
# Use auto_arma to find the best (p, d, q) order
stepwise_model = auto_arma(values, start_p=0, start_q=0,
                           max_p=5, max_q=5, m=12,
                           start_P=0, seasonal=True,
                           d=None, D=1, trace=True,
                           error_action='ignore',
                           suppress_warnings=True,
                           stepwise=True)
```

```
print(stepwise_model.summary())
```



```
Performing stepwise search to minimize aic
ARIMA(0,0,0)(0,1,1)[12] intercept : AIC=206.176, Time=0.41 sec
ARIMA(0,0,0)(0,1,0)[12] intercept : AIC=204.178, Time=0.05 sec
ARIMA(1,0,0)(1,1,0)[12] intercept : AIC=206.081, Time=1.37 sec
ARIMA(0,0,1)(0,1,1)[12] intercept : AIC=206.679, Time=4.49 sec
ARIMA(0,0,0)(0,1,0)[12] intercept : AIC=214.676, Time=0.09 sec
ARIMA(0,0,0)(1,1,0)[12] intercept : AIC=206.176, Time=0.86 sec
ARIMA(0,0,0)(1,1,1)[12] intercept : AIC=208.176, Time=0.36 sec
ARIMA(1,0,0)(0,1,0)[12] intercept : AIC=204.081, Time=0.18 sec
ARIMA(1,0,0)(0,1,1)[12] intercept : AIC=206.081, Time=1.11 sec
ARIMA(1,0,0)(1,1,1)[12] intercept : AIC=208.081, Time=0.28 sec
ARIMA(2,0,0)(0,1,0)[12] intercept : AIC=205.828, Time=0.15 sec
ARIMA(1,0,1)(0,1,0)[12] intercept : AIC=205.984, Time=0.15 sec
ARIMA(0,0,1)(0,1,0)[12] intercept : AIC=204.681, Time=0.11 sec
ARIMA(2,0,1)(0,1,0)[12] intercept : AIC=206.865, Time=0.21 sec
ARIMA(1,0,0)(0,1,0)[12] intercept : AIC=206.720, Time=0.04 sec
```

```
Best model: ARIMA(1,0,0)(0,1,0)[12] intercept
Total fit time: 9.886 seconds
```

SARIMAX Results

```
=====
Dep. Variable: y No. Observations: 31
Model: SARIMAX(1, 0, 0)x(0, 1, 0, 12) Log Likelihood -99.040
Date: Sat, 14 Dec 2024 AIC 204.081
Time: 03:36:12 BIC 206.914
Sample: 0 HQIC 204.560
- 31
Covariance Type: opg
=====
```

	coef	std err	z	P> z	[0.025	0.975]
intercept	30.4637	13.975	2.180	0.029	3.073	57.854
ar.L1	0.3213	0.277	1.158	0.247	-0.223	0.865


```

sigma2      1962.3022    948.219    2.069    0.039    103.827    3820.777
=====
Ljung-Box (L1) (Q):      0.02    Jarque-Bera (JB):      1.33
Prob(Q):      0.90    Prob(JB):      0.51
Heteroskedasticity (H):    1.10    Skew:      0.50
Prob(H) (two-sided):    0.91    Kurtosis:      2.17
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```

import matplotlib.pyplot as plt
from statsmodels.tsa.arima.model import ARIMA

# Assuming bay_of_bengal_data is defined as in your previous code
bay_of_bengal_data = yearly_changes.get('Bay Bengal', {})
values = list(bay_of_bengal_data.values())
values = [x for x in values if not pd.isna(x)]
years = list(bay_of_bengal_data.keys())
years = years[:len(values)]

# Fit the ARIMA model (example order (1, 0, 0))
model = ARIMA(values, order=(1, 0, 0))
model_fit = model.fit()

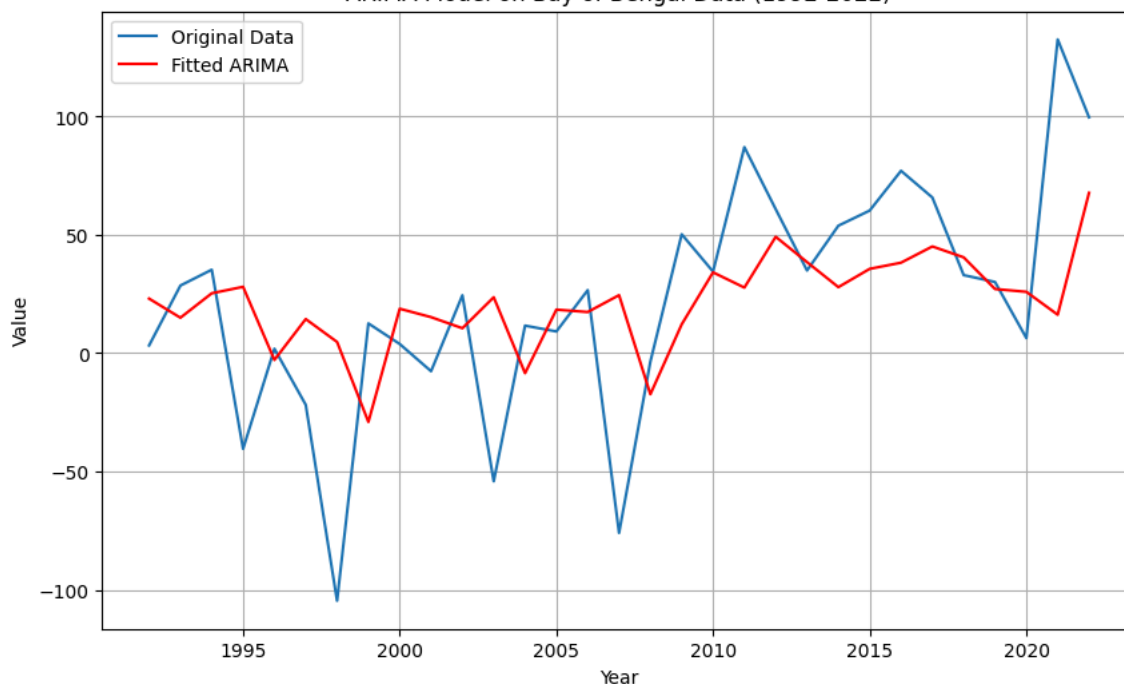
# Plot the original data and the fitted values
plt.figure(figsize=(10, 6))
plt.plot(years, values, label='Original Data')
plt.plot(years, model_fit.fittedvalues, label='Fitted ARIMA', color='red')

plt.xlabel('Year')
plt.ylabel('Value')
plt.title('ARIMA Model on Bay of Bengal Data (1992-2022)')
plt.legend()
plt.grid(True)
plt.show()

```



ARIMA Model on Bay of Bengal Data (1992-2022)



```

bay_of_bengal_data = yearly_changes.get('Bay Bengal', {})
values = list(bay_of_bengal_data.values())
values = [x for x in values if not pd.isna(x)]
years = list(bay_of_bengal_data.keys())
years = years[:len(values)]

# Fit the ARIMA(1, 0, 0) model
model = ARIMA(values, order=(1, 0, 0)) # (p, d, q) order
model_fit = model.fit()

# Make predictions for the next 5 years
predictions = model_fit.predict(start=len(values), end=len(values) + 4)

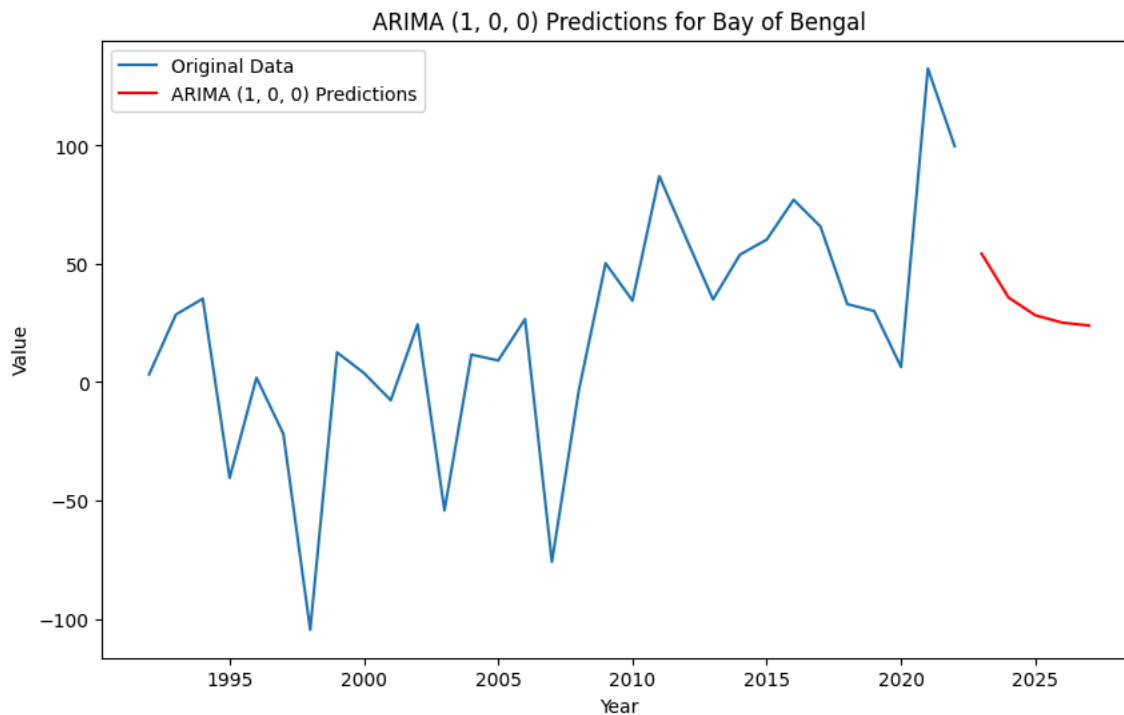
# Create a list of years for the predictions
future_years = list(range(years[-1] + 1, years[-1] + 6))

```

```
# Plotting
plt.figure(figsize=(10, 6))
plt.plot(years, values, label='Original Data')
plt.plot(future_years, predictions, label='ARIMA (1, 0, 0) Predictions', color='red')

plt.xlabel('Year')
plt.ylabel('Value')
plt.title('ARIMA (1, 0, 0) Predictions for Bay of Bengal')
plt.legend()
plt.show()

# Print the predictions with corresponding years
for year, prediction in zip(future_years, predictions):
    print(f"Predicted value for {year}: {prediction}")
```



```
Predicted value for 2023: 54.3249113938531
Predicted value for 2024: 35.814727471378234
Predicted value for 2025: 28.2604062392197
Predicted value for 2026: 25.177359116746214
Predicted value for 2027: 23.919115090939002
```

```
from statsmodels.tsa.arima.model import ARIMA

# Assuming 'arabian_sea_data' is defined as in your previous code
arabian_sea_data = yearly_changes.get('Arabian Sea', {})
values = list(arabian_sea_data.values())
values = [x for x in values if not pd.isna(x)]

# Use auto_arima to find the best (p, d, q) order
stepwise_model = auto_arima(values, start_p=0, start_q=0,
                             max_p=5, max_q=5, m=12,
                             start_P=0, seasonal=False,
                             d=None, D=1, trace=False,
                             error_action='ignore',
                             suppress_warnings=True,
                             stepwise=True)

print(stepwise_model.summary())
```



```
/usr/local/lib/python3.10/dist-packages/pmdarima/arima/_validation.py:62: UserWarning: m (12) set for non-seasonal fit. Setting to 6
warnings.warn("m (%i) set for non-seasonal fit. Setting to 0" % m)
```

```
SARIMAX Results
=====
Dep. Variable:          y      No. Observations:          31
Model:                SARIMAX(0, 1, 1)  Log Likelihood      -138.910
Date:                 Sat, 14 Dec 2024    AIC                  283.821
Time:                 03:36:29           BIC                  288.024
Sample:              0                HQIC                  285.166
                             - 31
Covariance Type:      opg
=====
coef    std err          z      P>|z|      [0.025    0.975]
-----
-----
```

```

intercept      2.9022      1.479      1.962      0.050      0.003      5.801
ma.L1          -0.7161      0.181     -3.951      0.000     -1.071     -0.361
sigma2         601.1656     165.203      3.639      0.000     277.374     924.958
=====
Ljung-Box (L1) (Q):                0.05   Jarque-Bera (JB):                0.43
Prob(Q):                          0.82   Prob(JB):                  0.81
Heteroskedasticity (H):            0.85   Skew:                      -0.28
Prob(H) (two-sided):              0.80   Kurtosis:                  2.84
=====

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```

```

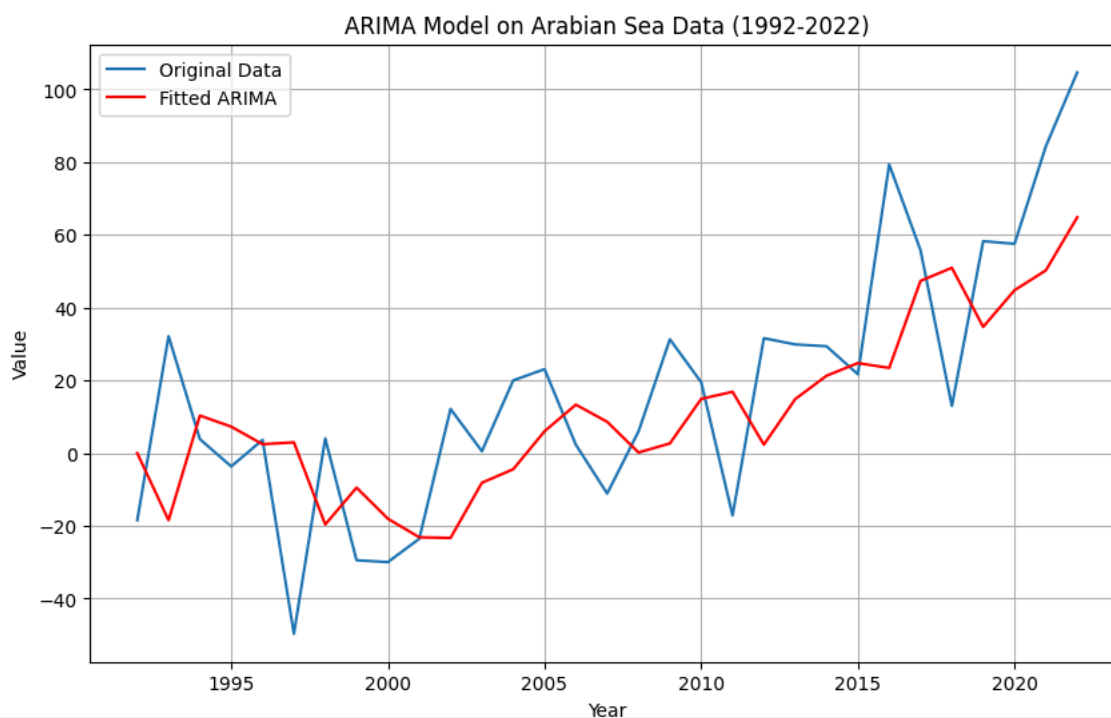
arabian_sea_data = yearly_changes.get('Arabian Sea', {})
values = list(arabian_sea_data.values())
values = [x for x in values if not pd.isna(x)]
years = list(arabian_sea_data.keys())
years = years[:len(values)]

# Fit the ARIMA model (example order (0, 1, 1))
model = ARIMA(values, order=(0, 1, 1))
model_fit = model.fit()

# Plot the original data and the fitted values
plt.figure(figsize=(10, 6))
plt.plot(years, values, label='Original Data')
plt.plot(years, model_fit.fittedvalues, label='Fitted ARIMA', color='red')

plt.xlabel('Year')
plt.ylabel('Value')
plt.title('ARIMA Model on Arabian Sea Data (1992-2022)')
plt.legend()
plt.grid(True)
plt.show()

```



```

arabian_sea_data = yearly_changes.get('Arabian Sea', {})
values = list(arabian_sea_data.values())
values = [x for x in values if not pd.isna(x)]
years = list(arabian_sea_data.keys())
years = years[:len(values)]

# Fit the ARIMA(0, 1, 1) model
model = ARIMA(values, order=(0, 1, 1))
model_fit = model.fit()

# Make predictions for the next 5 years
predictions = model_fit.predict(start=len(values), end=len(values) + 4)

# Create a list of years for the predictions
future_years = list(range(years[-1] + 1, years[-1] + 6))

# Plotting

```

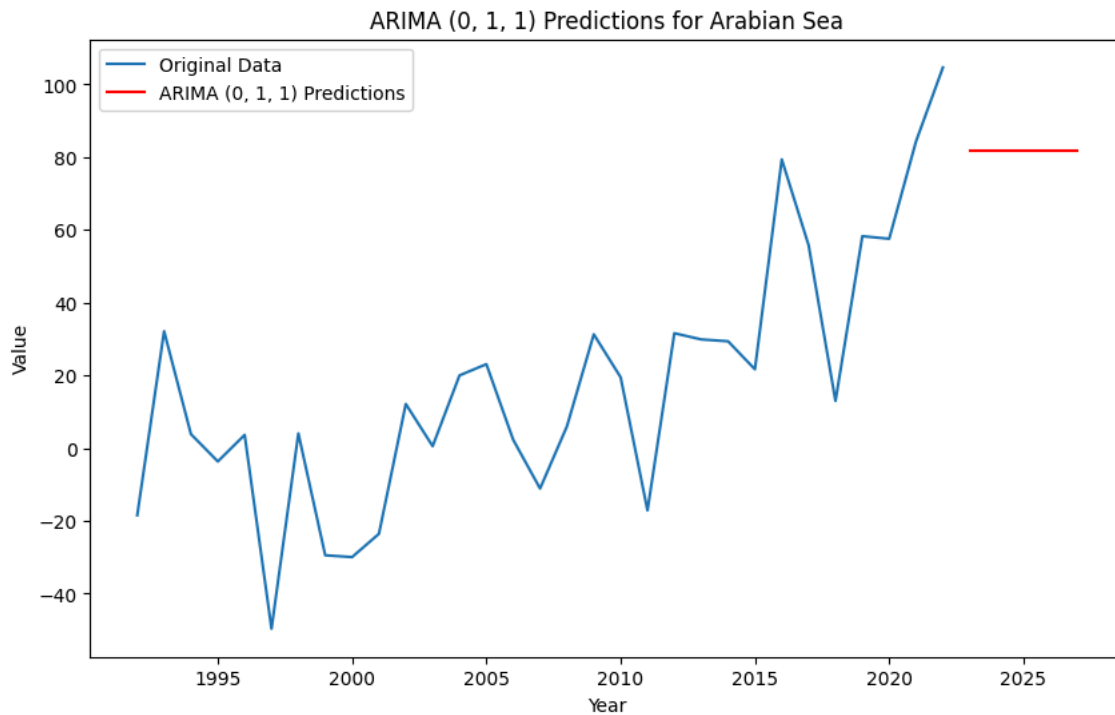
```

# Plotting
plt.figure(figsize=(10, 6))
plt.plot(years, values, label='Original Data')
plt.plot(future_years, predictions, label='ARIMA (0, 1, 1) Predictions', color='red')

plt.xlabel('Year')
plt.ylabel('Value')
plt.title('ARIMA (0, 1, 1) Predictions for Arabian Sea')
plt.legend()
plt.show()

# Print the predictions with corresponding years
print("Predictions for Arabian Sea:")
for year, prediction in zip(future_years, predictions):
    print(f"Predicted value for {year}: {prediction}")

```



```

Predictions for Arabian Sea:
Predicted value for 2023: 81.87949659111074
Predicted value for 2024: 81.87949659111074
Predicted value for 2025: 81.87949659111074
Predicted value for 2026: 81.87949659111074

```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
import numpy as np

```

```

def prepare_data(data):
    values = list(data.values())
    values = [x for x in values if not np.isnan(x)] # Remove NaN values
    # Normalize the data (important for LSTM)
    values = np.array(values) / np.max(values)

    # Create sequences for LSTM input
    sequence_length = 5 # Adjust as needed
    X, y = [], []
    for i in range(len(values) - sequence_length):
        X.append(values[i:i + sequence_length])
        y.append(values[i + sequence_length])
    X = np.array(X)
    y = np.array(y)
    return X, y

```

```

# Prepare data for each region
X_indian, y_indian = prepare_data(indian_ocean_data)
X_bay, y_bay = prepare_data(bay_of_bengal_data)
X_arabian, y_arabian = prepare_data(arabian_sea_data)

```

```

# Example LSTM model
def create_lstm_model():
    model = Sequential()
    model.add(LSTM(50, activation='relu', input_shape=(X_indian.shape[1], 1))) # Input shape adjusted

```

```

model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
return model

# Train the model for each region (example)
model_indian = create_lstm_model()
model_indian.fit(X_indian, y_indian, epochs=50, verbose=0) # Adjust epochs as needed

model_bay = create_lstm_model()
model_bay.fit(X_bay, y_bay, epochs=50, verbose=0)

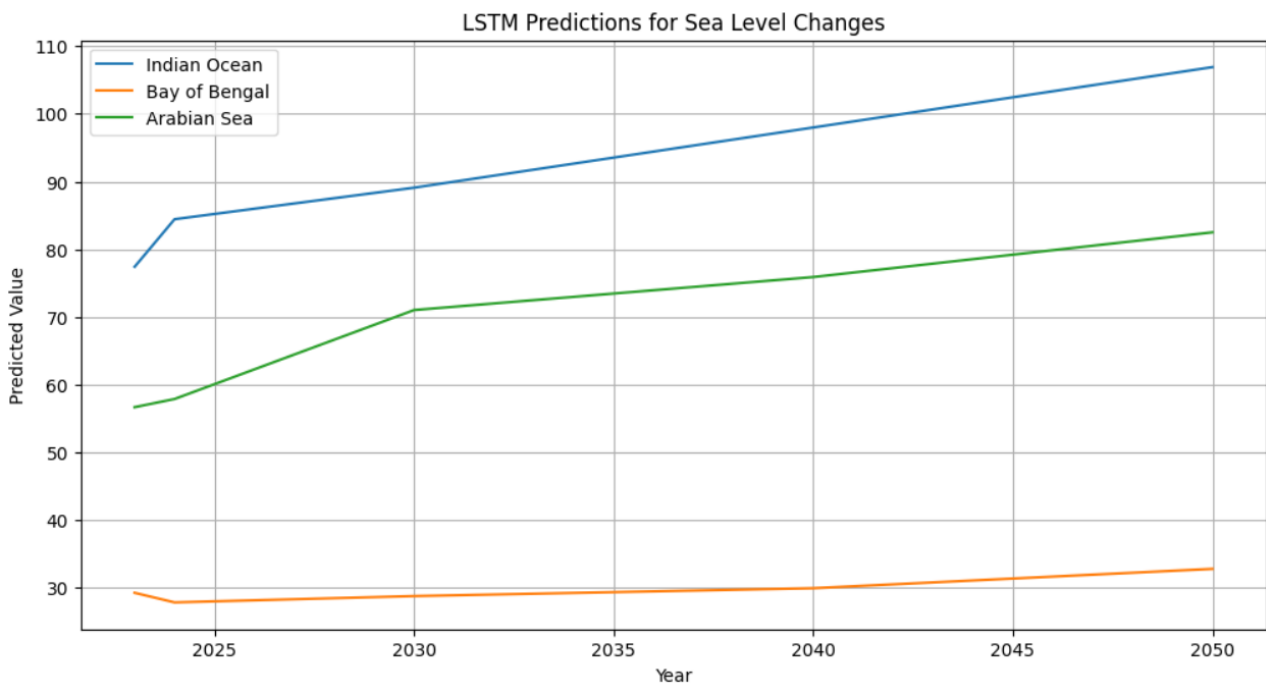
model_arabian = create_lstm_model()
model_arabian.fit(X_arabian, y_arabian, epochs=50, verbose=0)

# Function to predict values
def predict_future_values(model, X, years):
    predictions = []
    last_sequence = X[-1]
    for _ in range(len(years)):
        prediction = model.predict(np.array([last_sequence]))[0, 0]
        predictions.append(prediction * np.max(list(indian_ocean_data.values()))) # Denormalize
        last_sequence = np.append(last_sequence[1:], prediction) # Update the sequence
    return predictions

# Predict values for future years
future_years = [2023,2024,2030,2040,2050]
predicted_indian = predict_future_values(model_indian, X_indian, future_years)
print('Indian Ocean data :',predicted_indian)
predicted_bay = predict_future_values(model_bay, X_bay, future_years)
print('Bay of Bengal data :',predicted_bay)
predicted_arabian = predict_future_values(model_arabian, X_arabian, future_years)
print('Arabian Sea data :',predicted_arabian)
# Plot the predictions
plt.figure(figsize=(12, 6))
plt.plot(future_years, predicted_indian, label='Indian Ocean')
plt.plot(future_years, predicted_bay, label='Bay of Bengal')
plt.plot(future_years, predicted_arabian, label='Arabian Sea')

plt.xlabel('Year')
plt.ylabel('Predicted Value')
plt.title('LSTM Predictions for Sea Level Changes')
plt.legend()
plt.grid(True)
plt.show()

```



```

1/1 _____ 0s 153ms/step
1/1 _____ 0s 20ms/step
1/1 _____ 0s 20ms/step
1/1 _____ 0s 20ms/step
1/1 _____ 0s 21ms/step
Indian Ocean data : [77.41930939435959, 84.43952280461788, 89.09332427620888, 97.9837309718132, 106.89898720502853]
1/1 _____ 0s 164ms/step
1/1 _____ 0s 21ms/step
1/1 _____ 0s 20ms/step
1/1 _____ 0s 21ms/step
1/1 _____ 0s 27ms/step
Bay of Bengal data : [29.27138907402754, 27.855543135404588, 28.789355684518817, 29.95107430577278, 32.80738573372364]

```

```

print("LSTM Model Summary for Indian Ocean:")
model_indian.summary()

print("\nLSTM Model Summary for Bay of Bengal:")
model_bay.summary()

print("\nLSTM Model Summary for Arabian Sea:")
model_arabian.summary()

```

```

LSTM Model Summary for Indian Ocean:
Model: "sequential_3"

```

Layer (type)	Output Shape	Param #
lstm_3 (LSTM)	(None, 50)	10,400
dense_3 (Dense)	(None, 1)	51

```

Total params: 31,355 (122.48 KB)
Trainable params: 10,451 (40.82 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 20,904 (81.66 KB)

```

```

LSTM Model Summary for Bay of Bengal:
Model: "sequential_4"

```

Layer (type)	Output Shape	Param #
lstm_4 (LSTM)	(None, 50)	10,400
dense_4 (Dense)	(None, 1)	51

```

Total params: 31,355 (122.48 KB)
Trainable params: 10,451 (40.82 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 20,904 (81.66 KB)

```

```

LSTM Model Summary for Arabian Sea:
Model: "sequential_5"

```

Layer (type)	Output Shape	Param #
lstm_5 (LSTM)	(None, 50)	10,400
dense_5 (Dense)	(None, 1)	51

```

Total params: 31,355 (122.48 KB)
Trainable params: 10,451 (40.82 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 20,904 (81.66 KB)

```

Forest Area Cover Dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd
```

```
df = pd.read_csv('/content/drive/My Drive/Project sem1/Datasets/Forest_and_Carbon.csv')
```

```
# Now you can work with the dataframe 'df'
print(df.head())
```

```

ObjectId      Country ISO2  ISO3 \
0           1  Advanced Economies  NaN  AETMP
1           2  Advanced Economies  NaN  AETMP
2           3  Advanced Economies  NaN  AETMP
3           4  Advanced Economies  NaN  AETMP
4           5  Afghanistan, Islamic Rep. of  AF  AFG

Indicator      Unit \
0  Carbon stocks in forests  Million tonnes
1  Forest area  1000 HA
2  Index of carbon stocks in forests  Index
3  Index of forest extent  Index
4  Forest area  1000 HA

Source CTS_Code CTS_Name \
0  Food and Agriculture Organization of the Unite...  ECMFC  NaN
1  Food and Agriculture Organization of the Unite...  ECMFF  NaN
2  Food and Agriculture Organization of the Unite...  ECMFS  NaN
3  Food and Agriculture Organization of the Unite...  ECMFE  NaN
4  Food and Agriculture Organization of the Unite...  ECMFF  NaN

CTS_Full_Descriptor  ...  F2013 \
0  Environment, Climate Change, Mitigation, Fores...  ...  56891.748000
1  Environment, Climate Change, Mitigation, Fores...  ...  958658.324000
2  Environment, Climate Change, Mitigation, Fores...  ...  108.120860
3  Environment, Climate Change, Mitigation, Fores...  ...  101.355167
4  Environment, Climate Change, Mitigation, Fores...  ...  1208.440000

F2014      F2015      F2016      F2017      F2018 \
0  57114.846600  57337.946100  57568.174400  57777.849900  57959.762500
1  959818.234200  960978.144400  962013.845500  961845.120000  961963.710000
2  108.544852  108.968845  109.406386  109.804867  110.150586
3  101.477799  101.600432  101.709933  101.692094  101.704632
4  1208.440000  1208.440000  1208.440000  1208.440000  1208.440000

F2019      F2020      F2021      F2022
0  58035.051300  58102.552100  NaN  NaN
1  962092.030000  962242.860000  962383.873400  962531.435600
2  110.293670  110.421953  NaN  NaN
3  101.718199  101.734145  101.749054  101.764655
4  1208.440000  1208.440000  1208.440000  1208.440000

```

[5 rows x 41 columns]

```
df = df.drop('ISO2', axis=1)
df = df.drop('CTS_Name', axis=1)
```

```
print(df.head())
```

```

➡ ObjectId          Country  ISO3 \
0      1      Advanced Economies  AETMP
1      2      Advanced Economies  AETMP
2      3      Advanced Economies  AETMP
3      4      Advanced Economies  AETMP
4      5  Afghanistan, Islamic Rep. of  AFG

          Indicator          Unit \
0      Carbon stocks in forests  Million tonnes
1      Forest area          1000 HA
2  Index of carbon stocks in forests      Index
3      Index of forest extent      Index
4      Forest area          1000 HA

          Source  CTS_Code \
0  Food and Agriculture Organization of the Unite...  ECMFC
1  Food and Agriculture Organization of the Unite...  ECMFF
2  Food and Agriculture Organization of the Unite...  ECMFS
3  Food and Agriculture Organization of the Unite...  ECMFE
4  Food and Agriculture Organization of the Unite...  ECMFF

          CTS_Full_Descriptor      F1992 \
0  Environment, Climate Change, Mitigation, Fores...  52618.6601
1  Environment, Climate Change, Mitigation, Fores...  945840.6089
2  Environment, Climate Change, Mitigation, Fores...  100.0000
3  Environment, Climate Change, Mitigation, Fores...  100.0000
4  Environment, Climate Change, Mitigation, Fores...  1208.4400

          F1993  ...      F2013      F2014      F2015 \
0  53177.699800  ...  56891.748000  57114.846600  57337.946100
1  950628.754500  ...  958658.324000  959818.234200  960978.144400
2  101.062436  ...  108.120860  108.544852  108.968845
3  100.506232  ...  101.355167  101.477799  101.600432
4  1208.440000  ...  1208.440000  1208.440000  1208.440000

          F2016      F2017      F2018      F2019      F2020 \
0  57568.174400  57777.849900  57959.762500  58035.051300  58102.552100
1  962013.845500  961845.120000  961963.710000  962092.030000  962242.860000
2  109.406386  109.804867  110.150586  110.293670  110.421953
3  101.709933  101.692094  101.704632  101.718199  101.734145
4  1208.440000  1208.440000  1208.440000  1208.440000  1208.440000

          F2021      F2022
0      NaN      NaN
1  962383.873400  962531.435600
2      NaN      NaN
3  101.749054  101.764655
4  1208.440000  1208.440000
```

```
[5 rows x 39 columns]
```

```
df.dtypes
```




0

ObjectId	int64
Country	object
ISO3	object
Indicator	object
Unit	object
Source	object
CTS_Code	object
CTS_Full_Descriptor	object
F1992	float64
F1993	float64
F1994	float64
F1995	float64
F1996	float64
F1997	float64
F1998	float64
F1999	float64
F2000	float64
F2001	float64
F2002	float64
F2003	float64
F2004	float64
F2005	float64
F2006	float64
F2007	float64
F2008	float64
F2009	float64
F2010	float64
F2011	float64
F2012	float64
F2013	float64
F2014	float64

```
# Rename columns  
new_columns = {}
```

```
for col in df.columns:
    if col.startswith('F') and col[1:].isdigit():
        new_columns[col] = col[1:]
df = df.rename(columns=new_columns)

print(df.head())
df.dtypes
```



	ObjectId	Country	ISO3	\
0	1	Advanced Economies	AETMP	
1	2	Advanced Economies	AETMP	
2	3	Advanced Economies	AETMP	
3	4	Advanced Economies	AETMP	
4	5	Afghanistan, Islamic Rep. of	AFG	

	Indicator	Unit	\
0	Carbon stocks in forests	Million tonnes	
1	Forest area	1000 HA	
2	Index of carbon stocks in forests	Index	
3	Index of forest extent	Index	
4	Forest area	1000 HA	

	Source	CTS_Code	\
0	Food and Agriculture Organization of the Unite...	ECMFC	
1	Food and Agriculture Organization of the Unite...	ECMFF	
2	Food and Agriculture Organization of the Unite...	ECMFS	
3	Food and Agriculture Organization of the Unite...	ECMFE	
4	Food and Agriculture Organization of the Unite...	ECMFF	

	CTS_Full_Descriptor	1992	\
0	Environment, Climate Change, Mitigation, Fores...	52618.6601	
1	Environment, Climate Change, Mitigation, Fores...	945840.6089	
2	Environment, Climate Change, Mitigation, Fores...	100.0000	
3	Environment, Climate Change, Mitigation, Fores...	100.0000	
4	Environment, Climate Change, Mitigation, Fores...	1208.4400	

	1993	...	2013	2014	2015	\
0	53177.699800	...	56891.748000	57114.846600	57337.946100	
1	950628.754500	...	958658.324000	959818.234200	960978.144400	
2	101.062436	...	108.120860	108.544852	108.968845	
3	100.506232	...	101.355167	101.477799	101.600432	
4	1208.440000	...	1208.440000	1208.440000	1208.440000	

	2016	2017	2018	2019	2020	\
0	57568.174400	57777.849900	57959.762500	58035.051300	58102.552100	
1	962013.845500	961845.120000	961963.710000	962092.030000	962242.860000	
2	109.406386	109.804867	110.150586	110.293670	110.421953	
3	101.709933	101.692094	101.704632	101.718199	101.734145	
4	1208.440000	1208.440000	1208.440000	1208.440000	1208.440000	

	2021	2022
0	NaN	NaN
1	962383.873400	962531.435600
2	NaN	NaN
3	101.749054	101.764655
4	1208.440000	1208.440000

[5 rows x 39 columns]

0

ObjectId int64

Country object

ISO3 object

```
# Find missing values where country is India and indicator is forest area
forest_area_india = df[(df['Country'] == 'India') & (df['Indicator'] == 'Forest area')].i
```

forest_area_india



0

ObjectId 0

Country 0

ISO3 0

Indicator 0

Unit 0

Source 0

CTS_Code 0

CTS_Full_Descriptor 0

1992 0

1993 0

1994 0

1995 0

1996 0

1997 0

1998 0

1999 0

2000 0

2001 0

2002 0

2003 0

2004 0

2005 0

2006 0

2007 0

2008 0

2009 0

2010 0

2011 0

2012 0

2013 0

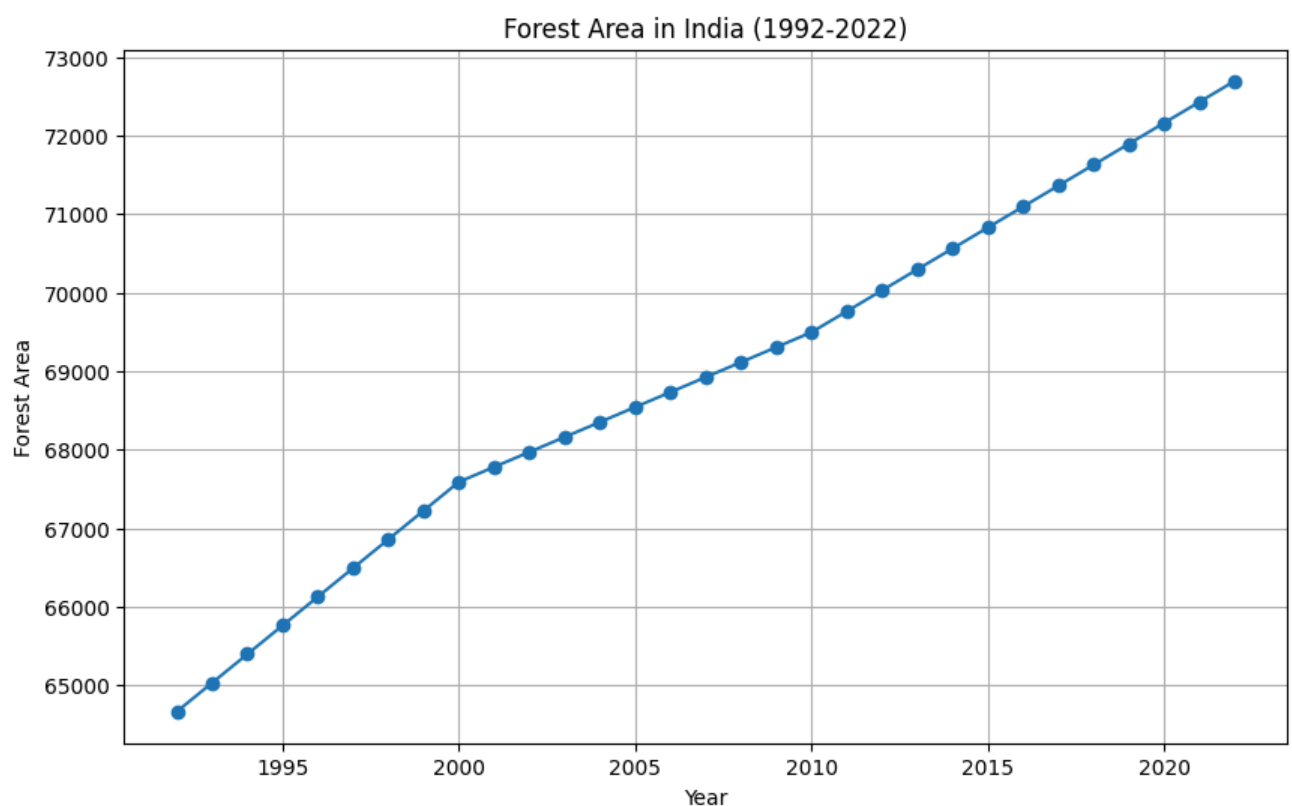
2014 0

at64

```
import matplotlib.pyplot as plt
# Filter data for India and Forest Area
india_forest_data = df[(df['Country'] == 'India') & (df['Indicator'] == 'Forest area')]

# Extract years and forest area values
years = [int(col) for col in india_forest_data.columns if col.isdigit() and 1992 <= int(c
forest_area = [india_forest_data[str(year)].iloc[0] for year in years]

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(years, forest_area, marker='o')
plt.xlabel('Year')
plt.ylabel('Forest Area')
plt.title('Forest Area in India (1992-2022)')
plt.grid(True)
plt.show()
```



```
print(india_forest_data)
```



ObjectId	Country	ISO3	Indicator	Unit	\
547	548	India	IND	Forest area	1000 HA
Source CTS_Code \					
547	Food and Agriculture Organization of the Unite...				ECMFF

	CTS_Full_Descriptor	1992	1993	...	\
547	Environment, Climate Change, Mitigation, Fores...	64668.6	65033.9	...	

	2013	2014	2015	2016	2017	2018	2019	2020	\
547	70295.2	70561.6	70828.0	71094.4	71360.8	71627.2	71893.6	72160.0	

	2021	2022
547	72426.4	72692.8

[1 rows x 39 columns]

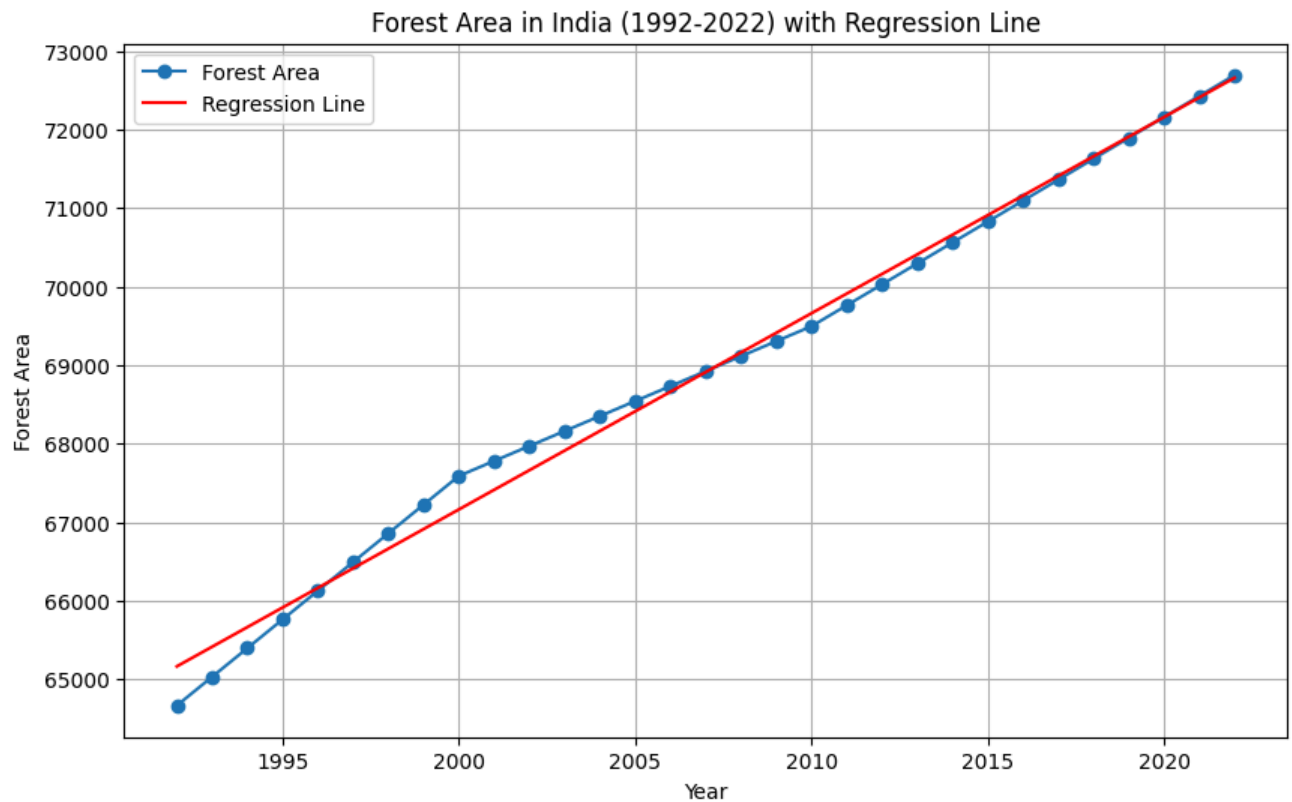
```
import numpy as np
from scipy.stats import linregress

# Calculate linear regression
slope, intercept, r_value, p_value, std_err = linregress(years, forest_area)

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(years, forest_area, marker='o', label='Forest Area')

# Add regression line
plt.plot(years, intercept + slope * np.array(years), 'r', label='Regression Line')

plt.xlabel('Year')
plt.ylabel('Forest Area')
plt.title('Forest Area in India (1992-2022) with Regression Line')
plt.grid(True)
plt.legend()
plt.show()
```



```
# Print the regression results
print(f"Slope: {slope}")
print(f"Intercept: {intercept}")
print(f"R-value: {r_value*100}%")
print(f"P-value: {p_value}")
print(f"Standard Error: {std_err}")

# R-squared value
r_squared = r_value**2
print(f"R-squared: {r_squared}")

# Summary of the linear regression
print("\nSummary of Linear Regression:")
print(f"The linear regression model shows a relationship between the year and forest area")
print(f"The slope ({slope:.2f}) indicates the average annual change in forest area.")
print(f"A positive slope suggests an increasing trend, while a negative slope suggests a")
print(f"The intercept ({intercept:.2f}) represents the estimated forest area at year 0 (n")
print(f"The R-squared value ({r_squared:.2f}) indicates the proportion of variance in for")
print(f"A higher R-squared value indicates a better fit of the model.")
print(f"The p-value ({p_value:.3f}) indicates the statistical significance of the relatic
```



Slope: 249.69532258064515
Intercept: -432226.0317741935
R-value: 99.58763709339708%
P-value: 8.753224918248456e-32
Standard Error: 4.223889949234077
R-squared: 0.9917697461846157

Summary of Linear Regression:

The linear regression model shows a relationship between the year and forest area in India. The slope (249.70) indicates the average annual change in forest area.

A positive slope suggests an increasing trend, while a negative slope suggests a decreasing trend. The intercept (-432226.03) represents the estimated forest area at year 0 (not meaningful in this context). The R-squared value (0.99) indicates the proportion of variance in forest area explained by the model. A higher R-squared value indicates a better fit of the model.

The p-value (0.000) indicates the statistical significance of the relationship. A small p-value suggests a strong relationship.

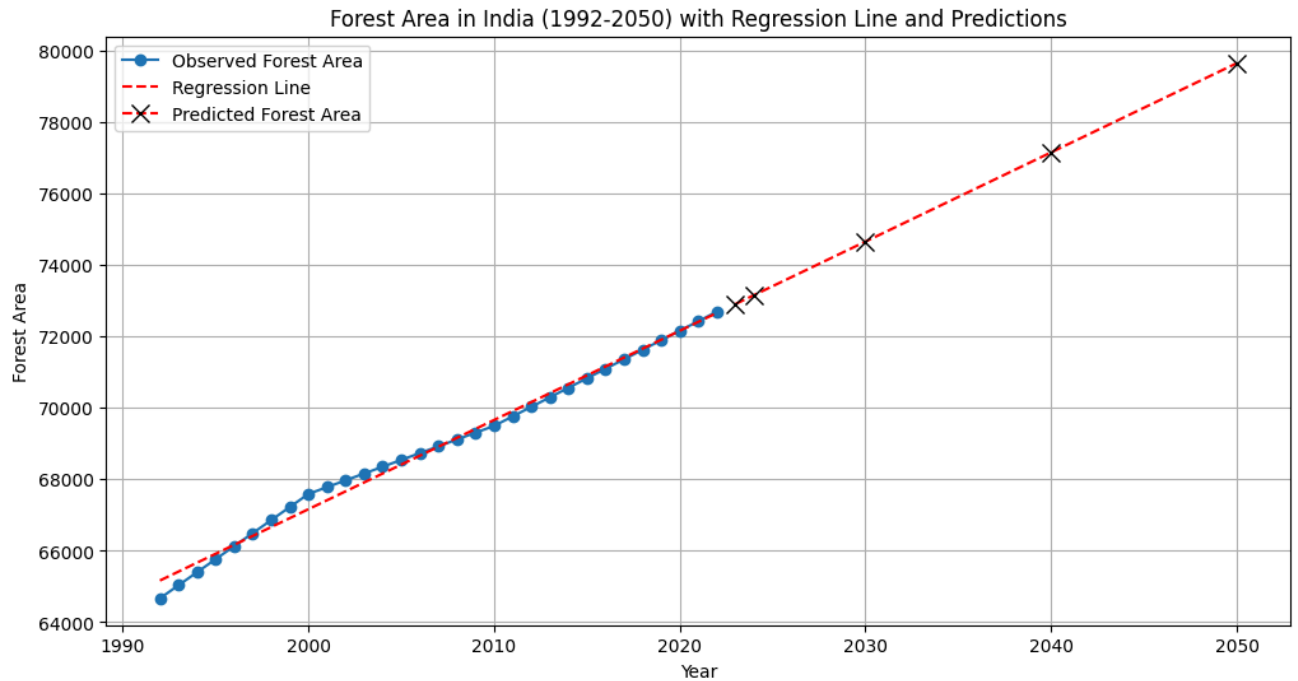


```
# Predict forest area for future years
future_years = [2023, 2024, 2030, 2040, 2050]
predicted_forest_area = intercept + slope * np.array(future_years)

# Extend the plot with predicted values
plt.figure(figsize=(12, 6))
plt.plot(years, forest_area, marker='o', label='Observed Forest Area')
plt.plot(years, intercept + slope * np.array(years), 'r', label='Regression Line', linestyle='solid')
plt.plot(future_years, predicted_forest_area, marker='x', linestyle='--', color='r', label='Predicted Forest Area')

plt.xlabel('Year')
plt.ylabel('Forest Area')
plt.title('Forest Area in India (1992-2050) with Regression Line and Predictions')
plt.grid(True)
plt.legend()
plt.show()

# Print the predicted values
for year, area in zip(future_years, predicted_forest_area):
    print(f"Predicted Forest Area for {year}: {area:.2f}")
```

Predicted Forest Area for 2023: 72907.61
Predicted Forest Area for 2024: 73157.30
Predicted Forest Area for 2030: 74655.47
Predicted Forest Area for 2040: 77152.43
Predicted Forest Area for 2050: 79649.38

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Calculate autocorrelation and partial autocorrelation
acf_vals = np.correlate(forest_area, forest_area, mode='full')

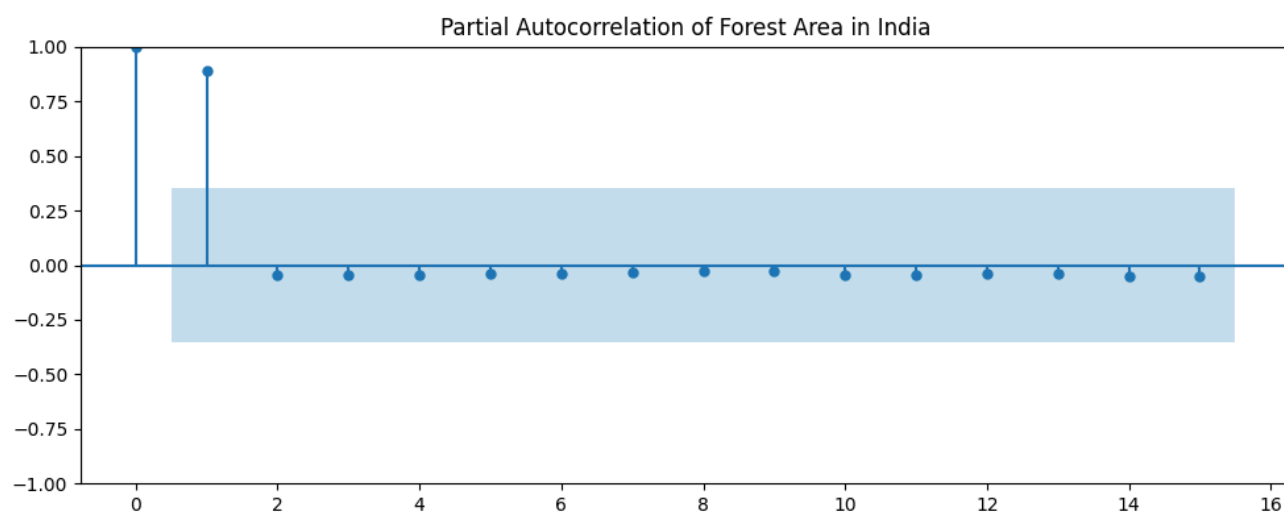
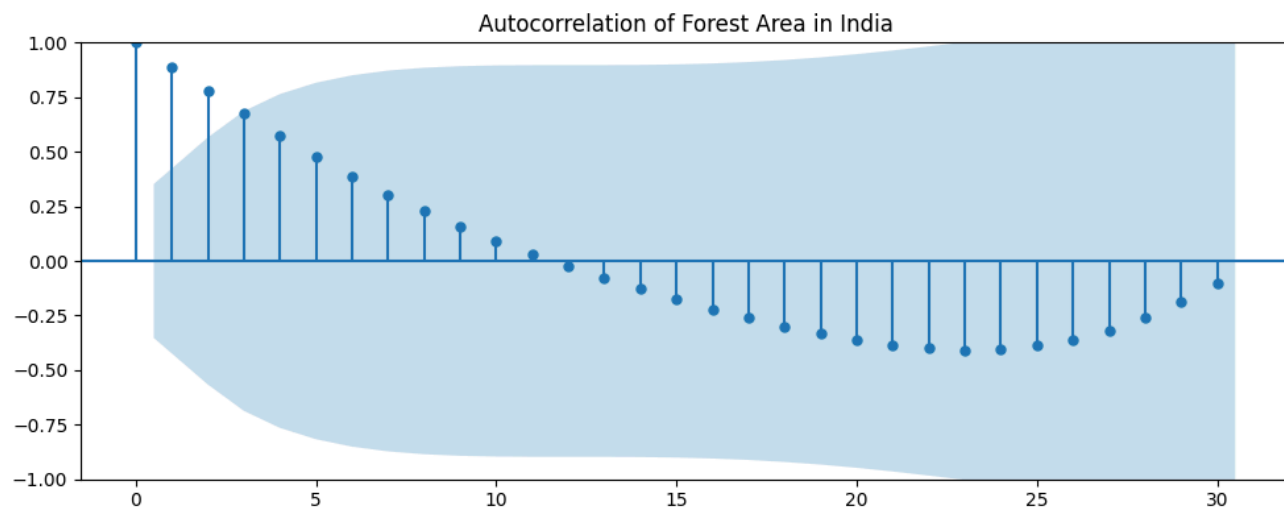
# Plotting
fig, axes = plt.subplots(2, 1, figsize=(10, 8))

# Autocorrelation plot using statsmodels
plot_acf(forest_area, lags=len(years)-1, ax=axes[0])
axes[0].set_title('Autocorrelation of Forest Area in India')

# Partial autocorrelation plot using statsmodels
# Limit lags to 50% of the sample size
max_lags = len(years) // 2 # Calculate maximum allowed lags
plot_pacf(forest_area, lags=max_lags, ax=axes[1]) # Changed to calculate maximum lags
```

```
axes[1].set_title('Partial Autocorrelation of Forest Area in India')
```

```
plt.tight_layout()  
plt.show()
```



```
from statsmodels.tsa.stattools import adfuller  
  
for d in range(3):  
    if d > 0:  
        differenced_series = np.diff(forest_area, n=d)
```

```

else:
    differenced_series = forest_area

result = adfuller(differenced_series)
print(f"Results for d = {d}")
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f' % (key, value))

if result[1] <= 0.05:
    print("Reject the null hypothesis. Time series is stationary.")
else:
    print("Fail to reject the null hypothesis. Time series is not stationary.")
print("-" * 50)

```

```

➡ Results for d = 0
ADF Statistic: -0.102038
p-value: 0.949235
Critical Values:
    1%: -3.679
    5%: -2.968
    10%: -2.623
Fail to reject the null hypothesis. Time series is not stationary.
-----
Results for d = 1
ADF Statistic: -5.346733
p-value: 0.000004
Critical Values:
    1%: -3.809
    5%: -3.022
    10%: -2.651
Reject the null hypothesis. Time series is stationary.
-----
Results for d = 2
ADF Statistic: -5.749507
p-value: 0.000001
Critical Values:
    1%: -3.833
    5%: -3.031
    10%: -2.656
Reject the null hypothesis. Time series is stationary.
-----

```

```

!pip install pmdarima
from statsmodels.tsa.arima.model import ARIMA
import pmdarima as pm
# Automatic ARIMA model selection using pmdarima
auto_arima_model = pm.auto_arima(forest_area,
                                start_p=0, start_q=0,
                                test='adf',
                                max_p=5, max_q=5, m=1, # Set appropriate max values
                                d=None, # Let auto_arima determine 'd'
                                seasonal=False, # No seasonality assumed in this examp
                                start_P=0,

```

```

D=0,
trace=True,
error_action='ignore',
suppress_warnings=True,
stepwise=True)

print(auto_arima_model.summary())

# Access the p, d, and q values:
p, d, q = auto_arima_model.order
print(f"Optimal p, d, q values: p={p}, d={d}, q={q}")

```



Collecting pmdarima

```

Downloading pmdarima-2.0.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.m
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/local/lib/pyt
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: statsmodels>=0.13.2 in /usr/local/lib/python3.10/dist-
Requirement already satisfied: urllib3 in /usr/local/lib/python3.10/dist-packages (fr
Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.10/di
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packa
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (f
Downloading pmdarima-2.0.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.man
2.1/2.1 MB 21.7 MB/s eta 0:00:00

```

Installing collected packages: pmdarima

Successfully installed pmdarima-2.0.4

Performing stepwise search to minimize aic

```

ARIMA(0,2,0)(0,0,0)[0] intercept : AIC=292.877, Time=0.07 sec
ARIMA(1,2,0)(0,0,0)[0] intercept : AIC=294.869, Time=0.05 sec
ARIMA(0,2,1)(0,0,0)[0] intercept : AIC=294.869, Time=0.56 sec
ARIMA(0,2,0)(0,0,0)[0]           : AIC=291.147, Time=0.02 sec
ARIMA(1,2,1)(0,0,0)[0] intercept : AIC=296.856, Time=0.99 sec

```

Best model: ARIMA(0,2,0)(0,0,0)[0]

Total fit time: 1.720 seconds

SARIMAX Results

```

=====
Dep. Variable:          y      No. Observations:          31
Model:                SARIMAX(0, 2, 0)  Log Likelihood      -144.574
Date:                 Sat, 14 Dec 2024  AIC                291.147
Time:                  03:59:23      BIC                292.514
Sample:                0            HQIC                291.575
                        - 31
Covariance Type:        opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
sigma2	1252.2601	103.327	12.119	0.000	1049.743	1454.777
Ljung-Box (L1) (Q):	0.00					420.07
Prob(Q):	0.96					0.00
Jarque-Bera (JB):						
Prob(JB):						

Heteroskedasticity (H):	0.00	Skew:	-3.58
Prob(H) (two-sided):	0.00	Kurtosis:	20.22
=====			

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).
Optimal p, d, q values: p=0, d=2, q=0

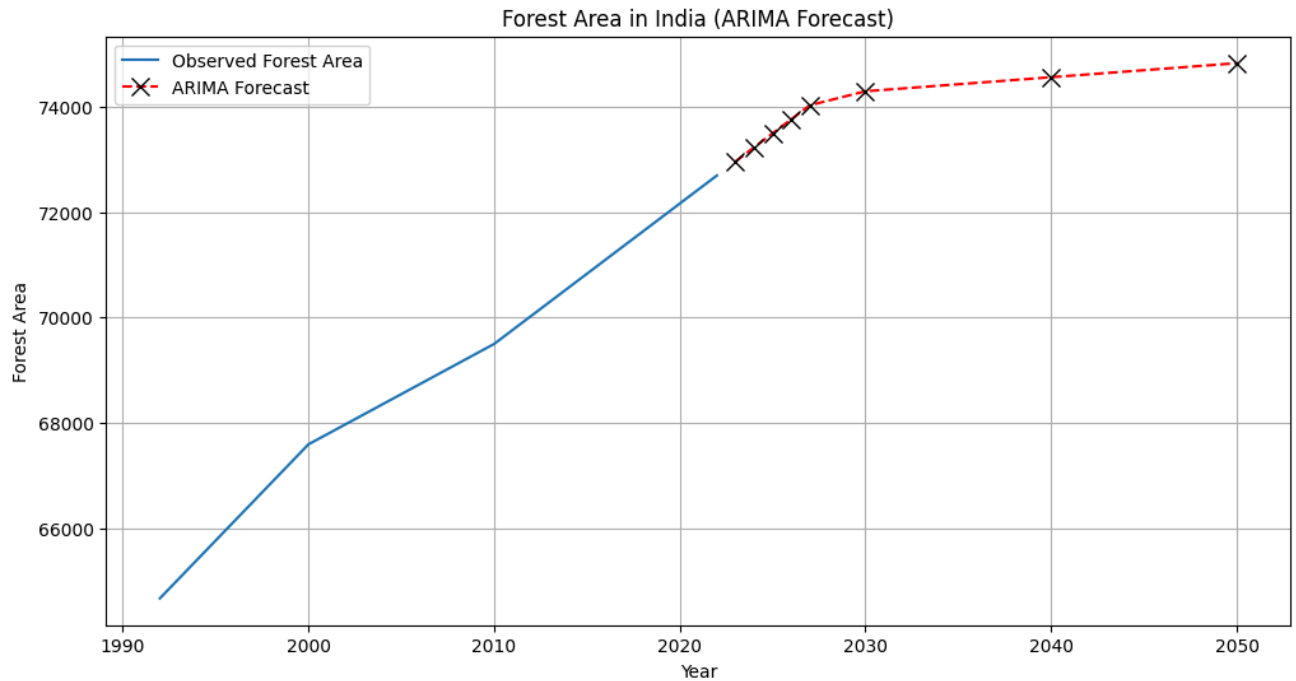
```
model = ARIMA(forest_area, order=(0, 2, 0))
model_fit = model.fit()

# Forecast future values
future_years = [2023, 2024, 2025, 2026, 2027, 2030, 2040, 2050]
forecast = model_fit.predict(start=len(forest_area), end=len(forest_area) + len(future_years),
                             step=1)

# Plotting the forecast
plt.figure(figsize=(12, 6))
plt.plot(years, forest_area, label='Observed Forest Area')
plt.plot(future_years, forecast, label='ARIMA Forecast', marker='x', linestyle='--', color='red')

plt.xlabel('Year')
plt.ylabel('Forest Area')
plt.title('Forest Area in India (ARIMA Forecast)')
plt.legend()
plt.grid(True)
plt.show()

# Print the forecasted values
for year, area in zip(future_years, forecast):
    print(f"Forecasted Forest Area for {year}: {area:.2f}")
```



Forecasted Forest Area for 2023: 72959.20
Forecasted Forest Area for 2024: 73225.60
Forecasted Forest Area for 2025: 73492.00
Forecasted Forest Area for 2026: 73758.40
Forecasted Forest Area for 2027: 74024.80
Forecasted Forest Area for 2030: 74291.20
Forecasted Forest Area for 2040: 74557.60
Forecasted Forest Area for 2050: 74824.00

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Data scaling
scaler = MinMaxScaler()
forest_area_scaled = scaler.fit_transform(np.array(forest_area).reshape(-1, 1))

# Prepare data for LSTM
def create_dataset(dataset, look_back=1):
    X, Y = [], []
    for i in range(len(dataset)-look_back-1):
        a = dataset[i:(i+look_back), 0]
```

```

        X.append(a)
        Y.append(dataset[i + look_back, 0])
    return np.array(X), np.array(Y)

look_back = 5
X, Y = create_dataset(forest_area_scaled, look_back)
X = np.reshape(X, (X.shape[0], X.shape[1], 1))

# LSTM model
model = Sequential()
model.add(LSTM(units=50, return_sequences=True, input_shape=(X.shape[1], 1)))
model.add(LSTM(units=50))
model.add(Dense(1))
model.compile(loss='mean_squared_error', optimizer='adam')
model.fit(X, Y, epochs=100, batch_size=1, verbose=2)

# Predictions
future_years = [2023, 2024, 2025, 2026, 2027, 2030, 2040, 2050]
inputs = forest_area_scaled[-look_back:]
predictions = []

for i in range(len(future_years)):
    input_resaped = np.reshape(inputs, (1, look_back, 1))
    prediction = model.predict(input_resaped, verbose=0)
    predictions.append(prediction[0,0])
    inputs = np.append(inputs[1:], prediction)

# Inverse transform to get actual values
predictions = scaler.inverse_transform(np.array(predictions).reshape(-1, 1))

```

```

➡ /usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/rnn.py:204: UserWarni
    super().__init__(**kwargs)
Epoch 1/100
25/25 - 10s - 397ms/step - loss: 0.1434
Epoch 2/100
25/25 - 0s - 13ms/step - loss: 0.0129
Epoch 3/100
25/25 - 1s - 24ms/step - loss: 0.0026
Epoch 4/100
25/25 - 0s - 9ms/step - loss: 0.0010
Epoch 5/100
25/25 - 0s - 15ms/step - loss: 8.2774e-04
Epoch 6/100
25/25 - 0s - 14ms/step - loss: 8.0157e-04
Epoch 7/100
25/25 - 0s - 10ms/step - loss: 4.6154e-04
Epoch 8/100
25/25 - 0s - 11ms/step - loss: 3.4955e-04
Epoch 9/100
25/25 - 0s - 16ms/step - loss: 3.4022e-04
Epoch 10/100
25/25 - 1s - 22ms/step - loss: 4.0881e-04
Epoch 11/100
25/25 - 0s - 13ms/step - loss: 3.7897e-04
Epoch 12/100

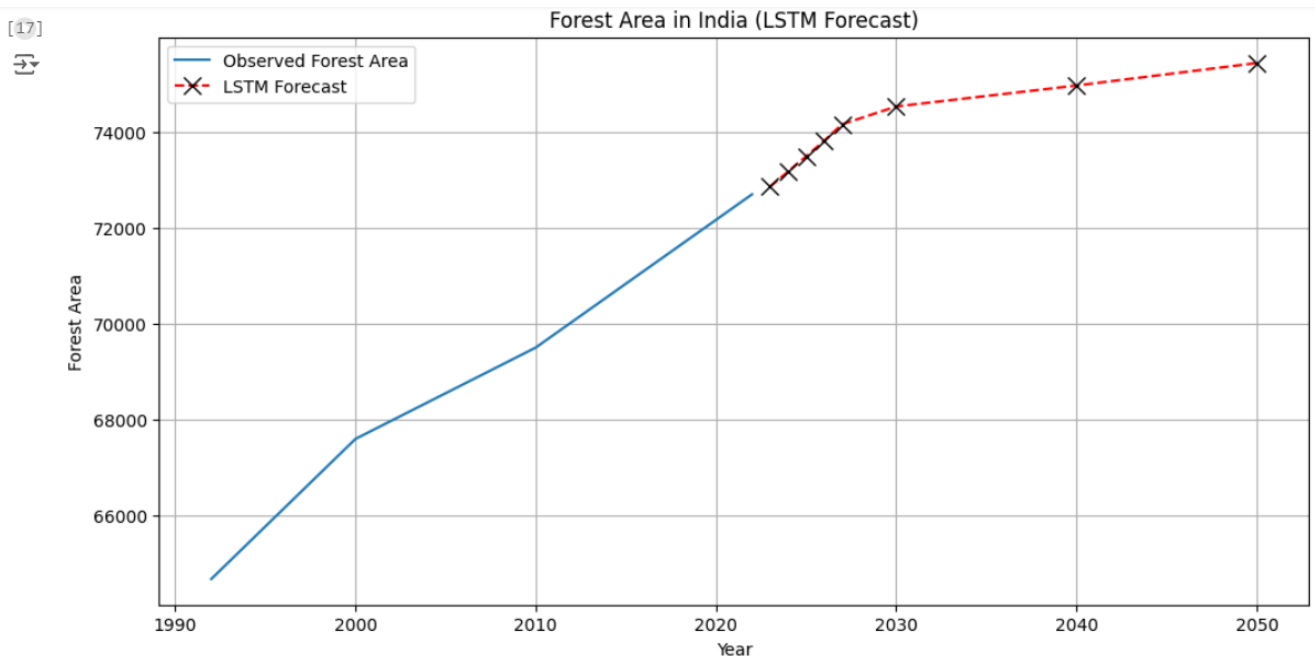
```

```
25/25 - 0s - 10ms/step - loss: 4.5805e-04
Epoch 13/100
25/25 - 0s - 8ms/step - loss: 6.0975e-04
Epoch 14/100
25/25 - 0s - 6ms/step - loss: 0.0013
Epoch 15/100
```

```
# Plotting
plt.figure(figsize=(12, 6))
plt.plot(years, forest_area, label='Observed Forest Area')
plt.plot(future_years, predictions, label='LSTM Forecast', marker='x', linestyle='--', color='r', markeredgecolor='black',ms='10')

plt.xlabel('Year')
plt.ylabel('Forest Area')
plt.title('Forest Area in India (LSTM Forecast)')
plt.legend()
plt.grid(True)
plt.show()

# Print predictions
for year, area in zip(future_years, predictions):
    print(f"Forecasted Forest Area for {year}: {area[0]:.2f}")
```



```
Forecasted Forest Area for 2023: 72853.39
Forecasted Forest Area for 2024: 73167.20
Forecasted Forest Area for 2025: 73483.06
Forecasted Forest Area for 2026: 73809.36
Forecasted Forest Area for 2027: 74154.13
Forecasted Forest Area for 2030: 74525.78
Forecasted Forest Area for 2040: 74962.31
Forecasted Forest Area for 2050: 75431.69
```

```
[18] model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 5, 50)	10,400
lstm_1 (LSTM)	(None, 50)	20,200
dense (Dense)	(None, 1)	51

```
Total params: 91,955 (359.20 KB)
Trainable params: 30,651 (119.73 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 61,304 (239.47 KB)
```